

FLOSS: Fast Linear Online Secret-Shared Shuffling

Ian Chang

University of Washington

Sela Navot

University of Washington

Alex Ozdemir

*Max Planck Institute for
Security and Privacy,
Georgia Tech*

Nirvan Tyagi

University of Washington

Abstract. Randomly permuting secret data vectors is a core building block in many privacy-preserving protocols, including those for analytics, advertising, and communication. Existing approaches either rely on computation-heavy public key cryptography and zero-knowledge proofs or scale poorly for large vectors due to use of a quasilinear-sized permutation network. This work presents a preprocessing approach to enable fast linear-time online shuffles in the malicious-secure two-party computation (2PC) setting. We propose FLOSS, a 2PC protocol for securely computing any *interactive arithmetic permutation circuit*, a notion we introduce to capture how higher level protocols are built on secret-shared field arithmetic and permutations. We show how secret-shared sorting (a subprotocol in data analytics) can be described as an arithmetic permutation circuit, and can thus be compiled to an efficient online 2PC protocol using FLOSS. Our implementation and evaluation confirm FLOSS performs online shuffles fast: shuffling 2^{20} elements in under 500ms, greater than $800\times$ faster than state-of-the-art alternatives.

Contents

1	Introduction	3
2	Preliminaries	4
3	Technical Overview	4
3.1	Fast Online Shuffling via Preprocessing	4
3.2	Authenticated Shuffle Tuple Preprocessing	7
3.3	Building Applications on Shuffling	8
3.4	Additional Related Work	9
4	Preprocessing Protocol: Generating Authenticated Shuffle Tuples	9
5	Online Protocol: Applying Authenticated Shuffle Tuples	9
6	Application: Sorting	10
7	Evaluation	11
7.1	Implementation	11
7.2	Online Shuffling Costs	13
7.3	Offline Shuffling Costs	14
7.4	Sorting Costs	14
A	Universal Composability Security Model	19
B	Building Block Protocols	20
B.1	Commitment Functionality and Protocol	20
B.2	SPDZ Protocol for Arithmetic Circuits	20
C	Secret Guessing Attack using Blind Authentication Checks	21
D	Security Proof for Online Protocol	26
E	Full Description of Preprocessing Protocol for Authenticated Shuffle Tuples	33
E.1	Permutation Network	36
E.2	Malicious-Secure Permutation Network Construction: Forward Shuffle Tuples	37
E.3	Constructing Inverse Shuffle Tuples	39
E.4	Constructing κ -Authenticated Shuffle Tuples	40
E.5	Security	40
F	Alternate Circuit for Arbitrary Permutation Application	41

1 Introduction

Randomly permuting private data is a core building block in constructing privacy-preserving protocols for analytics [NWI⁺15, MRR20, AHI⁺22, GKLX22, ACD⁺24, BBM⁺25], advertising [CJK⁺23], communication [AKTZ17, LYK⁺19, EB22], whistleblowing [KLDF16, Lin17], random-access memory [WHC⁺14, FJKW15, FOSZ23], and more. A common approach taken for permuting private data is to use multi-party computation (MPC) of a *secret-shared shuffle*. A data vector $\vec{x}$ is secret-shared across parties so each party holds only a share of the data $[\![\vec{x}]\!]$. Following a secret-shared shuffle, parties hold fresh shares $[\![\pi(\vec{x})]\!]$ for a random permutation π of the data vector.

Classical secret-shared shuffling techniques are based on *permutation networks*. That is, they represent a permutation network as an arithmetic circuit and evaluate it over the data shares using an appropriate MPC protocol [BGW88]. Since a permutation network for n elements has size $O(n \log n)$, this approach has the disadvantage of incurring $O(n \log n)$ MPC multiplication operations when shuffling n elements.

Recently, Chase et al. reduced the online cost of shuffling (in the honest-but-curious setting) by introducing the idea of a preprocessed *shuffle tuple* that encodes the permutation π [CGP20]. Parties consume a shuffle tuple to perform a secret-shared shuffle with efficient linear computation and communication. Creating the shuffle tuple still requires computing a secret-shared shuffle, e.g., via a permutation network, but this work can be pushed to an offline preprocessing phase. Intuitively, shuffle tuples are analogous to *multiplication triples* [Bea91], a popular preprocessing approach enabling fast online computation of multiplication in MPC. These strategies for pushing work into offline preprocessing phases and enabling fast online computation provide necessary flexibility for higher-level applications to meet latency and efficiency goals in practice.

In this work, we lift the shuffle tuple approach of Chase et al. [CGP20] from the honest-but-curious setting to the two-party (2PC) dishonest-majority malicious setting. The 2PC setting is the target of ongoing standards processes for privacy-preserving analytics [GPP⁺25] and advertising [PLS⁺25]. In this standards setting, a service provider wishing to compute telemetry or advertising attribution statistics over users pairs with a second infrastructure provider to jointly compute the desired statistics without either party learning sensitive user data; for example, Divvi Up [div] is a non-profit organization that acts as a second infrastructure provider to services such as Mozilla Firefox. While many basic statistics can be computed via relatively simple aggregation MPC protocols [CB17], there is growing interest in supporting more complex statistics like heavy hitters [AHI⁺22] and attribution [CJK⁺23]. Our work is directly applicable to enabling such statistics (that rely on shuffling for efficiency) in these standards.

Prior work has attempted lifting shuffle tuples to the malicious setting, but achieving proper security has proved to be challenging. Initial proposed approaches to construct “authenticated” shuffle tuples [EB22, Lau21] are vulnerable to a selective failure attack in which a malicious party can learn information about the secret permutation, as noted by Song et al. [SYB⁺24]. Song et al. [SYB⁺24] propose a new approach using cut-and-choose techniques to defend against this attack. Yet, we show that their protocol is also vulnerable to a subtle, novel selective failure attack. We provide details of the attack and a working proof-of-concept exploit on the protocol implementation. While we conjecture that the fix to Song et al.’s protocol is straightforward, we opt to instead construct a new protocol, FLOSS, that avoids the use of expensive cut-and-choose entirely. In this way, FLOSS achieves the fastest online time for malicious 2PC secret-shared shuffling, shuffling a vector of size 2^{20} in 0.5 seconds, improving $3600\times$ over Song et al. [SYB⁺24]. Offline preprocessing costs also compare favorably to prior work, scaling well and incurring less preprocessing overhead at large vector sizes. Table 1 summarizes the online and offline costs in comparison to other shuffling strategies.

Equipped with our new shuffling protocol, we turn to higher-level applications that use shuffling. We propose a new circuit abstraction for MPC that we call an *arithmetic permutation circuit* that extends the standard arithmetic circuit of addition and multiplication gates over a field $\mathbb{F}$ with special permutation gates over vectors of field elements. We believe this abstraction is sufficient to describe most (if not all) existing applications built on top of secret-shared shuffling. We prove that our protocol FLOSS securely implements this abstraction in the malicious 2PC setting. To demonstrate the power of the arithmetic permutation circuit abstraction, we consider *secret-shared sorting* as a case study, a protocol built on shuffling [HKI⁺12, FO21, AHI⁺22] that is itself a building block for many analytics applications [AHI⁺22, CJK⁺23, ACD⁺24, BBM⁺25].

We adapt the popular radix-sort-and-shuffle protocol [BTW12, BLT14, CHI⁺19, AHI⁺22] to be written as an arithmetic permutation circuit. Most descriptions of the radix-sort-and-shuffle protocol [AHI⁺22] invoke both shuffles and inverse shuffles for the *same* permutation π . The protocol can be adapted to work without inverse shuffles but incurs an overhead of $\approx 2\times$ more permutation gates. To support this inverse shuffle optimization, which is useful for a number of applications, we extend the arithmetic permutation circuit abstraction with inverse permutation gates, and further, propose a new *correlated* shuffle tuple preprocessing algorithm that produces authenticated shuffle tuples for a permutation π along with the correlated authenticated shuffle tuples for the inverse permutation π^{-1} . All together, the benefits of the arithmetic permutation abstraction include (1) simpler implementation (≈ 140 LoC for secret-shared sorting), (2) simpler proofs of security that deal only with the high-level function-

ality, and (3) automatic compilation to a fast online protocol via FLOSS (2¹³ 32-bit inputs sorted in 3 seconds).

In summary, our contributions include:

- A new authenticated shuffle tuple protocol enabling fast online permutations in the malicious 2PC setting.
- A new arithmetic permutation circuit abstraction and formalism on which to build higher-level MPC protocols.
- A preprocessing protocol based on permutation networks for constructing correlated authenticated shuffle tuples for permutation π and inverse π^{-1} .
- A framework FLOSS tying all of the above together to compile any arithmetic permutation circuit into an efficient online malicious 2PC protocol. We implement, evaluate, and prove security of FLOSS.
- A case-study of applying FLOSS in building and evaluating an efficient online sorting protocol.
- A proof-of-concept selective failure attack against the Song et al. shuffle tuple protocol.

2 Preliminaries

Notation. Let λ denote the security parameter and $\mathbb{F}$ be a finite field with $|\mathbb{F}| \geq 2^\lambda$. We use $\leftarrow$ for assignment and $\leftarrow \$$ for randomized assignment. In particular, if S is a finite set, $x \leftarrow \$ S$ indicates sampling a random element from S and storing it in x . If Eval is a randomized algorithm $x \leftarrow \$ \text{Eval}(y)$ indicates storing the output of Eval, when ran with input y and uniformly random coins, in x . Similarly, $\Pi.P_0 \leftrightarrow \Pi.P_1$ denotes an interactive protocol Π between two parties, P_0 and P_1 . We use $\perp$ to indicate an invalid computation along with pseudocode keyword “Require” to return $\perp$ if a property does not hold.

We use $[x_i]_{i=1}^m$ to denote a vector of elements $[x_1, \dots, x_m]$, $[n]$ to denote $[1, \dots, n]$, and $[\cdot]$ to denote an empty vector. We use $\vec{1}_n \in \mathbb{F}^n$ to denote the all-ones vector $(1, \dots, 1)$ of length n , and $\vec{0}_n \in \mathbb{F}^n$ to denote the all-zeros vector $(0, \dots, 0)$ of length n . Define $\mathbb{S}_n$ to be the set of all permutation functions. We abuse notation to interpret $\pi \in \mathbb{S}_n$ both as a function that can be applied to a vector $\vec{x} \in \mathbb{F}^n$ to permute its elements $\pi(\vec{x})$ and as a function that can be applied to a specific index position in $[n]$ that outputs the index of the permuted vector that position is mapped to. Lastly, we encode π as a vector of permuted indices $\vec{\pi} = \pi([n])$.

For secret sharing, we use $\llbracket x \rrbracket_b$ to denote additive secret shares of x held by P_b , and $\langle x \rangle_b^{(\gamma)} = (\llbracket x \rrbracket_b, \llbracket \gamma x \rrbracket_b)$ to denote the pair of additive data share and γ -authenticated share of x ; see Appendix B.2 for more details on the secret-sharing scheme employed by SPDZ. When it is clear from context, we omit the superscript (γ) and party subscript b . We denote operations over secret shares such as $\langle x \rangle \cdot \langle y \rangle$ to implicitly indicate interactive evaluation of a secret-shared multiplication.

Security model. We prove the security of our 2PC protocols in the Universal Composability (UC) framework [Can01]. At a high level, security of an MPC protocol means that an adversary cannot learn more about the honest party’s input than the output of the function being evaluated, and an adversary cannot skew the output beyond choosing their input.

We consider a strong threat model in which the adversary controls the circuit being evaluated, potentially adaptively in response to intermediate outputs, and may deviate from the protocol arbitrarily. Specifically, we target the MPC-with-abort setting [GL05], where correctness and privacy are guaranteed but the adversary may cause the protocol to abort at any point—potentially after learning its own output but before the honest party receives theirs. In the two-party setting (where honest-majority cannot be assumed), stronger guarantees such as guaranteed output delivery or fairness are known to be impossible [Cle86]. We refer the reader to Appendix A for a more detailed description of our security model.

3 Technical Overview

We overview the main technical contributions of our work putting it in context with relevant related work.

3.1 Fast Online Shuffling via Preprocessing

The goal of a 2PC secret-shared shuffle is to shuffle additive secret shares of a data vector $\vec{x} \in \mathbb{F}^n$ where party P_0 holds shares $\llbracket \vec{x} \rrbracket_0$ and party P_1 holds shares $\llbracket \vec{x} \rrbracket_1$ (s.t., $\llbracket \vec{x} \rrbracket_0 + \llbracket \vec{x} \rrbracket_1 = \vec{x}$) with a random permutation $\pi \in \text{Perm}(n)$ unknown to both parties. We will focus on a simpler primitive, referred to as a *one-sided shuffle*, in which shares are shuffled according to a permutation known by one party (w.l.o.g. P_0). That is, P_0 holds π_0 and $\llbracket \vec{x} \rrbracket_0$ and P_1 holds $\llbracket \vec{x} \rrbracket_1$. The parties learn fresh random shares $\llbracket \pi_0(\vec{x}) \rrbracket_0$ and $\llbracket \pi_0(\vec{x}) \rrbracket_1$, respectively, and P_1 learns nothing about π_0 . Two one-sided shuffles composed in sequence realize a full secret-shared shuffle: P_0 first shuffles with random π_0 , then P_1 shuffles with random π_1 to output $\llbracket \pi(\vec{x}) \rrbracket$ for $\pi = \pi_1 \circ \pi_0$.

Shuffle tuples in the honest-but-curious model. Our starting point is the work of Chase et al. who proposed a novel preprocessing technique for generating *shuffle tuples* encoding a permutation π held by party P_0 [CGP20, PRRS25]; these shuffle tuples can be consumed to permute secret shares efficiently. A preprocessed shuffle tuple for permutation π is constructed from a random vector $\vec{a} \in \mathbb{F}^n$ selected by P_1 and an arbitrary permutation π held by P_0 . The shuffle tuple consists of fresh shares of $\vec{a} = \pi(\vec{a})$, such that P_0 holds π and $\llbracket \vec{a} \rrbracket_0$ and P_1 holds $\vec{a}$ and $\llbracket \vec{a} \rrbracket_1$.

Constructing the shuffle tuple still requires permuting secret shares of $\vec{a}$ during preprocessing; we discuss preprocessing costs shortly. However, the online cost of applying the shuffle tuple to permute during protocol execution is low.

Protocol	Offline costs		Online costs	
MPC-Enc+Shuf [MSS14]	$O(\lambda n) \mathbb{E}$	–	$O(\lambda n) \mathbb{E}$	–
Linear-Only LHE [DNO25]	$O(n)$	–	$O(\lambda n) \mathbb{L}$	–
SimpleNet [Wak68]	$O(n \log n)$	532s	$O(n \log n)$	491s
PermNet [MSS14]	$O(n \log n)$	2436s	$O(n \log n)$	407s
OPM [SYB ⁺ 24]	$O(nT \log \frac{n}{T})$	6762s	$O(n \log \frac{n}{T})$	1835s
FLOSS (this work)	$O(n \log n)$	1270s	$O(n)$	0.5s

Table 1: Summary of costs for malicious-secure 2PC shuffling protocols for a vector of length n . Asymptotic costs for computation are provided and, for practically instantiable constructions, a concrete running time for shuffling a vector of length 2^{20} is included. Symbols $\mathbb{E}$ and $\mathbb{L}$ denote elliptic curve operation and linearly-homomorphic encryption addition, respectively; unspecified costs correspond to arithmetic over 128-bit fields.

$P_0.\text{BlindAuth}(\llbracket \gamma \rrbracket_0, \langle \alpha \rangle_0, [\langle z_i \rangle_0]_i^n)$ $\leftrightarrow P_1.\text{BlindAuth}(\llbracket \gamma \rrbracket_1, \langle \alpha \rangle_1, [\langle z_i \rangle_1]_i^n)$
1. P_0 and P_1 compute linear combination of inputs $(\llbracket z_i \rrbracket, \llbracket \gamma z_i \rrbracket) \leftarrow \langle z_i \rangle$ blinded by random $(\llbracket \alpha \rrbracket, \llbracket \gamma \alpha \rrbracket) \leftarrow \langle \alpha \rangle$: (a) P_0 and P_1 each sample $r_b \leftarrow \mathbb{F}$. (b) P_0 and P_1 each commit to r_b by invoking $F_{\text{Comm}}.\text{Comm}$. (c) P_0 and P_1 open commitments to receive r_{1-b} from each other via $F_{\text{Comm}}.\text{Open}$. (d) P_0 and P_1 compute $r \leftarrow r_0 + r_1$. (e) P_0 and P_1 compute linear combination: $\llbracket \ell \rrbracket \leftarrow \llbracket \alpha \rrbracket + \sum_{i=1}^n r^{i-1} \cdot \llbracket z_i \rrbracket, \quad \llbracket \gamma \ell \rrbracket \leftarrow \llbracket \gamma \alpha \rrbracket + \sum_{i=1}^n r^{i-1} \cdot \llbracket \gamma z_i \rrbracket$
2. P_0 and P_1 check $\llbracket \gamma \ell \rrbracket$ against $\llbracket \ell \rrbracket$: (a) P_0 and P_1 open ℓ by sending $\llbracket \ell \rrbracket$ to each other. (b) P_0 and P_1 compute $\llbracket \epsilon \rrbracket \leftarrow \llbracket \gamma \ell \rrbracket - \llbracket \gamma \rrbracket \cdot \ell$. (c) P_0 and P_1 each commit to $\llbracket \epsilon \rrbracket_b$ by invoking $F_{\text{Comm}}.\text{Comm}$. (d) P_0 and P_1 open commitments to receive $\llbracket \epsilon \rrbracket_{1-b}$ from each other via $F_{\text{Comm}}.\text{Open}$. (e) P_0 and P_1 output abort if: $\llbracket \epsilon \rrbracket_0 + \llbracket \epsilon \rrbracket_1 \neq 0$

Figure 1: The blind authentication check subprotocol used in FLOSS for checking a vector $\vec{z}$ of unopened data shares against authentication shares. The protocol also takes as input shares of a random blinding factor α .

To shuffle $\llbracket \vec{x} \rrbracket$, P_1 uses $\vec{a}$ to blind their shares, $\vec{c} \leftarrow \llbracket \vec{x} \rrbracket_1 - \vec{a}$, and sends blinded shares $\vec{c}$ to P_0 . Party P_1 simply sets their new shares for $\llbracket \pi(\vec{x}) \rrbracket_1 \leftarrow \llbracket \vec{a} \rrbracket_1$. Party P_0 applies permutation π to set their new shares as $\llbracket \pi(\vec{x}) \rrbracket_0 \leftarrow \pi(\llbracket \vec{x} \rrbracket_0) + \pi(\vec{c}) + \llbracket \vec{a} \rrbracket_0$. The resulting shares are indeed additive shares of $\pi(\vec{x})$:

$$\begin{aligned}
\llbracket \pi(\vec{x}) \rrbracket_0 + \llbracket \pi(\vec{x}) \rrbracket_1 &= \left(\pi(\llbracket \vec{x} \rrbracket_0) + \pi(\vec{c}) + \llbracket \vec{a} \rrbracket_0 \right) + \llbracket \vec{a} \rrbracket_1 \\
&= \pi(\llbracket \vec{x} \rrbracket_0) + \pi(\llbracket \vec{x} \rrbracket_1 - \vec{a}) + \llbracket \pi(\vec{a}) \rrbracket_0 + \llbracket \pi(\vec{a}) \rrbracket_1 \\
&= \pi(\llbracket \vec{x} \rrbracket_0) + \pi(\llbracket \vec{x} \rrbracket_1) - \pi(\vec{a}) + \pi(\vec{a}) = \pi(\vec{x})
\end{aligned}$$

Overall, a one-sided shuffle is done with one round of communication of n field elements and $O(n)$ field additions by each party.

Pitfalls in lifting to malicious security model. A number of works have attempted to lift the notion of preprocessed shuffle

tuples to the malicious security setting with active adversaries that may deviate from the protocol.

A natural approach is to build on the authenticated secret sharing strategy of the SPDZ malicious-secure protocol for MPC of arithmetic circuits [DPSZ12]. In SPDZ, parties hold secret shares of data elements $x \in \mathbb{F}$, where $\mathbb{F}$ is a large field (exponential in the security parameter). Additionally, parties hold additive shares of an authentication key $\gamma \in \mathbb{F}$ where $\gamma = \llbracket \gamma \rrbracket_0 + \llbracket \gamma \rrbracket_1$, and data shares $\llbracket x \rrbracket$ are paired with *authenticated shares* $\llbracket \gamma x \rrbracket$, which we denote together as $\langle x \rangle \leftarrow (\llbracket x \rrbracket, \llbracket \gamma x \rrbracket)$. For completeness, we include a description of our treatment of the SPDZ protocol in Appendix B.2.

Consider now a pair of shuffle tuples over the same permutation π held by P_0 but for different random vectors selected by P_1 , $\vec{a}$ and $\vec{b}$. Party P_0 holds $(\pi, \llbracket \vec{a} \rrbracket_0, \llbracket \vec{b} \rrbracket_0)$ and P_1 holds $(\vec{a}, \vec{b}, \llbracket \vec{a} \rrbracket_1, \llbracket \vec{b} \rrbracket_1)$. To shuffle $\langle \vec{x} \rangle = (\llbracket \vec{x} \rrbracket, \llbracket \gamma \vec{x} \rrbracket)$, parties undergo the same procedure as above, using $\vec{a}$ to shuffle the data shares and retrieve $\llbracket \pi(\vec{x}) \rrbracket$ and using $\vec{b}$ to shuffle the authenticated shares and retrieve $\llbracket \pi(\gamma \vec{x}) \rrbracket$ which together form $\langle \pi(\vec{x}) \rangle$. Of course, adversaries may deviate from the protocol and the resulting vectors may not actually have this form. One strategy in prior work [EB22, Lau21] is to perform a *blind authentication check* on the claimed permuted shares (e.g., such as the one presented in Figure 1). A blind authentication check checks that the authenticated shares are indeed valid with respect to the data shares and secret authentication key γ without revealing the values underlying the shares.

Pitfall 1: Permutation guessing selective failure attack. Unfortunately, this blind authentication check on its own does not prevent malicious behavior. Song et al. [SYB⁺24] describe a selective failure attack that allows a malicious P_1 to guess partial mappings of the secret permutation π used by P_0 ¹. We detail the attack, as it provides intuition for our final construction.

Consider party P_1 who guesses index i maps to index j in permutation π . Party P_1 additively perturbs their share at index i : $\llbracket x'_i \rrbracket_1 \leftarrow \llbracket x_i \rrbracket_1 + \delta$. The parties execute the above

¹The general attack strategy had also been previously described by Araki et al. [AFO⁺21].

shuffling procedure using $\vec{a}$ to shuffle data shares $\vec{x}'$ (with perturbed x'_i at index i) and $\vec{b}$ to shuffle authenticated data shares $\gamma\vec{x}$ (unmodified by P_1). Following this, parties receive shares $\llbracket\pi(\vec{x}')\rrbracket$ and $\llbracket\pi(\gamma\vec{x})\rrbracket$. Prior to the authentication check, P_1 unperturbs index j of the permuted data shares: $\llbracket\pi(\vec{x}')_j\rrbracket_1 \leftarrow \llbracket\pi(\vec{x}')_j\rrbracket_1 - \delta$. Party P_1 proceeds with the blind authentication check using $\llbracket\pi(\vec{x}')_j\rrbracket$ (perturbed index i pre-shuffle and unperturbed index j post-shuffle) and $\llbracket\pi(\gamma\vec{x})\rrbracket$ (unmodified). With these perturbations, authentication succeeds if and only if π maps index i to j (except with a small probability), which allows P_1 to learn about π .

Song et al. propose a new approach to address this selective failure attack via repeated shuffling and cut-and-choose. First, they observe that for a *random* permutation, the chance that this attack succeeds is not too high—it is just $\tilde{O}(n^{-1})$. So, they reduce the failure probability by applying B shuffle tuples in sequence, with an authentication check after each one. Second, they modify the honest-but-curious oblivious permutation matrix (OPM) preprocessing algorithm of Chase et al. to produce shuffle tuples quickly but such that they may be “leaky.” Thus, to prevent leakage from “leaky” shuffle tuples, they introduce a cut-and-choose selection process in which a large number ($\gg B$) of shuffle tuples are preprocessed, and parties randomly select B tuples to apply to perform a shuffle (referred to as OPM in Table 1). A side effect of this technique is that it can only be used to apply random permutations which will limit flexibility down the line.

Looking forward, we take a different approach. We prevent the permutation-guessing attack via a new explicit authentication mechanism. This avoids the high concrete costs of repeated shuffling and cut-and-choose and enables greater flexibility in permutation selection.

Pitfall 2: Input guessing selective failure attack. We discover that without careful usage, the blind authentication check (see Figure 1) as used in prior shuffling protocols [Lau21, EB22, SYB⁺24] can leak information about the secrets.

Suppose a malicious party, say P_0 , holding shares $\llbracket\vec{x}\rrbracket_0$, $\llbracket\gamma\vec{x}\rrbracket_0$, and $\llbracket\gamma\rrbracket_0$ wants to guess the value of $\vec{x}$. To illustrate the attack, first consider if P_0 has access to an oracle for blind authentication checks in which they may input malformed shares. Specifically, the oracle takes $(\llbracket\vec{x}\rrbracket_0, \llbracket\gamma\vec{x}\rrbracket_0, \llbracket\gamma\rrbracket_0)$ from P_0 and $(\llbracket\vec{x}\rrbracket_1, \llbracket\gamma\vec{x}\rrbracket_1, \llbracket\gamma\rrbracket_1)$ from honest P_1 . Then, it verifies that $(\llbracket\gamma\rrbracket_0 + \llbracket\gamma\rrbracket_1) \cdot (\llbracket\vec{x}\rrbracket_0 + \llbracket\vec{x}\rrbracket_1) = \llbracket\gamma\vec{x}\rrbracket_0 + \llbracket\gamma\vec{x}\rrbracket_1$, returning whether equality holds. To guess whether $\vec{x} = \vec{x}'$, P_0 will instead input additively-modified shares $(\llbracket\vec{x}\rrbracket_0, \llbracket\gamma\vec{x}\rrbracket_0 + \vec{x}', \llbracket\gamma\rrbracket_0 + 1)$ into the oracle. The oracle checks:

$$\begin{aligned} (\llbracket\gamma\rrbracket_0 + 1 + \llbracket\gamma\rrbracket_1) \cdot (\llbracket\vec{x}\rrbracket_0 + \llbracket\vec{x}\rrbracket_1) &\stackrel{?}{=} \llbracket\gamma\vec{x}\rrbracket_0 + \vec{x}' + \llbracket\gamma\vec{x}\rrbracket_1 \\ (\gamma + 1) \cdot \vec{x} &\stackrel{?}{=} \gamma\vec{x} + \vec{x}'; \end{aligned}$$

which holds if and only if $\vec{x} = \vec{x}'$. Thus, the oracle’s response reveals whether the guess was correct.

In practice, the blind authentication check protocol (shown in Figure 1) does not provide this oracle functionality, so the

above attack is not directly applicable. Namely, a malicious P_0 cannot simply run the protocol with a perturbed key share $\llbracket\gamma\rrbracket_0 + 1$ due to the use of preprocessed material in SPDZ, e.g., multiplication triples, that encode γ . This surfaces in the protocol in Figure 1 via the construction of the randomized blind $\llbracket\alpha\rrbracket, \llbracket\gamma\alpha\rrbracket$.

However, we observe that most implementations of SPDZ-like protocols allow for ongoing preprocessing phases to be run as needed (e.g., when multiplication triples run out), and rerunning the preprocessing does not verify that the produced preprocessing material was generated with the *same authentication key* [mpsa]. In this case, a malicious P_0 may run a preprocessing phase with perturbed authentication key $\llbracket\gamma\rrbracket_0 + 1$, generate perturbed multiplication triples, and use those to instantiate the above oracle for an input-guessing attack. We implement and demonstrate this attack against the prototype implementation of Song et al. [SYB⁺24] which allows for computation of multiplication triples on-the-fly. As far as we know, generating perturbed preprocessing material is not a security issue in SPDZ, since authentication checks are only performed on opened values. This selective failure attack only arises with the introduction of blind authentication checks.

A natural mitigation against this attack is to restrict generation of preprocessing material to a one time step at the beginning of the protocol, which enforces that all preprocessing is done with respect to a single authentication key. We explicitly enforce this in our protocol description and implementation. However, enforcing this can limit design choices, especially in cases where the amount of preprocessing material is not known in advance or is too large to generate in one shot. Alternatively, we propose a lightweight modification to incorporate an additional layer of authentication, essentially an additional multiplicative MAC β to ensure the integrity of γ , which prevents the attack without limiting on-demand preprocessing.

We describe the attack details in Appendix C and further discuss mitigations.

Our approach: Shuffle tuples in the malicious model. A key characteristic of the permutation-guessing selective failure attack is the malicious party P_1 ’s use of additive perturbation. A natural strategy for preventing attacks via additive perturbation is using secret shares for an unknown multiplicative factor. This is the strategy that SPDZ itself relies on with authenticated shares $\llbracket\gamma x\rrbracket$ for secret multiplicative authentication key $\llbracket\gamma\rrbracket$.

Consider the following extension to the shuffle tuple construction from before. Party P_1 again selects a random vector $\vec{a} \leftarrow \mathbb{F}^n$. Party P_0 holds an arbitrary permutation π but also samples a random multiplicative authentication key $\kappa \leftarrow \mathbb{F}$. The “ κ -authenticated” shuffle tuple consists of fresh shares of $\vec{a} = \pi(\vec{a})$ as before along with fresh shares of $\kappa\vec{a} = \kappa \cdot \pi(\vec{a})$. All together, P_0 holds $(\pi, \kappa, \llbracket\vec{a}\rrbracket_0, \llbracket\kappa\vec{a}\rrbracket_0)$ and P_1 holds $(\vec{a}, \llbracket\vec{a}\rrbracket_1, \llbracket\kappa\vec{a}\rrbracket_1)$.

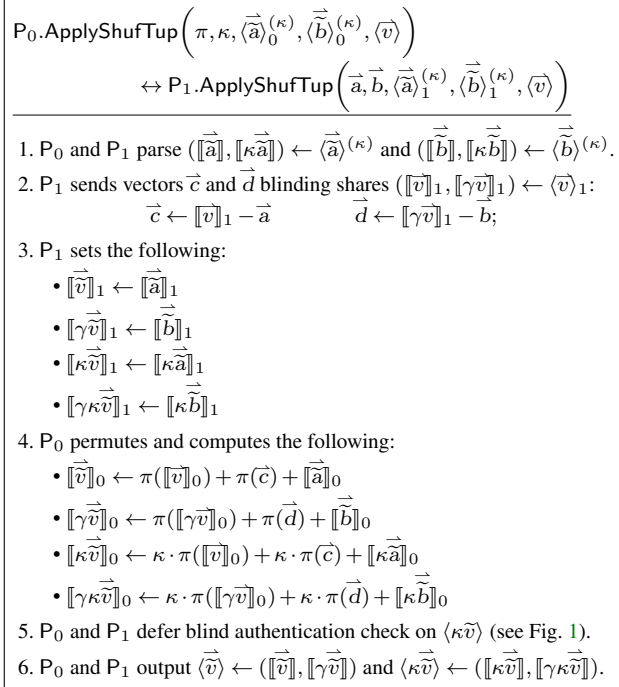


Figure 2: Fast online application of preprocessed authenticated shuffle tuple to permute secret shares in FLOSS.

Consider first using this shuffle tuple to shuffle the data share $\llbracket \vec{x} \rrbracket$ of $(\llbracket \vec{x} \rrbracket, \llbracket \gamma \vec{x} \rrbracket) \leftarrow \langle \vec{x} \rangle$. We will handle the authenticated shares $\llbracket \gamma \vec{x} \rrbracket$ separately. As before, P_1 sends blinded shares $\vec{c} \leftarrow \llbracket \vec{x} \rrbracket_1 - \vec{a}$ to P_0 . Party P_1 sets new shares:

$$\llbracket \pi(\vec{x}) \rrbracket_1 \leftarrow \llbracket \vec{a} \rrbracket_1, \quad \llbracket \kappa \cdot \pi(\vec{x}) \rrbracket_1 \leftarrow \llbracket \kappa \vec{a} \rrbracket_1$$

Party P_0 uses π and κ to compute its new shares:

$$\llbracket \pi(\vec{x}) \rrbracket_0 \leftarrow \pi(\llbracket \vec{x} \rrbracket_0) + \pi(\vec{c}) + \llbracket \vec{a} \rrbracket_0$$

$$\llbracket \kappa \cdot \pi(\vec{x}) \rrbracket_0 \leftarrow \kappa \cdot \pi(\llbracket \vec{x} \rrbracket_0) + \kappa \cdot \pi(\vec{c}) + \llbracket \kappa \vec{a} \rrbracket_0$$

We can verify that P_0 's use of κ indeed results in valid shares of $\kappa \cdot \pi(\vec{x})$:

$$\llbracket \kappa \cdot \pi(\vec{x}) \rrbracket_0 + \llbracket \kappa \cdot \pi(\vec{x}) \rrbracket_1$$

$$= \kappa \cdot \pi(\llbracket \vec{x} \rrbracket_0) + \kappa \cdot \pi(\vec{c}) + \llbracket \kappa \vec{a} \rrbracket_0 + \llbracket \kappa \vec{a} \rrbracket_1$$

$$= \kappa \cdot \pi(\llbracket \vec{x} \rrbracket_0) + \kappa \cdot \pi(\llbracket \vec{x} \rrbracket_1) - \kappa \cdot \pi(\vec{a}) + \pi(\kappa \cdot \vec{a}) = \kappa \cdot \pi(\vec{x})$$

In this manner, P_0 and P_1 obtain shares $\llbracket \kappa \cdot \pi(\vec{x}) \rrbracket$. If P_0 and P_1 have another κ -authenticated shuffle tuple for $\vec{b}$, they perform the same process on the authenticated shares $\llbracket \gamma \vec{x} \rrbracket$ to obtain shares $\llbracket \gamma \cdot \pi(\vec{x}) \rrbracket$ and $\llbracket \gamma \kappa \cdot \pi(\vec{x}) \rrbracket$.

The key insight is now P_0 and P_1 verify correct application of the permutation by performing the blind authentication check of γ on the κ -shares $\langle \kappa \cdot \pi(\vec{x}) \rangle \leftarrow (\llbracket \kappa \cdot \pi(\vec{x}) \rrbracket, \llbracket \gamma \kappa \cdot \pi(\vec{x}) \rrbracket)$ instead of on $\langle \pi(\vec{x}) \rangle$. Let us provide some intuition for why this eliminates the permutation-guessing attack. Recall when P_1 mounts the attack guessing index i maps to index j , P_1 perturbs $\llbracket x'_i \rrbracket_1 \leftarrow \llbracket x_i \rrbracket_1 + \delta$ and later attempts to unperturb $\llbracket \pi(\vec{x}') \rrbracket_1 \leftarrow \llbracket \pi(\vec{x}') \rrbracket_1 - \delta$ prior to the blind authentication check over $\langle \pi(\vec{x}') \rangle$. Now since the blind authentication check is performed over $\langle \kappa \cdot \pi(\vec{x}) \rangle$, to correct the original perturbation of δ , P_1 must unperturb with $\delta\kappa$ for the attack to succeed;

P_1 cannot guess $\delta\kappa$ since κ is random and unknown.

Overall, the application of the κ -authenticated shuffle tuple is simple and efficient, avoiding the additional costs of repeated application of shuffle tuples of Song et al. [SYB⁺24]. Next we discuss strategies for constructing κ -authenticated shuffle tuples during preprocessing.

3.2 Authenticated Shuffle Tuple Preprocessing

κ -authenticated shuffle tuples enable efficient online shuffling. However, this online efficiency comes at the cost of a more expensive preprocessing protocol. Creation of authenticated shuffle tuples can be bootstrapped from any malicious-secure shuffling protocol that operates over SPDZ authenticated shares (which includes all that we are aware of). For example, consider the generation of the κ -authenticated shuffle tuple for vector $\vec{a}$ selected by P_1 . Party P_1 inputs $\vec{a}$ to create shares $\langle \vec{a} \rangle \leftarrow (\llbracket \vec{a} \rrbracket, \llbracket \gamma \vec{a} \rrbracket)$. Parties use an existing malicious-secure shuffling protocol to get shares of $\langle \vec{a} \rangle = \langle \pi(\vec{a}) \rangle$ for π held by P_0 . To add κ authentication, P_0 inputs κ to share $\langle \kappa \rangle$, and parties perform authenticated multiplication, e.g., via multiplication triples, to compute shares $\langle \kappa \vec{a} \rangle$. As a final step, parties verify all computed shares $\llbracket \vec{a} \rrbracket$ and $\llbracket \kappa \vec{a} \rrbracket$ were computed correctly by performing a blind authentication check on $\langle \vec{a} \rangle$ and $\langle \kappa \vec{a} \rangle$, before discarding the γ -authenticated shares.² The protocol is repeated for random vector $\vec{b}$ using the same κ and π .

There are two high level strategies taken in prior work for shuffling (summarized in Table 1): (1) Homomorphic encryption schemes (with proofs of knowledge) and (2) secure computation of a permutation network. Mohassel et al. and Dittmer et al. both propose linear cost protocols that rely on encryption schemes with homomorphic properties. Mohassel et al. [MSS14] use MPC to encode and encrypt secret data into El Gamal elliptic curve ciphertexts [SA19] and then have the permuting party verifiably shuffle the ciphertexts using a proof of knowledge [Nef01, Gro09, Gro10, BG12]. Dittmer et al. [DNO25] propose a protocol using a linearly-homomorphic encryption (LHE) scheme to encrypt and randomize shares. Their protocol is secure if the LHE scheme satisfies a “linear-only” property—that is, only linear operations can be performed over ciphertexts—and supports efficient ciphertext validity checks. However, it is unclear how to instantiate an LHE scheme with these properties over fields. Instead, one can extend a candidate LHE scheme with zero-knowledge proofs of knowledge to restrict to linear-only operations or to prove ciphertext validity. Ultimately, protocols taking the homomorphic encryption approach are concretely expensive and have not yet been implemented.

²The authentication key γ used for the preprocessing is necessary to prevent malicious behavior during preprocessing but it is not part of the produced κ -authenticated shuffle tuple. It may be different from the authentication key used in the downstream FLOSS protocol.

$P_0.\text{ApplyPerm}(\langle \rho \rangle_0 \in \mathbb{F}^n, \langle x \rangle_0 \in \mathbb{F}^n)$ $\leftrightarrow P_1.\text{ApplyPerm}(\langle \rho \rangle_1 \in \mathbb{F}^n, \langle x \rangle_1 \in \mathbb{F}^n)$ Input: Shares of permutation $\rho \in \mathbb{S}_n$ as $\langle \rho([n]) \rangle$, shares of input $\langle x \rangle$. Output: Shares of permutation applied to inputs $\langle \rho(x) \rangle$. Assume: id_{π_b} identifies a random permutation $\pi_b \leftarrow \mathbb{S}_n$ for P_b. 1. $\langle \pi_0(\rho) \rangle \leftarrow \text{Permute}(\text{id}_{\pi_0}, \langle \rho \rangle)$ 2. $\langle \pi_1(\pi_0(\rho)) \rangle \leftarrow \text{Permute}(\text{id}_{\pi_1}, \langle \pi_0(\rho) \rangle)$ 3. $\pi_1(\pi_0(\rho)) \leftarrow \text{Reconstruct}(\langle \pi_1(\pi_0(\rho)) \rangle)$ 4. Parse $\pi_1(\pi_0(\rho))$ as $(\pi_1 \circ \pi_0 \circ \rho) \in \mathbb{S}_n$ 5. Locally permute shares with public reconstructed permutation: $\langle (\pi_1 \circ \pi_0 \circ \rho)(x) \rangle \leftarrow (\pi_1 \circ \pi_0 \circ \rho)(\langle x \rangle)$ 6. $\langle (\pi_0 \circ \rho)(x) \rangle \leftarrow \text{PermuteInv}(\text{id}_{\pi_1}, \langle (\pi_1 \circ \pi_0 \circ \rho)(x) \rangle)$ 7. $\langle \rho(x) \rangle \leftarrow \text{PermuteInv}(\text{id}_{\pi_0}, \langle (\pi_0 \circ \rho)(x) \rangle)$ 8. Output $\langle \rho(x) \rangle$

Figure 3: A description of an arithmetic permutation circuit for applying an arbitrary secret-shared permutation ρ to a vector of data $\vec{x}$. This is a useful subcircuit in many protocols including sorting.

Alternatively, securely computing a permutation network [Wak68] incurs $O(n \log n)$ costs for shuffling n items due to the number of switches in the permutation network. This approach can be instantiated efficiently in practice. To achieve a constant-round protocol, Mohassel et al. [MS13, MSS14] introduce a permutation-encoding technique that avoids running the permutation network as a $(\log n)$ -depth circuit. Song et al. [SYB+24], as discussed above, integrate higher degree OPM switches into the permutation network; then, they must handle imperfect OPM usage with a cut-and-choose amplification step.

Therefore, the two permutation network approaches are the most practical candidates for preprocessing shuffle tuples. However, looking forward to the next section, we explore shuffling in higher level protocols—we find that it will be useful to apply the same permutation repeatedly or apply the correlated inverse permutation of a previously applied permutation. Creating shuffle tuples with these correlated permutations using the circuit-description permutation network approach [MSS14] is simple, as the control bits encoding the permutation network can be reused across multiple runs (and used in reverse). On the other hand, Song et al.’s leaky OPM approach requires a random cut-and-choose selection which means that the permutation is random and will not correlate with prior permutations. Therefore, we construct our preprocessing protocol for authenticated shuffle tuples from circuit-description permutation networks.

3.3 Building Applications on Shuffling

Our motivation for studying shuffling is the vast set of higher level protocols that use shuffling as a building block. However, shuffling breaks the common arithmetic circuit interface (i.e., secure computation of addition and multiplication) that many MPC protocols (e.g., SPDZ [DPSZ12]) target. Thus,

we introduce an *arithmetic permutation circuit* interface that explicitly includes (one-sided) permutations as a supported operation. We formalize interactive arithmetic permutation circuits as an ideal functionality in the universal composability (UC) model (see Figure 16) and prove FLOSS secure with respect to the functionality.

We present *secret-shared sorting* as a case study to demonstrate the utility of our new abstraction. Sorting is a building block used to construct higher level protocols for data analytics including filtering and joins [MRR20, BDG+22, CJK+23] and histogram computation [ACD+24, AHI+22]. We demonstrate how to express the efficient radix-sort-and-shuffle protocol of Asharov et al. [AHI+22] as an arithmetic permutation circuit.

First, consider a subprotocol for interpreting a secret-shared vector as an arbitrary permutation ρ and applying that permutation to secret shares of a data vector $\vec{x}$. This subprotocol is used in the radix sort algorithm and is commonly used in other algorithms as well. The radix sort algorithm produces shares of a permutation that represents the stable sorting for a single bit position. Then to compute the final sorting permutation, the algorithm applies each bit-position’s stable sorting permutation in sequence. Observe that this task differs from what our preprocessed shuffle tuples accomplish. First, this permutation is unknown to the parties at the preprocessing stage, and second, the permutation is encoded as secret shares.

Asharov et al. [AHI+22] propose a neat approach to instantiating this subprotocol using random permutations (that we observe can be preprocessed) (see Figure 3). Parties “blind” the permutation ρ by applying random permutations π_0 and π_1 held by each party, respectively, and then reconstruct the blinded permutation $\pi_1 \circ \pi_0 \circ \rho$. Parties apply the reconstructed permutation locally to their shares of $\vec{x}$. Then, they remove the blinding permutations by sequentially applying inverse permutations π_1^{-1} and π_0^{-1} . Naively, this does not fit our model: parties can apply random preprocessed permutations π_0 and π_1 but it is unclear how to apply the inverse permutations π_1^{-1} and π_0^{-1} . One solution is that we can construct an alternate protocol that avoids the use of inverse permutations, but this approach incurs eight applications of forward permutations (see Appendix F). If we incorporate inverse permutations directly into our preprocessing and the arithmetic permutation circuit abstraction, we reduce the overhead by $2\times$. In particular, the Mohassel et al. [MSS14] permutation network approach (or any circuit-description permutation network) can be easily adapted to compute correlated inverse shuffle tuples by reusing the permutation network control bits and running the permutation network circuit in reverse.

Our arithmetic permutation circuit functionality allows parties to input permutations during the preprocessing stage by specifying the number of forward and inverse shuffle tuples to generate. Inverse permutations enable optimized descriptions of higher level applications in the arithmetization. In Section 6, we complete the description of the radix-sort-and-

shuffle algorithm as an arithmetic permutation circuit.

3.4 Additional Related Work

Gao et al. [GZZ24] is the most closely related work, as they construct authenticated shuffle tuples in the malicious-secure MPC setting using the same high-level strategy. We highlight some key differences. Gao et al. focus on extensibility and generalization, whereas we focus on practicality. They present their protocol as generic to the underlying authenticated secret-sharing scheme and extend it to greater than two parties; further, no implementation nor evaluation are provided. In contrast, we focus on the 2PC setting and build our shuffle tuples to be compatible with the popular SPDZ secret-sharing and MPC protocols [DPSZ12, Kel20, KOS16, KPR18], optimizing the design for these settings. For example, as in SPDZ for application of multiplication triples, we show how to defer and batch together expensive authentication checks until share reconstruction.

Separately, ORQ [BBM⁺25] is a recent system targeting MPC private analytics implementing protocols for database queries and complex joins. ORQ similarly observes using correlated inverse shuffle tuples as an optimization. However, ORQ makes use of shuffle tuples in the honest-but-curious dishonest majority setting, whereas we work in the malicious 2PC dishonest majority setting. The protocols of ORQ may be written in the arithmetic permutation circuit abstraction, and FLOSS can be used to lift their work to the malicious-secure 2PC setting.

4 Preprocessing Protocol: Generating Authenticated Shuffle Tuples

An authenticated shuffle tuple for permutation π chosen by party P_0 consists of the following values for P_0 (left) and P_1 (right) where P_0 chooses random $\kappa \leftarrow \mathbb{F}$ and P_1 chooses random vectors $\vec{a}, \vec{b} \leftarrow \mathbb{F}^n$:

$$\left(\pi, \kappa, \langle \pi(\vec{a}) \rangle_0^{(\kappa)}, \langle \pi(\vec{b}) \rangle_0^{(\kappa)} \right), \left(\vec{a}, \vec{b}, \langle \pi(\vec{a}) \rangle_1^{(\kappa)}, \langle \pi(\vec{b}) \rangle_1^{(\kappa)} \right).$$

Each party selects permutations for which to generate shuffle tuples. We employ a permutation network approach to generate authenticated shuffle tuples of the above form. W.l.o.g. given π and κ from P_0 and $\vec{a}$ and $\vec{b}$ from P_1 , consider a circuit description of a permutation network that takes $\vec{a}$ as input from P_1 and control bits for the network encoding π from P_0 . Using the Mohassel et al. [MSS14] construction for malicious-secure permutation networks which itself builds on the SPDZ malicious-secure arithmetic circuit computation protocol [DPSZ12], the parties can securely compute authenticated shares of $\langle \pi(\vec{a}) \rangle^{(\gamma)}$ with respect to some authentication key γ used by the SPDZ protocol.

Further, for each permutation π , parties may also specify the number of shuffle tuples to generate for π and the number of shuffle tuples to generate for the inverse permutation π^{-1} . Parties reapply the same permutation to $\vec{b}$ or to fresh $(\vec{a}', \vec{b}')$

values to generate additional shuffle tuples for π . They simply reuse the inputted authenticated control bits to rerun the permutation network circuit. Similarly, parties will reuse the same control bits and reverse traversal order to run an inverse permutation network circuit. This generates shares of inverse permutations $\langle \pi^{-1}(\vec{a}) \rangle^{(\gamma)}$.

To complete the generation of the authenticated shuffle tuples, P_0 inputs the authentication key κ for the tuple and parties run secret-shared multiplication of the corresponding $(\langle \pi(\vec{a}) \rangle^{(\gamma)}, \langle \pi(\vec{b}) \rangle^{(\gamma)})$ with $\langle \kappa \rangle$ to obtain the authenticated shares $(\langle \kappa \pi(\vec{a}) \rangle^{(\gamma)}, \langle \kappa \pi(\vec{b}) \rangle^{(\gamma)})$. Finally, the parties perform a blind authentication check with $\llbracket \gamma \rrbracket$ over the produced vectors $(\langle \pi(\vec{a}) \rangle^{(\gamma)}, \langle \pi(\vec{b}) \rangle^{(\gamma)}, \langle \kappa \pi(\vec{a}) \rangle^{(\gamma)}, \langle \kappa \pi(\vec{b}) \rangle^{(\gamma)})$. If the check passes, parties discard the γ -authenticated shares and output $\langle \pi(\vec{a}) \rangle^{(\kappa)}, \langle \pi(\vec{b}) \rangle^{(\kappa)}$.

We defer the full construction description—including the description of a permutation network as a circuit—and security argument of our authenticated shuffle tuple generation protocol to Appendix E. We provide the following corollary from the security proof of the original permutation network protocol of Mohassel et al. [MSS14], on which our construction is based:

Corollary 1 (Informal). *The protocol $\text{PermNetSTPrep}[\mathbb{F}]$ (Figure 22) UC-realizes the ideal functionality $F_{\text{ShuffleTuplePrep}}$ (Figure 18) in the $(F_{\text{SPDZPrep}}, F_{\text{Comm}})$ -hybrid model.*

Sketch: Aside from the reuse of control bits across multiple runs of the permutation network circuit, our protocol departs from Mohassel et al. only at the end when multiplying by the shuffle tuple key κ and performing the blind authentication check. The multiplication is secure by the security of SPDZ multiplication, and the blind authentication check ensures that any tampering during the permutation network protocol or κ -multiplication is detected with overwhelming probability.

5 Online Protocol: Applying Authenticated Shuffle Tuples

Our protocol FLOSS extends the SPDZ protocol [DPSZ12] for arithmetic circuits (see Appendix B.2) to support permutation gates via preprocessed authenticated shuffle tuples. As such, the two main protocol extensions to SPDZ consist of (1) the application of authenticated shuffle tuples to realize a permutation gate (subroutine shown in Figure 2), and (2) the blind authentication check used to detect malicious behavior during shuffling (subroutine shown in Figure 1). As in SPDZ, the authentication checks incurred during a permutation gate application are deferred until a value reconstruction; this minimizes the number of communication rounds as many authentication checks can be batched together.

We formally prove FLOSS secure with respect to a new arithmetic permutation circuit functionality that we define.

The functionality extends the standard arithmetic circuit functionality of SPDZ to include permutation and inverse permutation gates. We defer the complete protocol description, functionality definition, and security proof to Appendix D. Our main result is captured by the following theorem:

Theorem 2 (Informal). *The protocol FLOSS $[\mathbb{F}]$ (Figure 17) UC-realizes the ideal functionality $F_{\text{ArithPermCirc}}[\mathbb{F}]$ (Figure 16) in the $(F_{\text{SPDZPrep}}, F_{\text{ShufTuplePrep}}, F_{\text{Comm}})$ -hybrid model.*

Sketch: For the standard arithmetic gates, security follows from SPDZ. The proof relies on two key properties: (1) all messages received by the adversary are uniformly random (enabling simulation), and (2) any deviation in a value sent to the honest party is caught by the subsequent authentication check with overwhelming probability (ensuring correctness and preventing selective failure attacks).

The main challenge is extending deviation-detection to permutation gates, which is non-trivial when the honest party applies the permutation. Recall from Section 3 that a naive approach, which performs authentication checks directly on $\langle \pi(\vec{x}) \rangle$, is vulnerable to a selective failure attack: the adversary can introduce a perturbation before the shuffle and “fix” it afterward by guessing where the perturbation lands under π , passing the authentication check with non-negligible probability.

Our proof shows that by checking validity of the κ -multiplied shares instead, any adversarial deviation is caught unless the adversary guesses κ . Recall that the adversary sends blinded shares $\vec{c}$ and $\vec{d}$ to the honest party (see Figure 2). Suppose the adversary perturbs these by $\vec{\delta}_c$ and $\vec{\delta}_d$, respectively. The blind authentication check computes a random linear combination over the κ -multiplied shares (using a public random challenge r) and verifies that the parties’ shares of a verification value ϵ sum to zero. Tracing through shows that the honest party’s share takes the form

$$\llbracket \epsilon \rrbracket_1 = (\text{terms independent of } \kappa) + \kappa \cdot \sum_{j=1}^m r^{j-1} \pi(\vec{\delta}_d - \gamma \vec{\delta}_c)_j.$$

Assuming the adversary introduced some nonzero perturbation and did not guess γ , the vector $\pi(\vec{\delta}_d - \gamma \vec{\delta}_c)$ is nonzero, so the sum is a nonzero polynomial in r . By the Fundamental Theorem of Algebra, this polynomial evaluates to a nonzero value with overwhelming probability over the choice of r . Thus $\llbracket \epsilon \rrbracket_1$ is a non-constant affine function of the hidden key κ , making it uniformly random from the adversary’s view. The adversary must output $-\llbracket \epsilon \rrbracket_1$ to pass, which succeeds with negligible probability. We use this intuition to build a full simulation-based proof in Appendix D.

6 Application: Sorting

In this section, we construct a malicious-secure 2PC radix sort protocol as an arithmetic permutation circuit. We show

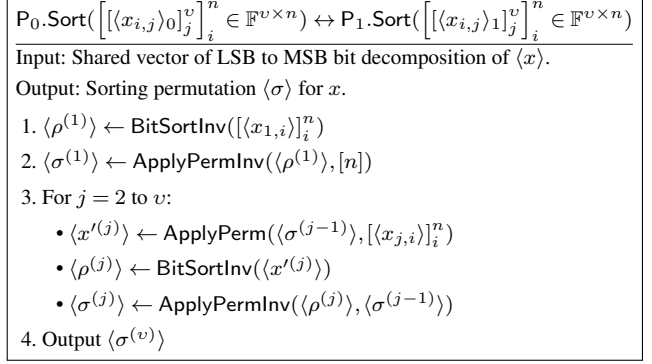


Figure 4: An arithmetic permutation circuit for radix-sorting secret shares of a bit-decomposed vector $\vec{x}$. Each element of $\vec{x}$ is represented using v bits.

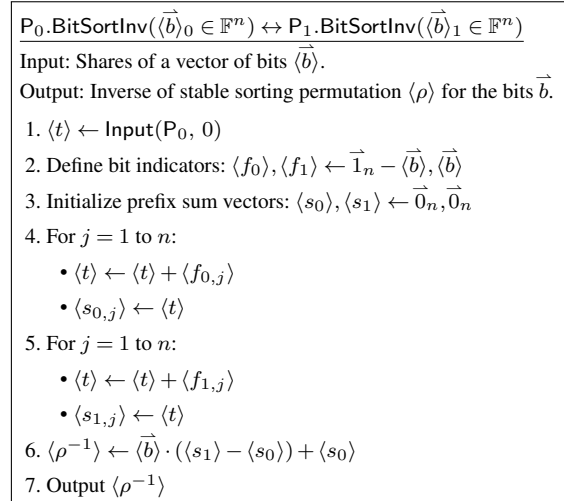


Figure 5: An arithmetic permutation circuit for producing the inverse sorting permutation (e.g., the destination encoding) for secret shares of a vector of bits.

that the high level strategy taken in the honest-majority 3PC radix sort protocol of Asharov et al. [AHI⁺22] maps naturally onto the arithmetic permutation circuit model. The radix sort protocol is provided in Figure 4.

In radix sort, parties accumulate a sorting permutation bit-by-bit. Parties begin by computing a stable sorting permutation on the least-significant bit. Then in sequence, parties will apply the accumulated sorting permutation to the next-significant bit, compute a stable sorting permutation for that bit position, then compose the current bit position’s stable sorting permutation with the accumulated sorting permutation. We break this up into a few subcircuits.

The first subcircuit BitSortInv (see Figure 5) produces shares of (the inverse of) the stable sorting permutation for a vector of secret-shared bits. Asharov et al.’s approach for BitSortInv translates directly as it can be expressed entirely using arithmetic operations (additions and multiplications); it does not rely on permutations.

$P_0.\text{ApplyPermInv}(\langle \rho \rangle_0 \in \mathbb{F}^n, \langle x \rangle_0 \in \mathbb{F}^n)$ $\leftrightarrow P_1.\text{ApplyPermInv}(\langle \rho \rangle_1 \in \mathbb{F}^n, \langle x \rangle_1 \in \mathbb{F}^n)$
Input: shares of permutation $\langle \rho \rangle$, shares of input $\langle x \rangle$. Output: shares of inverse permutation applied to inputs $\langle \rho^{-1}(x) \rangle$.
1. $\pi_b \leftarrow \$\text{Perm}(n)$ 2. $\text{Prep}((\text{id}_{\pi_b}, n_{\pi_b} = 2, n_{\pi_b, \text{inv}} = 0, \pi_b))$ 3. $\langle \pi_0(\rho) \rangle \leftarrow \text{Permute}(\text{id}_{\pi_0}, \langle \rho \rangle)$ 4. $\langle \pi_1(\pi_0(\rho)) \rangle \leftarrow \text{Permute}(\text{id}_{\pi_1}, \langle \pi_0(\rho) \rangle)$ 5. $\pi_1 \circ \pi_0 \circ \rho \leftarrow \text{Reconstruct}(\langle \pi_1(\pi_0(\rho)) \rangle)$ 6. $\rho^{-1} \circ \pi_0^{-1} \circ \pi_1^{-1} \leftarrow (\pi_1 \circ \pi_0 \circ \rho)^{-1}$ 7. $\langle \pi_0(x) \rangle \leftarrow \text{Permute}(\text{id}_{\pi_0}, \langle x \rangle)$ 8. $\langle \pi_1(\pi_0(x)) \rangle \leftarrow \text{Permute}(\text{id}_{\pi_1}, \langle \pi_0(x) \rangle)$ 9. $\langle \rho^{-1}(x) \rangle \leftarrow (\rho^{-1} \circ \pi_0^{-1} \circ \pi_1^{-1})(\langle \pi_1(\pi_0(x)) \rangle)$ 10. Output $\langle \rho^{-1}(x) \rangle$

Figure 6: An arithmetic permutation circuit for applying the inverse of an arbitrary secret-shared permutation ρ to a vector of data $\vec{x}$.

The other two subcircuits, `ApplyPerm` and `ApplyPermInv`, apply and unapply (i.e., apply the inverse of) an arbitrary permutation encoded as secret-shared indices $\langle \rho \rangle$ to a vector of secret-shared data $\langle x \rangle$, respectively. These protocols make use of permutations, and so, we depart from the permuting strategy taken in the 3PC setting [AKK⁺23]. Nevertheless, the high level approach remains the same: a random permutation unknown to all parties is applied to $\langle \rho \rangle$ to blind ρ , and the resulting blinded permutation is reconstructed and applied locally by each party to $\langle x \rangle$. We instantiate this by having each party, in sequence, perform a one-sided permutation (as a gate in the arithmetic permutation circuit model) on $\langle \rho \rangle$ using a preprocessed random permutation π_b to produce $\langle \pi_1 \circ \pi_0 \circ \rho \rangle$. If there exists an honest party that selects π_b during preprocessing at random, then the resulting blinding permutation $\pi_1 \circ \pi_0$ is uniformly random and hides ρ . The full protocols are given in Figures 3 and 6. Note that `ApplyPerm` makes use of inverse permutation gates for π_1 and π_0 to remove the blinding. We also provide an alternate construction of `ApplyPerm` that avoids using inverse permutation gates in Appendix F but it incurs double the permutation overhead.

All together, building these protocols as arithmetic permutation circuits allows arguing security by reasoning only about the correctness of the circuit (e.g., that it correctly implements sorting) and the simulatability of the reconstruct gates (e.g., the reconstructed blinded permutation appears random to the adversary). It separates the need to argue about the simulatability of the implementation of the arithmetic and permutation gates.

7 Evaluation

We evaluate FLOSS along two axes: *offline* costs and *online* costs. We define offline costs as computations that may be performed before the inputs are known. This consists of

generation of preprocessed correlated randomness, including generation of shuffle tuples. For offline costs, we focus on monetary costs as a combination of computation time and bandwidth. Parties have flexibility to run offline computations when convenient so specific wall-clock time measurements are less meaningful. In contrast, online costs are computations performed after the inputs are known, and wall-clock time (latency) is often the most important metric.

We estimate monetary costs using AWS pricing as of February 2026. For offline compute costs, we assume execution on a `c5.18xlarge` spot instance priced at \$0.93/hr, and we multiply this rate by the measured runtime of the offline portion. For storage costs, we compute the total volume of offline data generated and multiply it by the S3 Standard storage rate of \$0.023/(GB · month), prorated for the total duration (offline and online) of the protocol.

We wish to answer the following questions about FLOSS in comparison to malicious-secure 2PC baselines:

- *Online shuffling overhead:* How much faster and more communication-efficient is FLOSS’s one-sided shuffle compared to the baselines?
- *Offline shuffling overhead:* What is the offline cost required to enable online efficiency?
- *Higher level applications:* Can we use FLOSS as an interface to construct a sorting protocol with favorable monetary, computation, and communication costs to 2PC sorting alternatives?

We find that FLOSS greatly improves online costs without incurring significant offline cost overheads compared to prior approaches. In fact, for large vector sizes, FLOSS achieves both lower online *and* lower offline costs than all other prior approaches targeting fast online time. Our results are summarized in Figure 7 for shuffling and Figure 9 for sorting which plot offline monetary cost against online time; the plots use reverse axes so that better outcomes (cheaper and faster) trend toward the upper right. For shuffling 2^{20} values, FLOSS improves by $800\times$ in online time with $2\times$ less offline cost.

7.1 Implementation

We implement FLOSS in Rust. We provide a base implementation of the SPDZ protocol [DPSZ12] for arithmetic circuits (see Figure 14 and Appendix B.2). Our implementation of arithmetic permutation circuits uses relevant SPDZ gates/gadgets like addition and multiplication and extends to include permutation gates (see Figure 17). We provide an example implementation of a custom sorting gate constructed with addition, multiplication, and permutation gates. The implementation provides a framework to define further higher-level custom gates on top of the gates we provide to easily reuse and construct more complex interactive arithmetic permutation circuits. Our implementation is field-agnostic using the `arkworks` library; our experiments use the 128-bit prime $p = 2^{127} + 55 \cdot 2^{15} + 1$ used by MP-SPDZ [mpsb].

To generate SPDZ multiplication triples and input tuples, we call out to the MP-SPDZ library [mpsb] which supports various protocols including MASCOT [KOS16] and LowGear [KPR18]. While LowGear is more efficient, the MP-SPDZ implementation of LowGear does not support generation of small batches of triples ($< 10^6$). Thus, our benchmarks use MASCOT for smaller vector sizes and LowGear for larger vector sizes. We run both protocols with security parameter $\lambda = 40$. For the offline generation of shuffle tuples in FLOSS (see Figure 18 and Appendix E), we instantiate the permutation network approach using the naive $\log(n)$ -round circuit evaluation protocol (in contrast, to the online-optimized constant-online-round protocol of Mohassel et al. [MSS14]). We select the Waksman network [Wak68] to use as the underlying permutation network topology.

In total, our implementation of FLOSS consists of ≈ 15000 lines of code (LoC); it is available open source.

Shuffling baselines. We compare to three baselines for malicious-secure 2PC shuffling. First, we consider the circuit-based permutation network construction proposed by Mohassel et al. [MSS14]. This construction avoids the expensive $\log n$ rounds of interaction needed to naively evaluate a permutation network of depth $\log n$ via an offline permutation encoding technique to achieve constant online rounds. We refer to this baseline as *PermNet*. We reimplement this construction on top of our SPDZ interface in ≈ 1300 LoC.

The second baseline is the malicious-secure adaptation of the Chase et al. [CGP20] honest-but-curious shuffle tuple construction proposed by Song et al. [SYB+24]. We use a prototype implementation provided by the authors over the same prime field. We refer to this baseline as *OPM*: a key characteristic is its use of oblivious permutation matrices for offline computation of shuffle tuples.

OPM has a number of parameters that influence its costs. First, the size T of the oblivious permutation matrix used in the offline portion is parameterized; these are incorporated into a permutation network as higher degree switches. Roughly, larger T incurs longer compute time but lower communication cost during the offline computation. We refer to Table III and IV in [SYB+24] for details; our experiments use $T = 2^4$. Second, the cut-and-choose parameter N together with T determine the number of shuffle tuples B to be applied for each shuffle. Larger N and larger T result in smaller B , but require a larger one-time offline cost in which NB shuffle tuples are created and shared. For example, to shuffle a vector of length $n = 2^{18}$ for $T = 2^4$ and $N = 2^{10}$ times, the one-time offline cost requires computing and sending $2^{10} \cdot B$ shuffle tuples for $B = 108$ shuffle tuples, taking $\approx 2^{10} \cdot 171s = 2$ days of time and incurring $\approx 2^{10} \cdot 2.0GB = 2TB$ in communication costs. We omit this one-time cost in our experiments. The prototype implementation computes the *marginal* offline cost, which is the cost to generate and apply B shuffle tuples excluding the N initial repetitions. We report on the marginal

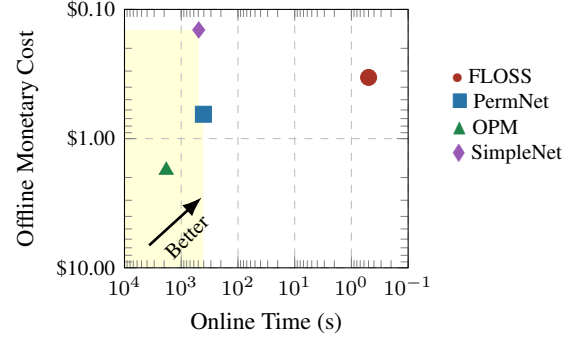


Figure 7: Cost summary of protocols for shuffling 2^{20} entries. FLOSS has $\geq 800\times$ faster online time than prior protocols, and $2\times$ cheaper offline monetary cost than prior protocols outside of SimpleNet. We highlight prior work in yellow.

cost setting $N = 2^4$. This estimates what an “amortized” cost for a single shuffle would be if $N = 2^4$ shuffles are performed.

Lastly, we also include comparison against the simple $\log(n)$ -round permutation network evaluated as a circuit. This provides a baseline for low offline costs and a straightforward but expensive online protocol. We refer to this as SimpleNet.

Sorting baselines. We evaluate three baseline sorting algorithms: Radix-Sort-and-Shuffle [AHI+22], Shuffle-then-Sort [HKI+12], and a sorting network [JKU11].

Both Radix-Sort-and-Shuffle and Shuffle-then-Sort rely on a shuffling subroutine. We describe the Radix-Sort-and-Shuffle algorithm in Section 6 as an arithmetic permutation circuit. Similarly, Shuffle-then-Sort can also be described generically as an arithmetic permutation circuit. Here, inputs are shuffled and then sorted using a quicksort algorithm in which comparison bits are reconstructed to determine sorting. We implement both of these sorting algorithms instantiating the underlying shuffling subroutine using the shuffling protocols: FLOSS, PermNet, SimpleNet, and OPM. Note, we do not instantiate the Radix-Shuffle-and-Sort algorithm using OPM because it is not clear how to do this. OPM only allows applying random permutations, and these permutations can only be applied once. Table 2 summarizes which shuffling protocols are compatible with each sorting algorithm. We refer to these baselines Radix[FLOSS], Radix[PermNet], Radix[SimpleNet], StS[FLOSS], StS[PermNet], StS[SimpleNet], and StS[OPM], respectively.

Lastly, we compare to the canonical sorting network [JKU11] implemented in MP-SPDZ [mpsb] in which comparison bits are held secret and values are swapped through the sorting network obliviously. We refer to this baseline as SortNet. It does not use a shuffling subroutine.

Experimental setup. We run the two parties on two different Amazon EC2 c5.18xlarge instances, each with 72 vCPUs. Our benchmarks are run multithreaded.

Sorting algorithm	Shuffle			
	FLOSS	OPM	PermNet	SimpleNet
Radix-Sort-and-Shuffle [AHI+22]	✓		✓	✓
Shuffle-then-Sort [HKI+12]	✓	✓	✓	✓

Table 2: Summary of compatible shuffling protocols for instantiating shuffling-based sorting algorithms.

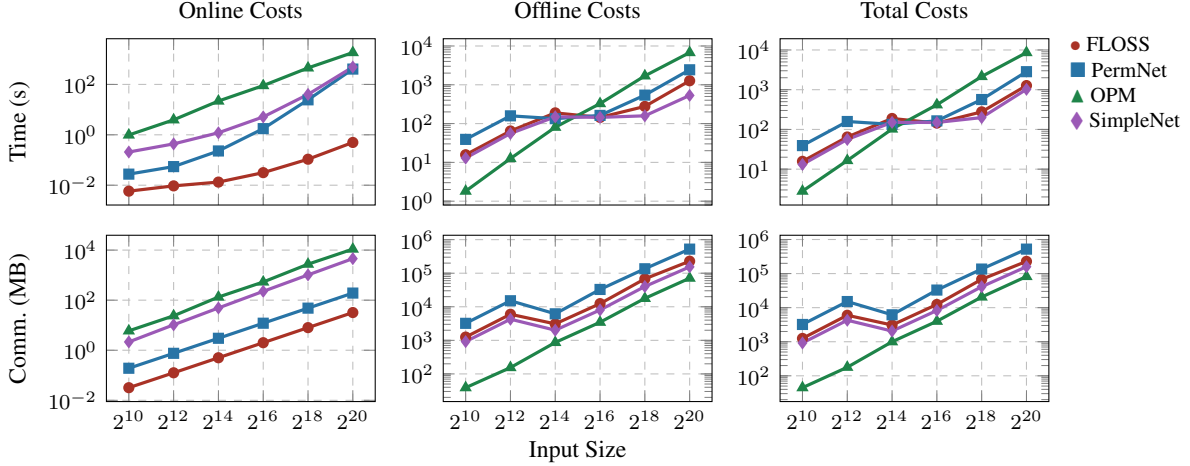


Figure 8: Computation time and communication costs for shuffling broken down by online and offline phases. FLOSS has lower online costs than all baselines.

7.2 Online Shuffling Costs

We microbenchmark the online costs of performing a single one-sided shuffle in Figure 8. As expected, FLOSS outperforms the baselines in both computation time and communication costs, due to efficient application of authenticated shuffle tuples making up the online costs. For input size 2^{20} , the online protocol completes in < 500 ms and sends 33.6MB, which corresponds approximately to sending two field vectors of size 2^{20} .

Of the 500 ms reported for the online time of FLOSS, the majority of the cost ≈ 400 ms comes from executing the permutation gate, while the remaining ≈ 100 ms corresponds to the cost of the deferred blind authentication check. This corresponds to $800\times$ and $3600\times$ (406s and 1835s) computation and $6\times$ and $341\times$ (201.3MB and 11.5GB) communication improvements over PermNet and OPM, respectively. Intuitively FLOSS outperforms the baselines because PermNet incurs additional costs based on the size of the permutation network (as we discuss next) and requires interaction to prepare the inputs and outputs, while OPM incurs higher concrete costs due to repeated application of B shuffle tuples from its cut-and-choose analysis.

As far as scaling, the online computational costs of FLOSS scale linearly in the vector size n , while OPM and PermNet scale with $O(n \log n)$ due to parameters that depend on a logarithmic depth permutation network. For FLOSS, increasing vector size from 2^{10} to 2^{20} leads to computational costs that

are $85\times$ higher and communication costs that are $\approx 2^{10}\times$ higher. We observe sublinear scaling in FLOSS for small vector lengths from 2^{10} to 2^{18} in compute time, increasing only $18\times$ from 5.8ms to 107ms instead of the expected $2^8 = 256\times$. From 2^{18} to 2^{20} , there is a superlinear $5\times$ increase in compute time from 107ms to 498ms, instead of the expected $4\times$. We hypothesize this superlinear scaling is due to cache thrashing; at 2^{20} , these operations consume 32MB, which exceeds the 24.8MB L3 cache of our c5.18xlarge instance [Inc26, Ama26]. We extend our experiments to verify behavior after 2^{20} elements, and we see that computational overhead stabilizes with linear growth. Shuffling 2^{20} elements with FLOSS takes 0.50s, shuffling 2^{21} elements takes 0.98s, and shuffling 2^{22} elements takes 1.99s. We expect that linear growth continues from there.

For the same jump in vector size, 2^{10} to 2^{20} ($\approx 1000\times$ larger), OPM incurs $1800\times$ more computation from 1s to 1835s and $1850\times$ more communication from 6.22MB to 11.5GB. For PermNet, the computation costs are $14000\times$ higher from 27.5 ms to 406.5s, and the communication costs are $1000\times$ higher from 200KB to 201.3MB. PermNet grows linearly in communication costs due to only needing to send field elements corresponding to n input and output wires of the network.

Size	FLOSS offline (s)			PermNet offline (s)		
	SPDZ	Circuit	Total	SPDZ	Circuit	Total
2^{10}	15.30	0.43	15.73	38.98	0.20	39.17
2^{12}	63.64	0.82	64.46	157.75	0.53	158.28
2^{14}	186.76	2.01	188.77	131.94	1.88	133.82
2^{16}	137.02	7.27	144.29	152.07	10.45	162.52
2^{18}	230.92	45.95	276.88	461.12	80.48	541.60
2^{20}	755.84	514.54	1270.38	1762.53	673.53	2436.06

Table 3: Breakdown of offline cost for FLOSS and PermNet between cost to generate SPDZ correlated randomness preprocessing and permutation-specific preprocessing. FLOSS requires $\approx 2\times$ less SPDZ preprocessing than PermNet.

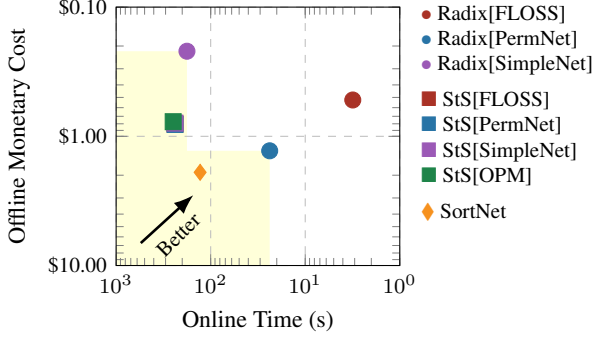


Figure 9: Cost summary of protocols for sorting 2^{13} 32-bit decomposed values. Radix-Sort-and-Shuffle instantiated with FLOSS has $\geq 5\times$ faster online time and $1.5\times$ cheaper offline monetary cost than prior protocols outside of SimpleNet. We highlight prior work in yellow.

7.3 Offline Shuffling Costs

As shown in Figure 7, which summarizes the offline AWS compute cost vs. online-time tradeoff for input size 2^{20} , FLOSS achieves the best offline cost efficiency among all baselines other than SimpleNet. FLOSS costs \$0.33, PermNet costs \$0.64, and OPM costs \$1.75, while SimpleNet costs only \$0.14 (less than half of FLOSS).

We provide microbenchmarks for the offline compute time and bandwidth costs of generating a shuffle tuple in Figure 8. For lower input sizes ($\leq 2^{14}$), OPM has a less costly offline computation than FLOSS. However, OPM’s offline and total costs exceed FLOSS at larger input sizes ($\geq 2^{16}$) due to increasing cut-and-choose overhead.

The offline portion for SPDZ-based protocols (FLOSS and PermNet) each consist of two components: first, generating SPDZ triples and input tuples, and second, computing relevant preprocessing for the permutation network circuit (e.g., authenticated shuffle tuples for FLOSS). For small vector instances, the cost for generating the SPDZ correlated randomness dominates the cost of the permutation preprocessing. However, for larger vector instances, the share of permutation preprocessing increases, and starts to approach even by vector length 2^{20} . Table 3 details this breakdown in offline costs. For

shuffling input size 2^{20} in FLOSS and PermNet, respectively, 60% and 72% (756s out of 1270s and 1763s out of 2436s) of the offline cost is generating the SPDZ triples and input tuples, while 40% and 28% (514s out of 1270s and 673s out of 2436s) of the offline cost is computing the permutation network circuit. Recall, that our implementation replaces MAS-COT SPDZ preprocessing in the offline phase with LowGear at a certain instance size. This surfaces as a dip in the plots in Figure 8. Lastly, in terms of scaling, the offline computational and communication costs of FLOSS, PermNet, SimpleNet, and OPM scale on the order of $O(n \log n)$, as all constructions rely on logarithmic-depth permutation networks.

7.4 Sorting Costs

We compare the costs of combining different shuffling protocols with different sorting protocols for sorting bit-decomposed 32-bit values. As shown in Figure 9, which summarizes the offline AWS compute cost vs. online-time tradeoff for sorting input size 2^{13} , Radix[FLOSS] achieves the best offline cost efficiency. Radix[FLOSS] costs \$0.52, StS[OPM] costs \$0.77, StS[FLOSS] costs \$0.80, StS[PermNet] costs \$0.80, Radix[PermNet] costs \$1.29, and SortNet costs \$1.90 to perform offline computation, respectively. To provide a more detailed view across input sizes, we provide microbenchmarks for the online and offline compute times and bandwidth costs of sorting in Figure 10.

Radix[FLOSS] achieves the lowest online shuffle cost across all evaluated input sizes. For input size 2^{13} , Radix[FLOSS] has the lowest online computational and communication cost of 3.1s and 98.1MB. The second-best baseline, Radix[PermNet], has online computational and communication cost of 23.8s and 407MB.

For inputs ranging from 2^9 to 2^{13} , the online shuffle time of Radix[FLOSS] increases from 1s to 3s ($3\times$), while online communication increases from 6.2MB to 98.1MB ($16\times$). In contrast, Radix[PermNet] incurs significantly higher online shuffle overheads, with shuffle time increasing from 4.7s to 23.8s ($6\times$) and communication increasing from 26.2MB to 407.1MB ($15.5\times$) over the same range. The online shuffle cost of PermNet comprises the majority of its online sorting cost (approximately 85–90%), whereas the online shuffle cost of FLOSS accounts for only a small fraction (approximately 20–30%) of the total online sorting cost.

Asymptotically, the online computational overhead of Radix[FLOSS] is $O(v \cdot n)$, while all other evaluated online shuffles incur $O(v \cdot n \log n)$ overhead. The $O(n \log n)$ online cost arises from dependence on logarithmic-depth network structures, as in SortNet sorting and OPM, and PermNet shuffling.

As expected, offline costs dominate the total runtime across all constructions. For example, at input size 2^{13} , OPM-then-sort incurs offline computation and communication costs of 2915s and 84.5GB, while SortNet incurs 7284s and 211GB. Radix[FLOSS] incurs lower computational cost

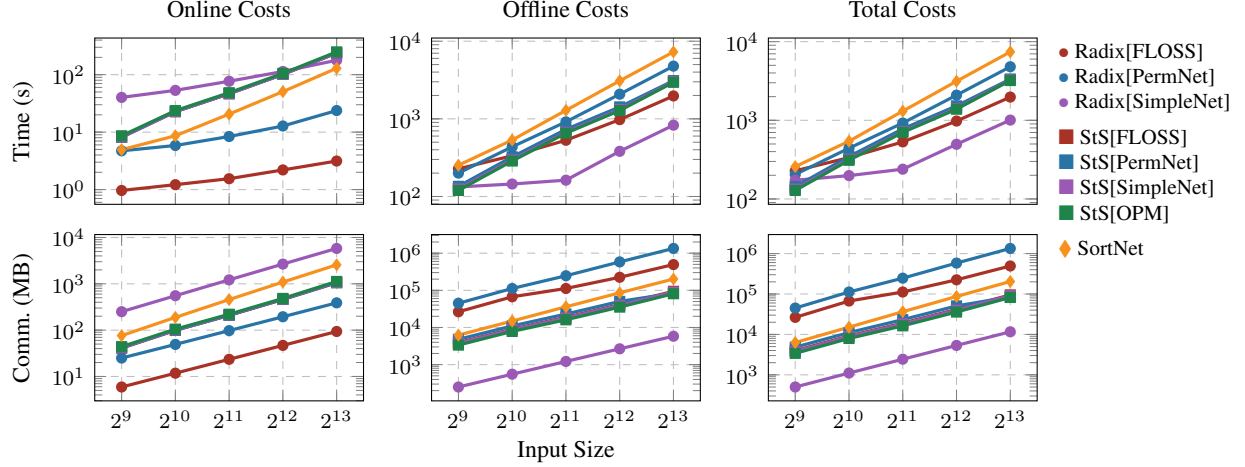


Figure 10: Computation time and communication costs for sorting broken down by online and offline phases. FLOSS when combined with the Radix-Sort-and-Shuffle algorithm has lower online costs than all baselines.

at 1972s and higher communication cost at 514GB. The higher communication cost reflects the additional communication cost for permutation network evaluation via multiplication triples. Despite this, Radix[FLOSS] achieves the lowest growth in total cost as input size increases from 2^9 to 2^{13} , with total computation increasing from 228s to 1976s ($8.7\times$) and total communication increasing from 27.7GB to 518GB ($18.7\times$). Radix[PermNet] exhibits higher total growth, increasing from 205s to 4798s ($23.4\times$) and from 47.1GB to 1408GB ($29.9\times$). Shuffle-then-Sort baselines incur similar total growth, with computation increasing from 127s to 3153s ($24.8\times$) and communication increasing from 3.5GB to 85.7GB ($24.2\times$). In comparison, SortNet scales poorly, with total computation increasing from 258s to 7414s ($28.8\times$) and total communication increasing from 6.6GB to 214.0GB ($32.4\times$). Overall, all preprocessing protocols exhibit $O(n \log n)$ offline scaling due to reliance on logarithmic-depth permutation networks, while the smaller constant factors of permutation networks and radix sorting explain the favorable growth behavior of Radix[FLOSS] and Radix[PermNet].

Acknowledgements

We thank Song et al. for providing access to and permission to redistribute their code for the OPM construction. We also thank Stefano Tessaro for his helpful suggestions on the protocol design and proof.

This worked was funded in part by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy – EXC 2092 CASA – 390781972.

Ethical Considerations

We present FLOSS to enable faster linear-time online shuffles in the malicious-secure 2PC setting. This primitive is a core building block in many privacy-preserving protocols, including those for privacy-preserving analytics, advertising, and anonymous broadcasting. Consequently, FLOSS can be deployed at scale to process sensitive and socially significant data.

FLOSS provides a faster and more communication-efficient primitive for secret-shared shuffling, thereby lowering the barrier to deploying privacy-preserving systems. This improved efficiency is beneficent for internet users and internet services as it makes advanced private analytics, such as oblivious joins and encrypted database operations, more practical to deploy which, in turn, enable services to provide better user experiences while respecting users' privacy. Providing more performant privacy-preserving primitives can lead to wider deployment of privacy-preserving collection, where most services collect and train on clear data. At the same time, users should be wary of internet services using the guise of privacy-preserving computation to collect sensitive statistics. While the cryptographic protocols presented here enable privacy-preserving collection of a statistic, they say nothing about the privacy implications of the collected statistic itself. Users should have transparency into what statistic the aggregation service is computing over their data, and any deployments should carefully consider the privacy implications of the collected statistic.

Open Science

We provide open access to all artifacts described in this paper.

All code needed to reproduce benchmarking of FLOSS is available via a public repository³. The repository includes scripts to automatically set up the AWS instances and install prerequisites, as well as a README file with instructions on how to run the experiments and tests. It also includes:

- Our full system implementation of FLOSS, and a framework for constructing 2PC arithmetic permutation circuits.
- MP-SPDZ compilation instructions for MASCOT and LowGear.
- Our MP-SPDZ circuits for Quicksort and sorting network.
- The source code and benchmarks for OPM, obtained from the authors of [SYB⁺24].
- Scripts to run and plot the aforementioned experiments.

In a second repository⁴, we provide our code and instructions to run the proof-of-concept attack against the implementation of OPM (see Appendix C for details). This attack is lightweight and intended to be run locally using the provided Dockerfile.

³<https://github.com/iachang/floss>

⁴<https://github.com/selनावot/mosac-zero-check>

References

- [ACD⁺24] Erik Anderson, Melissa Chase, F. Betül Durak, Kim Laine, and Chenkai Weng. Precio: Private aggregate measurement via oblivious shuffling. In *CCS*, pages 1819–1833. ACM, 2024.
- [AFO⁺21] Toshinori Araki, Jun Furukawa, Kazuma Ohara, Benny Pinkas, Hanan Rosemarin, and Hikaru Tsuchida. Secure graph analysis at scale. In *CCS*, pages 610–629. ACM, 2021.
- [AHI⁺22] Gilad Asharov, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Ariel Nof, Benny Pinkas, Katsumi Takahashi, and Junichi Tomida. Efficient secure three-party sorting with applications to data analysis and heavy hitters. In *CCS*, pages 125–138. ACM, 2022.
- [AKK⁺23] Pranav Shriram A, Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, Bhavish Raj Gopal, and Somya Sangal. Ruffle: Rapid 3-party shuffle protocols. *Proc. Priv. Enhancing Technol.*, 2023(3):24–42, 2023.
- [AKTZ17] Nikolaos Alexopoulos, Aggelos Kiayias, Riivo Talviste, and Thomas Zacharias. Mcmix: Anonymous messaging via secure multiparty computation. In *USENIX Security Symposium*, pages 1217–1234. USENIX Association, 2017.
- [Ama26] Amazon Web Services. Specifications for amazon ec2 compute optimized instances (including c5.18xlarge), 2026. AWS official EC2 documentation listing c5 instance types such as c5.18xlarge.
- [BBM⁺25] Eli Baum, Sam Buxbaum, Nitin Mathai, Muhammad Faisal, Vasiliki Kalavri, Mayank Varia, and John Liagouris. ORQ: complex analytics on private data with strong security guarantees. In *SOSP*, pages 802–833. ACM, 2025.
- [BD02] Bruno Beauquier and Éric Darrot. On arbitrary size waksman networks and their vulnerability. *Parallel Process. Lett.*, 12(3-4):287–296, 2002.
- [BDG⁺22] Saikrishna Badrinarayanan, Sourav Das, Gayathri Garimella, Srinivasan Raghuraman, and Peter Rindal. Secret-shared joins with multiplicity from aggregation trees. In *CCS*, pages 209–222. ACM, 2022.
- [Bea91] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *CRYPTO*, volume 576 of *Lecture Notes in Computer Science*, pages 420–432. Springer, 1991.
- [BG12] Stephanie Bayer and Jens Groth. Efficient zero-knowledge argument for correctness of a shuffle. In *EUROCRYPT*, volume 7237 of *Lecture Notes in Computer Science*, pages 263–280. Springer, 2012.
- [BGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *STOC*, pages 1–10. ACM, 1988.
- [BLT14] Dan Bogdanov, Sven Laur, and Riivo Talviste. A practical analysis of oblivious sorting algorithms for secure multi-party computation. In *NordSec*, volume 8788 of *Lecture Notes in Computer Science*, pages 59–74. Springer, 2014.
- [BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *CCS*, pages 62–73. ACM, 1993.
- [BTW12] Dan Bogdanov, Riivo Talviste, and Jan Willemson. Deploying secure multi-party computation for financial data analysis - (short paper). In *Financial Cryptography*, volume 7397 of *Lecture Notes in Computer Science*, pages 57–64. Springer, 2012.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *FOCS*, pages 136–145. IEEE Computer Society, 2001.
- [CB17] Henry Corrigan-Gibbs and Dan Boneh. Prio: Private, robust, and scalable computation of aggregate statistics. In *NSDI*, pages 259–282. USENIX Association, 2017.
- [CCL15] Ran Canetti, Asaf Cohen, and Yehuda Lindell. A simpler variant of universally composable security for standard multiparty computation. In *CRYPTO (2)*, volume 9216 of *Lecture Notes in Computer Science*, pages 3–22. Springer, 2015.
- [CGP20] Melissa Chase, Esha Ghosh, and Oxana Poburinnaya. Secret-shared shuffle. In *ASIACRYPT (3)*, volume 12493 of *Lecture Notes in Computer Science*, pages 342–372. Springer, 2020.
- [CHI⁺19] Koji Chida, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Naoto Kiribuchi, and Benny Pinkas. An efficient secure three-party sorting protocol with an honest majority. *IACR Cryptol. ePrint Arch.*, page 695, 2019.
- [CJK⁺23] Benjamin M. Case, Richa Jain, Alex Koshelev, Andy Leiserson, Daniel Masny, Ben Savage, Erik Taubeneck, Martin Thomson, and Taiki Yamaguchi. Interoperable private attribution: A distributed attribution and aggregation protocol. *IACR Cryptol. ePrint Arch.*, page 437, 2023.
- [Cle86] Richard Cleve. Limits on the security of coin flips when half the processors are faulty (extended abstract). In *STOC*, pages 364–369. ACM, 1986.
- [div] Divvi up: A privacy respecting telemetry service. <https://divviup.org/>.
- [DKL⁺13] Ivan Damgård, Marcel Keller, Enrique Larraia, Valerio Pastro, Peter Scholl, and Nigel P. Smart. Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits. In *ESORICS*, volume 8134 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2013.
- [DNO25] Samuel Dittmer, Rohit Nema, and Rafail Ostrovsky. Linear secret-shared shuffle with malicious security. *Cryptology ePrint Archive*, Paper 2025/2137, 2025.
- [DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *CRYPTO*, volume 7417 of *Lecture Notes in Computer Science*, pages 643–662. Springer, 2012.
- [EB22] Saba Eskandarian and Dan Boneh. Clarion: Anonymous communication from multiparty shuffling protocols. In *NDSS*. The Internet Society, 2022.
- [FJKW15] Sky Faber, Stanislaw Jarecki, Sotirios Kentros, and Boyang Wei. Three-party ORAM for secure computation. In *ASIACRYPT (1)*, volume 9452 of *Lecture Notes in Computer Science*, pages 360–385. Springer, 2015.
- [FO21] Brett Hemenway Falk and Rafail Ostrovsky. Secure merge with $o(n \log \log n)$ secure operations. In *ITC*, volume 199 of *LIPIcs*, pages 7:1–7:29. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

- [FOSZ23] Brett Hemenway Falk, Rafail Ostrovsky, Matan Shtepel, and Jacob Zhang. Gigadram: Breaking the billion address barrier. In *USENIX Security Symposium*, pages 3871–3888. USENIX Association, 2023.
- [GKLX22] S. Dov Gordon, Jonathan Katz, Mingyu Liang, and Jiayu Xu. Spreading the privacy blanket: - differentially oblivious shuffling for differential privacy. In *ACNS*, volume 13269 of *Lecture Notes in Computer Science*, pages 501–520. Springer, 2022.
- [GL05] Shafi Goldwasser and Yehuda Lindell. Secure multi-party computation without agreement. *J. Cryptol.*, 18(3):247–287, 2005.
- [GPP⁺25] Tim Geoghegan, Christopher Patton, Brandon Pitman, Eric Rescorla, and Christopher A. Wood. Distributed Aggregation Protocol for Privacy Preserving Measurement. Internet-Draft draft-ietf-ppm-dap-14, Internet Engineering Task Force, February 2025. Work in Progress.
- [Gro09] Jens Groth. Linear algebra with sub-linear zero-knowledge arguments. In *CRYPTO*, volume 5677 of *Lecture Notes in Computer Science*, pages 192–208. Springer, 2009.
- [Gro10] Jens Groth. A verifiable secret shuffle of homomorphic encryptions. *J. Cryptol.*, 23(4):546–579, 2010.
- [GZZ24] Jiacheng Gao, Yuan Zhang, and Sheng Zhong. Multiparty shuffle: Linear online phase is almost for free. *IACR Cryptol. ePrint Arch.*, page 1936, 2024.
- [HKI⁺12] Koki Hamada, Ryo Kikuchi, Dai Ikarashi, Koji Chida, and Katsumi Takahashi. Practically efficient multi-party sorting protocols from comparison sort algorithms. In *ICISC*, volume 7839 of *Lecture Notes in Computer Science*, pages 202–216. Springer, 2012.
- [Inc26] Primate Labs Inc. Amazon ec2 c5.xlarge benchmark results (intel xeon platinum 8124m), 2026. Geekbench Browser — CPU result showing L3 Cache = 24.8 MB for Intel Xeon Platinum 8124M on EC2.
- [JKU11] Kristján Valur Jónsson, Gunnar Kreitz, and Misbah Uddin. Secure multi-party sorting and applications. *IACR Cryptol. ePrint Arch.*, page 122, 2011.
- [Kel20] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *CCS*, pages 1575–1590. ACM, 2020.
- [KLDF16] Albert Kwon, David Lazar, Srinivas Devadas, and Bryan Ford. Riffle: An efficient communication system with strong anonymity. *Proc. Priv. Enhancing Technol.*, 2016(2):115–134, 2016.
- [KNOS25] Alexander Kyster, Frederik Huss Nielsen, Sabine Oechsner, and Peter Scholl. Rushing at SPDZ: on the practical security of malicious MPC implementations. In *SP*, pages 2491–2508. IEEE, 2025.
- [KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: faster malicious arithmetic secure computation with oblivious transfer. In *CCS*, pages 830–842. ACM, 2016.
- [KPR18] Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making SPDZ great again. In *EUROCRYPT* (3), volume 10822 of *Lecture Notes in Computer Science*, pages 158–189. Springer, 2018.
- [Lau21] Peeter Laud. Linear-time oblivious permutations for SPDZ. In *CANS*, volume 13099 of *Lecture Notes in Computer Science*, pages 245–252. Springer, 2021.
- [Lin17] Yehuda Lindell. How to simulate it - A tutorial on the simulation proof technique. In *Tutorials on the Foundations of Cryptography*, pages 277–346. Springer International Publishing, 2017.
- [LYK⁺19] Donghang Lu, Thomas Yurek, Samarth Kulshreshtha, Rahul Govind, Aniket Kate, and Andrew Miller. Honeybadgermpc and asynchromix: Practical asynchronous MPC and its application to anonymous communication. In *CCS*, pages 887–903. ACM, 2019.
- [mpsa] MP-SPDZ: Separate preprocessing documentation. <https://mp-spdz.readthedocs.io/en/latest/preprocessing.html#separate-preprocessing>.
- [mpsb] MP-SPDZ: Multi-protocol spdz library. <https://github.com/data61/MP-SPDZ>.
- [MRR20] Payman Mohassel, Peter Rindal, and Mike Rosulek. Fast database joins and PSI for secret shared data. In *CCS*, pages 1271–1287. ACM, 2020.
- [MS13] Payman Mohassel and Seyed Saeed Sadeghian. How to hide circuits in MPC an efficient framework for private function evaluation. In *EUROCRYPT*, pages 557–574. Springer, 2013.
- [MSS14] Payman Mohassel, Seyed Saeed Sadeghian, and Nigel P. Smart. Actively secure private function evaluation. In *ASIACRYPT*, pages 486–505. Springer, 2014.
- [Nef01] C. Andrew Neff. A verifiable secret shuffle and its application to e-voting. In *CCS*, pages 116–125. ACM, 2001.
- [NWI⁺15] Kartik Nayak, Xiao Shaun Wang, Stratis Ioannidis, Udi Weinsberg, Nina Taft, and Elaine Shi. Graphsc: Parallel secure computation made easy. In *IEEE Symposium on Security and Privacy*, pages 377–394. IEEE Computer Society, 2015.
- [PLS⁺25] Andrew Paseltiner, Andy Leiserson, Benjamin Savage, Charlie Harrison, and Martin Thomson. Privacy-Preserving Attribution: Level 1. First-public-working-draft, World Wide Web Consortium, April 2025. Work in Progress.
- [PRRS25] Stanislav Peceny, Srinivasan Raghuraman, Peter Rindal, and Harshal Shah. Efficient permutation correlations and batched random access for two-party computation. In *PKC* (4), volume 15677 of *Lecture Notes in Computer Science*, pages 76–109. Springer, 2025.
- [SA19] Nigel P. Smart and Younes Talibi Alaoui. Distributing any elliptic curve based protocol. In *IMACC*, volume 11929 of *Lecture Notes in Computer Science*, pages 342–366. Springer, 2019.
- [SYB⁺24] Xiangfu Song, Dong Yin, Jianli Bai, Changyu Dong, and Ee-Chien Chang. Secret-shared shuffle with malicious security. In *NDSS*. The Internet Society, 2024.
- [Wak68] Abraham Waksman. A permutation network. *J. ACM*, 15(1):159–163, 1968.
- [WHC⁺14] Xiao Shaun Wang, Yan Huang, T.-H. Hubert Chan, Abhi Shelat, and Elaine Shi. SCORAM: oblivious RAM for secure computation. In *CCS*, pages 191–202. ACM, 2014.

Game $\text{REAL}_{\text{Env}, \text{C}, \Pi, \text{A}}^{2\text{pc}, b}(\lambda)$ $\text{st} \leftarrow \Pi.P_b.\text{Init}(\lambda)$ $(\text{in}_A, \text{in}_b, \text{st}_{\text{Env}}) \leftarrow \text{Env.In}(\lambda)$ $(\text{out}_A, \text{out}_b) \leftarrow (\text{A}.\Pi(\text{in}_A) \leftrightarrow \text{C}^{\Pi\text{ALG}}(\text{in}_b))$ $b' \leftarrow \text{Env.Out}(\text{out}_A, \text{out}_b, \text{st}_{\text{Env}})$ Oracle $\Pi\text{ALG}(\text{Alg}, \text{in})$ $(\text{st}, \text{out}) \leftarrow (\Pi.P_b.\text{Alg}(\text{st}, \text{in}) \leftrightarrow \text{A})$ Return out	Game $\text{IDEAL}_{\text{Env}, \text{C}, \text{F}, \text{S}}^{2\text{pc}, b}(\lambda)$ $\text{st}_F \leftarrow \text{F.Init}(\lambda)$ $(\text{in}_S, \text{in}_b, \text{st}_{\text{Env}}) \leftarrow \text{Env.In}(\lambda)$ $\text{outs} \leftarrow (\text{S}.\Pi(\text{in}_S) \leftrightarrow \text{F}); \text{out}_b \leftarrow \text{C}^{\text{FALG}}(\text{in}_b)$ $b' \leftarrow \text{Env.Out}(\text{outs}, \text{out}_b, \text{st}_{\text{Env}})$ Oracle $\text{FALG}(\text{Alg}, \text{in})$ $\text{out} \leftarrow (\text{P}_b \leftrightarrow \text{F}.\text{Alg}(\text{in}))$ Return out
--	---

Figure 11: Two-party UC security definition.

A Universal Composability Security Model

We use Universal Composability (UC) [Can01] to prove the security of all our protocols. In particular, we adapt the Simplified UC framework (SUC) of Canetti et al. [CCL15]. Our adaptations include further simplifications due to only considering two-party protocols with static corruptions, as well as added support for control functions to model adversary-controlled and adaptively chosen circuits.

In this section we provide a brief overview, but assume some familiarity with the UC framework; we refer the reader to [Can01, CCL15] for more details.

Modeling corrupt parties. The security of a protocol Π is defined by comparing a “real” execution of Π to an “ideal” execution of a trusted and interactive functionality F . In the real execution, an adversary A corrupts one party P_b (where $b \in \{0, 1\}$) statically at the beginning of the protocol execution. The adversary controls all messages sent by the corrupted party, and it can deviate arbitrarily from the protocol specification and adaptively choose its strategy based on observed protocol messages. In the ideal execution, a simulator S plays the role of the adversary but interacts with an ideal functionality F instead of a real protocol.

Modeling honest parties. We assume a strong threat model where the adversary controls the circuit being evaluated. To model this, we use the notion of control functions [Can01], where an adversary-controlled algorithm C decides which operations to invoke based on intermediate results (e.g., choosing what to multiply or open based on a previously opened value). In the real world, C controls the behavior of the honest party; in the ideal world, it makes direct calls to the ideal functionality.

Modeling communication. We assume the adversary controls the entire communication channel. This is natural in the two-party setting, since all communication between the parties in the real world includes a corrupt party.

In the ideal world, the simulator additionally controls the communication between the honest party and the ideal functionality, including the ability to delay or drop messages. Importantly, however, the simulator cannot read or modify the contents of these messages. This standard abstraction of adversarial control immediately captures the adversary’s ability to prevent the honest party from receiving the final output, which is inherent in the MPC with abort setting [Cle86, GL05].

Composition with hybrid functionalities. Protocols may be analyzed in hybrid models where they have access to ideal functionalities. In this model, protocol execution may additionally involve oracle calls to ideal functionalities. Similarly to the ideal world, the adversary controls the routing of communication between the honest party and any hybrid functionalities (but cannot read or modify the contents of these messages).

Formally, we write $\Pi[F_1, \dots, F_k]$ to denote that protocol Π operates in a hybrid model with access to ideal functionalities $F_1, \dots, F_k$. Standard composition theorems [Can01, CCL15] guarantee that if $\Pi[F]$ securely realizes some functionality F' in the F -hybrid model, and ρ securely realizes F , then the composed protocol $\Pi[\rho]$ (where all calls to F are replaced by executions of ρ) securely realizes F' .

Distinguishing advantage. The distinguishing party in UC is called the environment (Env), which controls all possible external contexts in which a protocol might be executed. The environment chooses the inputs for both parties and sees the output of both parties. Crucially, the adversary can send messages to the environment during protocol execution, so the simulator must produce a consistent intermediate view without rewinding or forking.

We say that protocol Π UC-realizes functionality F if for every adversary A against Π , there exists a simulator S such that no efficient environment Env can distinguish between the two executions. We define that formally below.

Definition 1 (Distinguishing Advantage for UC Security of Two-Party Protocols). *Let $\text{REAL}_{\text{Env}, \text{C}, \Pi, \text{A}}^{2\text{pc}, b}$ and $\text{IDEAL}_{\text{Env}, \text{C}, \text{F}, \text{S}}^{2\text{pc}, b}$ be the probabilistic games defined in Figure 11. The distinguishing advantage of Env between a real execution of protocol Π with A*

and an ideal execution of functionality F with simulator S when party P_b is corrupted, is defined as:

$$\text{Adv}_{\text{Env}, C, \Pi, A, F, S}^{2\text{pc}, b}(\lambda) = |\Pr[\text{REAL}_{\text{Env}, C, \Pi, A}^{2\text{pc}, b}(\lambda) = 1] - \Pr[\text{IDEAL}_{\text{Env}, C, F, S}^{2\text{pc}, b}(\lambda) = 1]|$$

Definition 2 (UC Security for Two-Party Protocols). A two-party protocol Π UC-realizes an ideal functionality F if for all $b \in \{0, 1\}$, for all probabilistic polynomial time (PPT) control functions C for the honest party P_{1-b} , and for all PPT adversaries A corrupting party P_b , there exists a PPT simulator S such that for all PPT environments Env :

$$\text{Adv}_{\text{Env}, C, \Pi, A, F, S}^{2\text{pc}, b}(\lambda) \leq \text{negl}(\lambda)$$

where $\text{negl}(\lambda)$ is a negligible function in the UC security parameter of the protocol λ .

B Building Block Protocols

B.1 Commitment Functionality and Protocol

In Figure 12 we describe the ideal functionality and a hash-and-reveal protocol for a hiding and binding commitment scheme. This scheme and functionality appear in Appendix A of [DKL⁺13], where it is also proved UC-secure in the random oracle model [BR93].

<u>Functionality F_{Comm}</u> <u>Init(λ):</u> Initialize $T \leftarrow \{\cdot\}$ mapping id to committed values. <u>Comm(id, v) from P_b:</u> 1. Require $\text{id} \notin T$. 2. Store $T[\text{id}] \leftarrow (b, v)$. 3. Send $(\text{comm}, \text{id}, b)$ to all parties. <u>Open(id) from P_b:</u> 1. Require $\text{id} \in T$ and $(b', v) \leftarrow T[\text{id}]$; $b' = b$. 2. Send $(\text{open}, \text{id}, v)$ to all parties.	<u>Protocol HComm[H]</u> <u>Init(λ):</u> 1. Initialize $T \leftarrow \{\cdot\}$ mapping id to commitment material. 2. Initialize $C \leftarrow \{\cdot\}$ mapping id to received commitments. <u>P_b.Comm(id, v) $\leftrightarrow$ P_{1-b}.RcvComm():</u> 1. Require $\text{id} \notin T \cup C$. 2. Sample $r \leftarrow \{0, 1\}^\lambda$ and compute $cm \leftarrow H(v \parallel r)$. 3. $T[\text{id}] \leftarrow (v, r, cm)$. 4. P_b sends (id, cm) to P_{1-b} who stores $C[\text{id}] \leftarrow cm$. 5. Output $(\text{comm}, \text{id}, b)$. <u>$P_b$.Open(id) $\leftrightarrow$ P_{1-b}.VerOpen(id):</u> 1. Require $\text{id} \in T$ and $\text{id} \in C$. 2. P_b reads $(v, r, cm) \leftarrow T[\text{id}]$ and sends (id, v, r). 3. P_{1-b} reads $cm \leftarrow C[\text{id}]$ and checks $cm = H(v \parallel r)$. If the check fails, P_{1-b} aborts. 4. Output $(\text{open}, \text{id}, v)$.
---	--

Figure 12: Commitment functionality F_{Comm} and hash and reveal protocol HComm[H] where hash function H is modeled as a random oracle.

B.2 SPDZ Protocol for Arithmetic Circuits

For completeness, we present a description of the SPDZ protocol for malicious-secure computation of arithmetic circuits over a finite field $\mathbb{F}$ that is of exponential size in the security parameter [DPSZ12, DKL⁺13]. The ideal functionality $F_{\text{ArithCirc}}$ for arithmetic circuit computation is given in Figure 13, the SPDZ protocol realizing this functionality is given in Figure 14, and the preprocessing functionality and authentication check subprotocol are given in Figure 15. We briefly describe the key components of SPDZ below.

Inspiration: Semi-honest MPC. The inspiration for SPDZ is the following method to compute arithmetic circuits in the semi-honest setting. At each point in the circuit, parties hold additive shares $\llbracket x \rrbracket_0$ and $\llbracket x \rrbracket_1$ such that $x = \llbracket x \rrbracket_0 + \llbracket x \rrbracket_1$ for the value x of each wire. Each party can input a value x by giving the other party an additive share, and parties can locally evaluate addition and scalar multiplication gates on secret shared data without interaction. To compute a multiplication gate on inputs v_1 and v_2 , parties use preprocessed Beaver triples [Bea91]: each party P_b holds input shares $\llbracket v_1 \rrbracket_b$ and $\llbracket v_2 \rrbracket_b$, and consumes shares of a triple $(\llbracket a \rrbracket_b, \llbracket b \rrbracket_b, \llbracket c \rrbracket_b)$ satisfying $c = ab$, with a and b uniformly random and unknown to either party. Each party computes

and sends $\llbracket d \rrbracket_b = \llbracket v_1 \rrbracket_b - \llbracket a \rrbracket_b$ and $\llbracket e \rrbracket_b = \llbracket v_2 \rrbracket_b - \llbracket b \rrbracket_b$ to the other party, which together open $d = v_1 - a$ and $e = v_2 - b$. Each party then locally computes their share of the product as $\llbracket v_1 \cdot v_2 \rrbracket_b = \llbracket c \rrbracket_b + d \cdot \llbracket b \rrbracket_b + e \cdot \llbracket a \rrbracket_b + d \cdot e/2$. To open a value, parties send their shares to each other and sum them using a commit and reveal protocol.

The efficiency of this protocol relies on the ability to generate Beaver triples, which is a well studied problem (e.g. [DPSZ12, DKL⁺13, KOS16, KPR18]) and is beyond the scope of this overview. However, a key observation is that the triples can be generated in an offline preprocessing phase before the inputs are known, allowing the online phase to be efficient.

SPDZ: Arithmetic circuits in the malicious setting. The SPDZ protocol lifts the semi-honest protocol to the malicious setting by adding authentication tags to all secret-shared values. At each point in the circuit, parties hold *authenticated shares*: each party P_b holds an additive share $\llbracket x \rrbracket_b$ of a secret value x together with a share $\llbracket \gamma x \rrbracket_b$ of the authentication tag γx , where $\gamma = \llbracket \gamma \rrbracket_0 + \llbracket \gamma \rrbracket_1$ is a global authentication key unknown to either party individually. We denote the pair $\langle x \rangle_b = (\llbracket x \rrbracket_b, \llbracket \gamma x \rrbracket_b)$ as an authenticated share.

The preprocessing functionality F_{SPDZPrep} (Figure 15, left) generates shares of a random authentication key $\llbracket \gamma \rrbracket_b$ for each party, authenticated Beaver triples $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ where $c = ab$, and input tuples of the form $(r, \langle r \rangle)$ where r is known to one of the parties and is secret shared between both.

To input a value x , party P_b sends $\epsilon = x - r$ to P_{1-b} , and both parties locally compute $\langle x \rangle = \langle r \rangle + \epsilon$. From there, evaluating the circuit follows the same structure as the semi-honest protocol, but operates over authenticated shares. Linear operations are performed locally on both the data shares and tag shares. To compute a multiplication gate on inputs v_1 and v_2 , each party P_b holds authenticated input shares $\langle v_1 \rangle_b$ and $\langle v_2 \rangle_b$, and consumes an authenticated triple $(\langle a \rangle_b, \langle b \rangle_b, \langle c \rangle_b)$. Each party computes and sends $\llbracket d \rrbracket_b = \llbracket v_1 \rrbracket_b - \llbracket a \rrbracket_b$ and $\llbracket e \rrbracket_b = \llbracket v_2 \rrbracket_b - \llbracket b \rrbracket_b$ to open d and e . Each party then locally computes their authenticated share of the product as $\langle v_1 \cdot v_2 \rangle_b = \langle c \rangle_b + d \cdot \langle b \rangle_b + e \cdot \langle a \rangle_b + d \cdot e/2$.

To reconstruct a value z , parties must first verify that all previously opened values (the d and e values from multiplications) have not been tampered with. Verification uses the authentication check Auth (Figure 15, right): given opened values $z_1, \dots, z_n$ and their tag shares $\llbracket \gamma z_i \rrbracket_b$, parties check that the tags are consistent with the values and the secret key γ , without revealing γ . Concretely, parties jointly sample a random coefficient r via committed coin-tossing, compute the random linear combination $\ell = \sum_i r^{i-1} z_i$ and tag share $\llbracket \gamma \ell \rrbracket_b = \sum_i r^{i-1} \llbracket \gamma z_i \rrbracket_b$, then each party computes $\llbracket \epsilon \rrbracket_b = \llbracket \gamma \ell \rrbracket_b - \llbracket \gamma \rrbracket_b \cdot \ell$ and parties verify $\llbracket \epsilon \rrbracket_0 + \llbracket \epsilon \rrbracket_1 = 0$ by exchanging committed shares. If any value was tampered with, the check fails with overwhelming probability, and both parties must abort the protocol. After the pending values pass verification, parties exchange shares $\llbracket z \rrbracket_b$ to reconstruct z , then run Auth again to verify z itself before concluding that it is the correct output. We point out that a circuit may contain many reconstruct gates, and evaluation can continue after this step.

We refer the reader to Figure 14 for the details of the SPDZ protocol.

Pitfall: MAC key leakage via failed checks. Kyser et al. [KNOS25] identify a vulnerability in SPDZ implementations that use parallelism or error recovery. They observe that the authentication check only hides γ when it *succeeds*. However, at the cost of failing the check, an adversary can learn the authentication key. Consider an adversary that sends an incorrect share $\llbracket x \rrbracket'_b = \llbracket x \rrbracket_b + \delta$ when opening x , causing parties to reconstruct $x' = x + \delta$. In the authentication check, after the honest party opens their commitment to $\llbracket \epsilon \rrbracket_{1-b}$, the adversary can compute $\llbracket \epsilon \rrbracket_b + \llbracket \epsilon \rrbracket_{1-b} = \gamma x - \gamma x' = -\gamma \delta$ and recover γ .

This cannot be exploited in sequential execution since the adversary learns γ only after the check fails. However, with multi-threading or error recovery, the adversary can learn γ in one context and exploit it to forge MACs in another. To fix this, Kyser et al. propose masking ϵ with a random value before opening: if $\epsilon = 0$ the masked value remains zero, otherwise it is randomized and reveals nothing about γ .

Both this vulnerability and the fix apply to our protocol FLOSS as well, both in the blind and non-blind authentication checks. However, since it is orthogonal to our contribution, we do not discuss it further.

C Secret Guessing Attack using Blind Authentication Checks

In this appendix, we expand on the input-guessing selective failure attack introduced in Section 3, describe its application to concrete protocols, and discuss mitigations.

Attack Mechanics. Recall from Section 3 that the input-guessing attack relies on accessing an oracle that checks whether authenticated shares are valid without revealing the underlying secret or authentication key. Concretely, such an oracle takes as input from each party P_i their shares of a secret $\llbracket x \rrbracket_i$, their shares of the authentication tag $\llbracket \gamma x \rrbracket_i$, and their shares of the authentication key $\llbracket \gamma \rrbracket_i$. The oracle reconstructs the secret $x = \llbracket x \rrbracket_0 + \llbracket x \rrbracket_1$, the tag $\gamma x = \llbracket \gamma x \rrbracket_0 + \llbracket \gamma x \rrbracket_1$, and the key $\gamma = \llbracket \gamma \rrbracket_0 + \llbracket \gamma \rrbracket_1$, then returns 1 if the authentication relation $\gamma \cdot x = \gamma x$ holds, and 0 otherwise, without revealing any more information about the secret values.

<u>Functionality $F_{\text{ArithCirc}}[\mathbb{F}]$</u> <u>Init($\lambda$):</u> 1. $T \leftarrow \{\cdot\}$: maps id to values in $\mathbb{F}$. 2. $ictr_0 \leftarrow 0$; $ictr_1 \leftarrow 0$: input operation counter. 3. $mctr \leftarrow 0$: multiplication operation counter. 4. $prep \leftarrow 0$: preprocessing flag. <u>Prep($n_{i,0}, n_{i,1}, n_m$) from P_b:</u> 1. Require $prep = 0$, preprocessing only called once. 2. If no pending message from P_{1-b}, set $(prep, n_{i,0}, n_{i,1}, n_m)$ from P_b as pending and return. Else parse $(op, n'_{i,0}, n'_{i,1}, n'_m)$ from P_{1-b} and require $op = prep$. 3. If mismatch, $\exists b', n_{i,b} \neq n'_{i,b}$ or $n_m \neq n'_m$, send abort to all parties. 4. Else, $ictr_b \leftarrow n_{i,b}$ and $mctr \leftarrow n_m$, $prep \leftarrow 1$. 5. Send $(prep, n_{i,0}, n_{i,1}, n_m)$ to all parties. <u>ScalAdd($id_1, \alpha \in \mathbb{F}, id_2$) from P_b:</u> 1. Require $id_1 \in T$, $id_2 \notin T$, and no pending message from P_b. 2. If no pending message from P_{1-b}, set $(scadd, id_1, \alpha, id_2)$ from P_b as pending and return. Else parse $(op, id'_1, \alpha', id'_2)$ from P_{1-b} and require $op = scadd$. 3. If mismatch, $\forall_{j \in \{1,2\}} id_j \neq id'_j$, or $\alpha \neq \alpha'$, store $T[id_2], T[id'_2] \leftarrow \perp$. 4. Else, store $T[id_2] \leftarrow T[id_1] + \alpha$. 5. Send $(scadd, id_1, \alpha, id_2)$ to P_b and $(scadd, id'_1, \alpha', id'_2)$ to P_{1-b}. <u>Add(id_1, id_2, id_3) from P_b:</u> 1. Require $id_1, id_2 \in T$, $id_3 \notin T$, and no pending message from P_b. 2. If no pending message from P_{1-b}, set (add, id_1, id_2, id_3) from P_b as pending and return. Else parse $(op, id'_1, id'_2, id'_3)$ from P_{1-b} and require $op = add$. 3. If mismatch, $\forall_{j \in \{1,2,3\}} id_j \neq id'_j$, store $T[id_3], T[id'_3] \leftarrow \perp$. 4. Else, store $T[id_3] \leftarrow T[id_1] + T[id_2]$. 5. Send (add, id_1, id_2, id_3) to P_b and $(add, id'_1, id'_2, id'_3)$ to P_{1-b}.	<u>Input($id, v \in \mathbb{F}$) from P_b:</u> 1. Require $id \notin T$ and $ictr_b > 0$. 2. Store $T[id] \leftarrow v$. 3. Decrement $ictr_b \leftarrow ictr_b - 1$. 4. Send (inp, id) to all parties. <u>Reconstruct(id) from P_b:</u> 1. Require $id \in T$. 2. If no pending message from P_{1-b}, set $(reconstruct, id)$ from P_b as pending and return. Else parse (op, id') from P_{1-b} and require $op = reconstruct$. 3. If $id \neq id'$ or $T[id] = \perp$, send abort to all parties. 4. Else, send $(reconstruct, id, T[id])$ to all parties. <u>ScalMul($id_1, \alpha \in \mathbb{F}, id_2$) from P_b:</u> 1. Require $id_1 \in T$, $id_2 \notin T$, and no pending message from P_b. 2. If no pending message from P_{1-b}, set $(scmul, id_1, \alpha, id_2)$ from P_b as pending and return. Else parse $(op, id'_1, \alpha', id'_2)$ from P_{1-b} and require $op = scmul$. 3. If mismatch, $\forall_{j \in \{1,2\}} id_j \neq id'_j$, or $\alpha \neq \alpha'$, store $T[id_2], T[id'_2] \leftarrow \perp$. 4. Else, store $T[id_2] \leftarrow \alpha T[id_1]$. 5. Send $(scmul, id_1, \alpha, id_2)$ to P_b and $(scmul, id'_1, \alpha', id'_2)$ to P_{1-b}. <u>Mul(id_1, id_2, id_3) from P_b:</u> 1. Require $id_1, id_2 \in T$, $id_3 \notin T$, $mctr > 0$, and no pending message from P_b. 2. If no pending message from P_{1-b}, set (mul, id_1, id_2, id_3) from P_b as pending and return. Else parse $(op, id'_1, id'_2, id'_3)$ from P_{1-b} and require $op = mul$. 3. If mismatch, $\forall_{j \in \{1,2,3\}} id_j \neq id'_j$, store $T[id_3], T[id'_3] \leftarrow \perp$. 4. Else, store $T[id_3] \leftarrow T[id_1] \cdot T[id_2]$. 5. Decrement $mctr \leftarrow mctr - 1$. 6. Send (mul, id_1, id_2, id_3) to P_b and $(mul, id'_1, id'_2, id'_3)$ to P_{1-b}.
--	--

Figure 13: Functionality for computation of arithmetic circuits over $\mathbb{F}$.

Protocol SPDZ $[\mathbb{F}, \text{FSPDZPrep}, \text{FComm}]$	
<u>$P_b.\text{Init}(\lambda)$:</u>	<u>$P_b.\text{Input}(\text{id}, v \in \mathbb{F}) \leftrightarrow P_{1-b}.\text{RcvInput}(\text{id})$:</u>
1. $T \leftarrow \{\cdot\}$: maps id to shares in $\mathbb{F}$.	1. Require $\text{id} \notin T$ and $L_{\text{inp},b} \neq [\cdot]$.
2. $L_{\text{inp},0}, L_{\text{inp},1} \leftarrow [\cdot]$: list of preprocessed input tuples.	2. P_b pops $(r, \langle r \rangle_b) \leftarrow L_{\text{inp},b}$ and P_{1-b} pops $\langle r \rangle_{1-b} \leftarrow L_{\text{inp},b}$.
3. $L_{\text{mul}} \leftarrow [\cdot]$: list of preprocessed multiplication tuples.	3. P_b computes $\epsilon \leftarrow v - r$ and sends ϵ to P_{1-b} .
4. $L_{\text{auth}} \leftarrow [\cdot]$: list of pending authentication checks.	4. P_0 and P_1 with $(\llbracket r \rrbracket, \llbracket \gamma r \rrbracket) \leftarrow \langle r \rangle$ compute: $\langle v \rangle = (\llbracket v \rrbracket, \llbracket \gamma v \rrbracket) \leftarrow (\llbracket r \rrbracket + \epsilon, \llbracket \gamma r \rrbracket + \llbracket \gamma \rrbracket \cdot \epsilon)$
5. $\llbracket \gamma \rrbracket_b \leftarrow \perp$: share of authentication key.	5. P_0 and P_1 store $T[\text{id}] \leftarrow \langle v \rangle$.
<u>$P_0.\text{Prep}(n_{\text{inp},0}, n_{\text{inp},1}, n_m) \leftrightarrow P_1.\text{Prep}(n_{\text{inp},0}, n_{\text{inp},1}, n_m)$:</u>	6. Output (inp, id).
1. P_0 and P_1 invoke $\text{FSPDZPrep}(n_{\text{inp},0}, n_{\text{inp},1}, n_m)$ for each P_b to receive (see Fig. 15):	<u>$P_0.\text{Reconstruct}(\text{id}) \leftrightarrow P_1.\text{Reconstruct}(\text{id})$:</u>
• $L_{\text{inp},b} \leftarrow [(r_{i,b}, \langle r_{i,b} \rangle_b = (\llbracket r_{i,b} \rrbracket_b, \llbracket \gamma r_{i,b} \rrbracket_b))_i]^{n_{\text{inp},b}}$	1. Require $\text{id} \in T$.
• $L_{\text{inp},1-b} \leftarrow [(\langle r_{i,1-b} \rangle_b = (\llbracket r_{i,1-b} \rrbracket_b, \llbracket \gamma r_{i,1-b} \rrbracket_b))_i]^{n_{\text{inp},1-b}}$	2. P_0 and P_1 run authentication check on all pending multiplication opens in L_{auth} (see Fig. 15):
• $L_{\text{mul}} \leftarrow [(\langle a_i \rangle_b, \langle b_i \rangle_b, \langle c_i \rangle_b)_i]^{n_m}$	(a) $[(z_i, \llbracket \gamma z_i \rrbracket)_i] \leftarrow L_{\text{auth}}; L_{\text{auth}} \leftarrow [\cdot]$
• $\llbracket \gamma \rrbracket_b$	(b) $P_0.\text{Auth}(\llbracket \gamma \rrbracket, [(z_i, \llbracket \gamma z_i \rrbracket)_i]) \leftrightarrow P_1.\text{Auth}(\llbracket \gamma \rrbracket, [(z_i, \llbracket \gamma z_i \rrbracket)_i])$
2. Output (prep, n_i, n_m).	3. P_0 and P_1 open $\langle v \rangle \leftarrow T[\text{id}]$ to v by sending $\llbracket v \rrbracket$ to each other.
<u>$P_b.\text{ScalAdd}(\text{id}_1, \alpha \in \mathbb{F}, \text{id}_2)$:</u>	4. P_0 and P_1 run authentication check on reconstructed value: $P_0.\text{Auth}(\llbracket \gamma \rrbracket, [(v, \llbracket \gamma v \rrbracket)]) \leftrightarrow P_1.\text{Auth}(\llbracket \gamma \rrbracket, [(v, \llbracket \gamma v \rrbracket)])$
1. Require $\text{id}_1 \in T, \text{id}_2 \notin T$.	5. Output ($\text{reconstruct}, \text{id}, v$).
2. P_b with $(\llbracket v_1 \rrbracket, \llbracket \gamma v_1 \rrbracket) \leftarrow T[\text{id}_1]$ computes: $\langle v_2 \rangle = (\llbracket v_2 \rrbracket, \llbracket \gamma v_2 \rrbracket) \leftarrow (\llbracket v_1 \rrbracket + \alpha/2, \llbracket \gamma v_1 \rrbracket + \llbracket \gamma \rrbracket \cdot \alpha/2)$	<u>$P_0.\text{Mul}(\text{id}_1, \text{id}_2, \text{id}_3) \leftrightarrow P_1.\text{Mul}(\text{id}_1, \text{id}_2, \text{id}_3)$:</u>
3. P_b stores $T[\text{id}_2] \leftarrow \langle v_2 \rangle$.	1. Require $\text{id}_1, \text{id}_2 \in T, \text{id}_3 \notin T$, and $L_{\text{mul}} \neq [\cdot]$.
4. Output ($\text{scadd}, \text{id}_1, \alpha, \text{id}_2$).	2. P_0 and P_1 pop $(\langle a \rangle, \langle b \rangle, \langle c \rangle) \leftarrow L_{\text{mul}}$.
<u>$P_b.\text{Add}(\text{id}_1, \text{id}_2, \text{id}_3)$:</u>	3. P_0 and P_1 with $\langle v_1 \rangle \leftarrow T[\text{id}_1]$ and $\langle v_2 \rangle \leftarrow T[\text{id}_2]$ compute: $\langle d \rangle \leftarrow \langle v_1 \rangle - \langle a \rangle \quad \langle e \rangle \leftarrow \langle v_2 \rangle - \langle b \rangle$
1. Require $\text{id}_1, \text{id}_2 \in T, \text{id}_3 \notin T$.	4. P_0 and P_1 open d and e by sending $\llbracket d \rrbracket$ and $\llbracket e \rrbracket$ to each other.
2. P_b with $(\llbracket v_1 \rrbracket, \llbracket \gamma v_1 \rrbracket) \leftarrow T[\text{id}_1]$ and $(\llbracket v_2 \rrbracket, \llbracket \gamma v_2 \rrbracket) \leftarrow T[\text{id}_2]$ computes: $\langle v_3 \rangle = (\llbracket v_3 \rrbracket, \llbracket \gamma v_3 \rrbracket) \leftarrow (\llbracket v_1 \rrbracket + \llbracket v_2 \rrbracket, \llbracket \gamma v_1 \rrbracket + \llbracket \gamma v_2 \rrbracket)$	5. P_0 and P_1 compute: $\langle v_3 \rangle \leftarrow \langle c \rangle + d \cdot \langle b \rangle + e \cdot \langle a \rangle + d \cdot e$
3. P_b stores $T[\text{id}_3] \leftarrow \langle v_3 \rangle$.	6. P_0 and P_1 store $T[\text{id}_3] \leftarrow \langle v_3 \rangle$.
4. Output ($\text{add}, \text{id}_1, \text{id}_2, \text{id}_3$).	7. P_0 and P_1 push $(d, \llbracket \gamma d \rrbracket)$ and $(e, \llbracket \gamma e \rrbracket)$ to L_{auth} for authentication check.
<u>$P_b.\text{ScalMul}(\text{id}_1, \alpha \in \mathbb{F}, \text{id}_2)$:</u>	8. Output ($\text{mul}, \text{id}_1, \text{id}_2, \text{id}_3$).
1. Require $\text{id}_1 \in T, \text{id}_2 \notin T$.	
2. P_b with $(\llbracket v_1 \rrbracket, \llbracket \gamma v_1 \rrbracket) \leftarrow T[\text{id}_1]$ computes: $\langle v_2 \rangle = (\llbracket v_2 \rrbracket, \llbracket \gamma v_2 \rrbracket) \leftarrow (\llbracket v_1 \rrbracket \cdot \alpha, \llbracket \gamma v_1 \rrbracket \cdot \alpha)$	
3. P_b stores $T[\text{id}_2] \leftarrow \langle v_2 \rangle$.	
4. Output ($\text{scmul}, \text{id}_1, \alpha, \text{id}_2$).	

Figure 14: SPDZ [DPSZ12, DKL⁺13] protocol for secure computation of arithmetic circuits over $\mathbb{F}$.

Functionality $F_{\text{SPDZPrep}}(\mathbb{F})$ Main($n_{i,0}, n_{i,1}, n_m$) from P_b: 1. If no pending message from P_{1-b}, set $(n_{i,0}, n_{i,1}, n_m)$ from P_b as pending and return. Else parse $(n'_{i,0}, n'_{i,1}, n'_m)$ from P_{1-b}. 2. If mismatch, $n_{i,b} \neq n'_{i,b}$ or $n_m \neq n'_m$, send abort to all parties. 3. Sample shares of random authentication key $\llbracket \gamma \rrbracket$: (a) Sample $\gamma \leftarrow \mathbb{F}$ (b) $\llbracket \gamma \rrbracket_0 \leftarrow \mathbb{F}$ (c) $\llbracket \gamma \rrbracket_1 \leftarrow \gamma - \llbracket \gamma \rrbracket_0$ 4. Compute input tuples for each $P_{b'}$: • $L_{\text{inp},b',b'} \leftarrow [(r_{i,b'}, \langle r_{i,b'} \rangle_{b'} = (\llbracket r_{i,b'} \rrbracket_{b'}, \llbracket \gamma r_{i,b'} \rrbracket_{b'}))]_i^{n_{i,b'}}$ • $L_{\text{inp},1-b',b'} \leftarrow [(\langle r_{i,b'} \rangle_{1-b'} = (\llbracket r_{i,b'} \rrbracket_{1-b'}, \llbracket \gamma r_{i,b'} \rrbracket_{1-b'}))]_i^{n_{i,b'}}$ (a) Sample $r_{i,b'} \leftarrow \mathbb{F}$ (b) $\llbracket r_{i,b'} \rrbracket_{b'} \leftarrow \mathbb{F}$, $\llbracket r_{i,b'} \rrbracket_{1-b'} \leftarrow r_{i,b'} - \llbracket r_{i,b'} \rrbracket_{b'}$ (c) $\llbracket \gamma r_{i,b'} \rrbracket_{b'} \leftarrow \mathbb{F}$, $\llbracket \gamma r_{i,b'} \rrbracket_{1-b'} \leftarrow \gamma r_{i,b'} - \llbracket \gamma r_{i,b'} \rrbracket_{b'}$ 5. Compute multiplication tuples for each $P_{b'}$: • $L_{\text{mul},b'} \leftarrow [(\langle a_i \rangle_{b'}, \langle b_i \rangle_{b'}, \langle c_i \rangle_{b'})]_i^{n_m}$ (a) Sample $a_i, b_i \leftarrow \mathbb{F}$ and set $c_i \leftarrow a_i \cdot b_i$ (b) $\langle a_i \rangle_{b'} \leftarrow \mathbb{F}^2$, $\langle a_i \rangle_{1-b'} \leftarrow (a_i - \llbracket a_i \rrbracket_{b'}, \gamma a_i - \llbracket \gamma a_i \rrbracket_{b'})$ (c) $\langle b_i \rangle_{b'} \leftarrow \mathbb{F}^2$, $\langle b_i \rangle_{1-b'} \leftarrow (b_i - \llbracket b_i \rrbracket_{b'}, \gamma b_i - \llbracket \gamma b_i \rrbracket_{b'})$ (d) $\langle c_i \rangle_{b'} \leftarrow \mathbb{F}^2$, $\langle c_i \rangle_{1-b'} \leftarrow (c_i - \llbracket c_i \rrbracket_{b'}, \gamma c_i - \llbracket \gamma c_i \rrbracket_{b'})$ 6. Send • $(\llbracket \gamma \rrbracket_0, L_{\text{inp},0,0}, L_{\text{inp},0,1}, L_{\text{mul},0})$ to P_0. • $(\llbracket \gamma \rrbracket_1, L_{\text{inp},1,0}, L_{\text{inp},1,1}, L_{\text{mul},1})$ to P_1.	$P_0.\text{Auth}(\llbracket \gamma \rrbracket_0, [(z_i, \llbracket \gamma z_i \rrbracket_0)]_i^n) \leftrightarrow P_1.\text{Auth}(\llbracket \gamma \rrbracket_1, [(z_i, \llbracket \gamma z_i \rrbracket_1)]_i^n)$ 1. P_0 and P_1 compute linear combination of inputs: $\ell, \llbracket \gamma \ell \rrbracket$ (a) P_0 and P_1 each sample $r_b \leftarrow \mathbb{F}$. (b) P_0 and P_1 each commit to r_b by invoking $F_{\text{Comm}}.\text{Comm}$. (c) P_0 and P_1 open commitments to receive r_{1-b} from each other via $F_{\text{Comm}}.\text{Open}$. (d) P_0 and P_1 compute $r \leftarrow r_0 + r_1$. (e) P_0 and P_1 compute linear combination: $\ell \leftarrow \sum_{i=1}^n r^{i-1} \cdot z_i \quad \llbracket \gamma \ell \rrbracket \leftarrow \sum_{i=1}^n r^{i-1} \cdot \llbracket \gamma z_i \rrbracket$ 2. P_0 and P_1 check $\llbracket \gamma \ell \rrbracket$ against ℓ: (a) P_0 and P_1 compute $\llbracket \epsilon \rrbracket \leftarrow \llbracket \gamma \ell \rrbracket - \llbracket \gamma \rrbracket \cdot \ell$. (b) P_0 and P_1 each commit to $\llbracket \epsilon \rrbracket_b$ by invoking $F_{\text{Comm}}.\text{Comm}$. (c) P_0 and P_1 open commitments to receive $\llbracket \epsilon \rrbracket_{1-b}$ from each other via $F_{\text{Comm}}.\text{Open}$. (d) P_0 and P_1 output abort if: $\llbracket \epsilon \rrbracket_0 + \llbracket \epsilon \rrbracket_1 \neq 0$
---	---

Figure 15: (Left) Functionality of preprocessing needed for the SPDZ [DPSZ12, DKL⁺13] protocol for secure computation of arithmetic circuits over $\mathbb{F}$. (Right) The authentication check subprotocol used in SPDZ for checking opened data values against authentication shares, in the F_{Comm} -hybrid model.

Suppose a malicious P_0 holds SPDZ shares $(\llbracket \vec{x} \rrbracket_0, \llbracket \gamma \vec{x} \rrbracket_0, \llbracket \gamma \rrbracket_0)$ and wishes to test whether $\vec{x} = \llbracket \vec{x} \rrbracket_0 + \llbracket \vec{x} \rrbracket_1$ equals some guess $\vec{x}'$. To do so, P_0 provides perturbed shares to the oracle: the data share $\llbracket \vec{x} \rrbracket_0$ unchanged, the authentication share perturbed to $\llbracket \gamma \vec{x} \rrbracket_0 + \vec{x}'$, and the key share perturbed to $\llbracket \gamma \rrbracket_0 + 1$. The oracle then checks:

$$(\llbracket \gamma \rrbracket_0 + 1 + \llbracket \gamma \rrbracket_1) \cdot (\llbracket \vec{x} \rrbracket_0 + \llbracket \vec{x} \rrbracket_1) \stackrel{?}{=} (\llbracket \gamma \vec{x} \rrbracket_0 + \vec{x}') + \llbracket \gamma \vec{x} \rrbracket_1$$

Substituting $\gamma = \llbracket \gamma \rrbracket_0 + \llbracket \gamma \rrbracket_1$ and $\vec{x} = \llbracket \vec{x} \rrbracket_0 + \llbracket \vec{x} \rrbracket_1$, this simplifies to $(\gamma + 1) \cdot \vec{x} = \gamma \vec{x} + \vec{x}'$, which holds if and only if $\vec{x} = \vec{x}'$. The oracle's response thus reveals whether the guess was correct.

The power of this attack depends on the oracle's interface. An oracle that accepts individual authenticated shares $\langle x \rangle$ allows the adversary to test guesses on single secret values. In contrast, an oracle that accepts a vector $\langle \vec{x} \rangle$ and returns whether *all* shares are simultaneously valid, which is more common in shuffling protocols, only allows testing a single guess on the entire vector per query.

Mitigation via γ -Bound Preprocessing. The blind authentication check protocol (Figure 1) does not directly provide the oracle described above. The protocol relies on preprocessed material, in particular input tuples, that are bound to the authentication key γ . If a malicious P_0 attempts to run the blind authentication protocol with a perturbed key share $\llbracket \gamma \rrbracket_0 + 1$, while using preprocessing material generated for the original key γ , the internal computations will be inconsistent and the check will fail regardless of whether the guess is correct.

This observation suggests a natural mitigation: restrict all preprocessing to occur in a single phase at the beginning of the protocol, which enforces that all required material (multiplication tuples, input tuples, etc.) is generated with respect to a single authentication key γ . With γ -bound preprocessing, a malicious party cannot introduce material consistent with a perturbed key, and thus the blind authentication check does not instantiate the input-guessing oracle.

Both our protocol and the protocol of Song et al. [SYB⁺24] are designed with this mitigation in mind. In our security definitions, we explicitly model preprocessing as occurring once at initialization, ensuring all material is γ -bound. This is also critical for our security proof (Lemma 4 explicitly assumes that the input tuples used are γ -bound).

Pitfall: On-Demand Preprocessing Re-Enables the Attack. While γ -bound preprocessing prevents the attack in principle, practical SPDZ implementations commonly support *on-demand* preprocessing, where new batches of preprocessing material are generated when needed during protocol execution. This is often the case in MP-SPDZ [Kel20], for example, and is a natural design choice when the number of needed preprocessing tuples is not known in advance or is too large to generate at once. However, on-demand preprocessing implementations typically do not verify that newly generated material is bound to the *same* authentication key γ as previously generated material.

On-demand preprocessing with potentially perturbed keys does not compromise the security of standard SPDZ, since introducing such inconsistencies is not helpful in revealing secrets. However, once blind authentication checks are introduced, on-demand preprocessing allows an adversary to bypass the γ -bound preprocessing restriction, which re-opens the door to the input-guessing attack. This pitfall is particularly relevant in our threat model, where the adversary controls which circuits are evaluated. A malicious party can easily trigger a situation requiring additional preprocessing, for example by requesting evaluation of a circuit that exceeds available material, and then supply perturbed key shares during the subsequent preprocessing phase.

We now show how a malicious P_0 can exploit on-demand preprocessing to successfully execute an input-guessing attack via the blind authentication check protocol (Figure 1). Suppose P_0 wishes to test whether $\vec{z} = \vec{z}'$ for some guess $\vec{z}'$. The attack proceeds as follows:

1. P_0 triggers an on-demand preprocessing phase to generate preprocessing materials used in generating the blinding factor α , providing perturbed key share $\llbracket \gamma \rrbracket_0 + 1$ (instead of $\llbracket \gamma \rrbracket_0$).
2. The preprocessing functionality reconstructs $\gamma' = (\llbracket \gamma \rrbracket_0 + 1) + \llbracket \gamma \rrbracket_1 = \gamma + 1$ and generates input tuples for α that are authenticated with respect to γ' rather than γ , or equivalently $\llbracket \gamma' \alpha \rrbracket_0 + \llbracket \gamma' \alpha \rrbracket_1 = (\gamma + 1)(\llbracket \alpha \rrbracket_0 + \llbracket \alpha \rrbracket_1)$.
3. Both parties execute the blind authentication check (Figure 1) on data shares $\langle \vec{z} \rangle$, using the perturbed input tuples to construct the blinding factor. However, the adversarial P_0 runs the protocol using perturbed data MAC shares $\llbracket \gamma z_i \rrbracket_0 + z'_i$ and the perturbed key share $\llbracket \gamma \rrbracket_0 + 1$.

We now verify that this attack reveals whether the guess $\vec{z}' = \vec{z}$ is correct. The protocol computes $\ell = \alpha + \sum_i r^{i-1} z_i$ for a jointly-sampled random r , and the reconstructed MAC is:

$$\llbracket \gamma \ell \rrbracket_0 + \llbracket \gamma \ell \rrbracket_1 = \underbrace{(\gamma + 1)\alpha}_{\text{from perturbed } \langle \alpha \rangle} + \underbrace{\gamma \sum_i r^{i-1} z_i + \sum_i r^{i-1} z'_i}_{\text{from } \langle \vec{z} \rangle \text{ with guess added}}$$

The final check verifies $\llbracket \epsilon \rrbracket_0 + \llbracket \epsilon \rrbracket_1 = 0$, where $\llbracket \epsilon \rrbracket_b = \llbracket \gamma \ell \rrbracket_b - \llbracket \gamma \rrbracket_b \cdot \ell$. Since P_0 uses key share $\llbracket \gamma \rrbracket_0 + 1$ and P_1 uses $\llbracket \gamma \rrbracket_1$, the reconstructed key is $\gamma + 1$, and the check becomes:

$$(\gamma + 1)\alpha + \gamma \sum_i r^{i-1} z_i + \sum_i r^{i-1} z'_i \stackrel{?}{=} (\gamma + 1) \cdot \ell = (\gamma + 1)\alpha + (\gamma + 1) \sum_i r^{i-1} z_i$$

which simplifies to $\sum_i r^{i-1} z'_i = \sum_i r^{i-1} z_i$, which holds (with overwhelming probability) if and only if $\vec{z}' = \vec{z}$.

Proof-of-Concept Attack against Song et al. The implementation of Song et al. [SYB⁺24] supports on-demand preprocessing, making it vulnerable to the attack described above. We implemented a proof-of-concept demonstrating that the input-guessing attack can be successfully executed against their prototype; see the Open Science section for code availability.

In our proof-of-concept, the malicious party tests whether the input vector equals the zero vector. The attack proceeds by triggering a shuffle operation, which consequently invokes a blind authentication check on the shuffled output. During this check, the adversary uses perturbed input tuples (generated via on-demand preprocessing with a perturbed key share) and a perturbed MAC key share (note that testing if $\vec{z}' = \vec{0}$ does not require modifying the tag share $\llbracket \gamma \vec{z} \rrbracket_0$). For simplicity, we implemented the attack with a preprocessing batch size of one, where all preprocessing material is generated on-demand exactly when it is needed. The attack extends naturally to larger batch sizes provided the adversary can trigger exhaustion of preprocessing material.

We note that both our protocol specification and implementation avoid this pitfall by enforcing γ -bound preprocessing: all preprocessing occurs in a single initialization phase before any secrets are shared. While this mitigates the vulnerability, it is a restrictive design choice. Below we discuss alternative mitigations that may allow for more flexible preprocessing; these are beyond the scope of this work.

Mitigation Using Double-MACed Secret Sharing. A more robust mitigation is to authenticate the authentication key itself. Recall that the attack relies on perturbing the key share $\llbracket \gamma \rrbracket_0$ without detection. In standard SPDZ, the key γ is stored only as additive shares, with no mechanism to verify that parties use consistent key shares.

This can be addressed by introducing a second random key β and extending the key material to include shares of γ , β , and $\gamma \cdot \beta$, all generated during preprocessing. Authentication checks are then modified to append β to the data vector being verified: instead of checking validity of $\langle \vec{z} \rangle$, parties check $\langle \vec{z} \parallel \beta \rangle$. Since β is uniformly random and unknown to the adversary, any deviation from the expected shares—including perturbations to the key share—causes the check to fail with overwhelming probability, independent of the secret data. This eliminates the potential for selective failure attacks, as the adversary cannot craft perturbations that pass or fail based on the value of $\vec{z}$.

We call this approach double-MACed secret sharing; it prevents the input-guessing attack even with on-demand preprocessing, at the cost of additional key material. Since this mitigation requires changes to the underlying SPDZ protocol itself and its secret sharing scheme, rather than simply adding permutation gates on top of standard SPDZ, full discussion of it is beyond the scope of this work.

We do note, however, that a notable benefit of double-MACed secret sharing is that it allows protocols to treat the blind authentication check as a black box. For example, the check can be implemented as a simple circuit that takes additively secret-shared inputs $\llbracket \vec{z} \parallel \beta \rrbracket$, a vector of secret shared tags $\llbracket y \rrbracket$, and a key γ , and returns whether the expected authentication relation holds for all elements ($\gamma \cdot (\vec{z} \parallel \beta) = y$). In contrast, the γ -bound preprocessing mitigation requires opening the box of the blind authentication check protocol in any security argument, and restricts preprocessing to be non-adaptive.

D Security Proof for Online Protocol

In this section we prove the security of our online protocol FLOSS, assuming the security of the offline protocol to generate shuffle tuples, as well as the security of the commitment scheme used.

Theorem 2 (Security of FLOSS). Let $\mathbb{F}$ be a finite field with $|\mathbb{F}| \geq 2^\lambda$. The protocol $\text{FLOSS}[\mathbb{F}, F_{\text{SPDZPrep}}, F_{\text{ShufTuplePrep}}, F_{\text{Comm}}]$ (Figure 17) UC-realizes the ideal functionality $F_{\text{ArithPermCirc}}[\mathbb{F}]$ (Figure 16) in the $(F_{\text{SPDZPrep}}, F_{\text{ShufTuplePrep}}, F_{\text{Comm}})$ -hybrid model.

Specifically, for all $b \in \{0, 1\}$ and for all PPT adversaries A corrupting party P_b , there exists a PPT simulator S such that for all PPT environments Env the advantage $\text{Adv}_{\text{Env}, C, \text{FLOSS}, A, F_{\text{ArithPermCirc}}, S}^{2\text{pc}, b}(\lambda)$ is negligible in λ .

Proof. Given an adversary A in the real world, we explicitly construct a simulator S in the ideal world in Figure 19. Without loss of generality, we assume that party P_0 is corrupted, i.e., $b = 0$; the case where party P_1 is corrupted is symmetric. The simulator S receives the input in_A from the environment and internally executes A on this input. Whenever A attempts to invoke

<u>Functionality $F_{\text{ArithPermCirc}}[\mathbb{F}]$</u> <u>Init($\lambda$):</u> 1. $T \leftarrow \{\cdot\}$: maps id to values in $\mathbb{F}$. 2. $T_{\pi,0}, T_{\pi,1} \leftarrow \{\cdot\}$: maps id to permutations. 3. $ictr_0 \leftarrow 0$; $ictr_1 \leftarrow 0$: input operation counter. 4. $mctr \leftarrow 0$: multiplication operation counter. 5. $prep \leftarrow 0$: preprocessing flag. <u>Prep($n_{i,0}, n_{i,1}, n_m, \text{in}_\pi$) from P_b:</u> 1. Parse in_π: • $[(\text{id}_{\pi,b,i}, n_{\pi,b,i}, n_{\pi,\text{inv},b,i}, \pi_{b,i} \in \text{Perm}(m_{b,i}))]_i^{n_{\pi,b}}$ • $[(\text{id}_{\pi,1-b,i}, n_{\pi,1-b,i}, n_{\pi,\text{inv},1-b,i})]_i^{n_{\pi,1-b}}$ 2. Require $prep = 0$, preprocessing only called once. 3. If no pending message from P_{1-b}, set $(\text{prep}, n_{i,0}, n_{i,1}, n_m, \text{in}_\pi)$ from P_b as pending and return. Else parse $(\text{op}, n'_{i,0}, n'_{i,1}, n'_m, \text{in}'_\pi)$ from P_{1-b} and require $\text{op} = \text{prep}$. 4. If mismatch, $\exists b', n_{i,b'} \neq n'_{i,b'}, n_m \neq n'_m, \exists i, b', \text{id}_{\pi,b',i} \neq \text{id}'_{\pi,b',i}, n_{\pi,b',i} \neq n'_{\pi,b',i}, n_{\pi,\text{inv},b',i} \neq n'_{\pi,\text{inv},b',i}$, send abort to all parties. 5. Else, • $ictr_b \leftarrow n_{i,b}$ and $mctr \leftarrow n_m, prep \leftarrow 1$. • $\forall i \in [n_{\pi,b'}] T_{\pi,b'}[\text{id}_{\pi,b',i}] \leftarrow (n_{\pi,b',i}, n_{\pi,\text{inv},b',i}, \pi_{b',i}, m_{b',i})$ 6. Send (prep) to all parties. <u>Input($\text{id}, v \in \mathbb{F}$) from P_b</u> Same as $F_{\text{ArithCirc}}[\mathbb{F}]$ (see Fig. 13). <u>Reconstruct(id) from P_b:</u> Same as $F_{\text{ArithCirc}}[\mathbb{F}]$ (see Fig. 13). <u>ScalAdd($\text{id}_1, \alpha \in \mathbb{F}, \text{id}_2$) from P_b</u> Same as $F_{\text{ArithCirc}}[\mathbb{F}]$ (see Fig. 13). <u>Add($\text{id}_1, \text{id}_2, \text{id}_3$) from P_b</u> Same as $F_{\text{ArithCirc}}[\mathbb{F}]$ (see Fig. 13). <u>ScalMul($\text{id}_1, \alpha \in \mathbb{F}, \text{id}_2$) from P_b</u> Same as $F_{\text{ArithCirc}}[\mathbb{F}]$ (see Fig. 13). <u>Mul($\text{id}_1, \text{id}_2, \text{id}_3$) from P_b</u> Same as $F_{\text{ArithCirc}}[\mathbb{F}]$ (see Fig. 13).	<u>Permute($[\text{id}_i]_i^m, [\tilde{\text{id}}_i]_i^m, \text{id}_\pi, b'$) from P_b:</u> 1. Require $\forall i \text{ id}_i \in T, \tilde{\text{id}}_i \notin T$ and no pending message from P_b. 2. Require $\text{id}_\pi \in T_{\pi,b'}$ and $(ctr, \text{invctr}, \pi, m_\pi) \leftarrow T_{\pi,b'}[\text{id}_\pi]$; $ctr > 0, m = m_\pi$. 3. If no pending message from P_{1-b}, set $(\text{perm}, [\text{id}_i]_i^m, [\tilde{\text{id}}_i]_i^m, \text{id}_\pi, b')$ from P_b as pending and return. Else parse $(\text{op}, [\text{id}'_i]_i^m, [\tilde{\text{id}}'_i]_i^m, \text{id}'_\pi, b'')$ from P_{1-b} and require $\text{op} = \text{perm}$. 4. If mismatch, $\exists i, \text{id}_i \neq \text{id}'_i, \tilde{\text{id}}_i \neq \tilde{\text{id}}'_i, \text{id}_\pi \neq \text{id}'_\pi$, then store $\forall i \in [m], T[\tilde{\text{id}}_i], T[\tilde{\text{id}}'_i] \leftarrow \perp$. 5. Else apply permutation, store $\forall i \in [m], T[\tilde{\text{id}}_{\pi(i)}] \leftarrow T[\text{id}_i]$. 6. Decrement $ctr \leftarrow ctr - 1$. 7. Send • $(\text{perm}, [\text{id}_i]_i^m, [\tilde{\text{id}}_i]_i^m, \text{id}_\pi, b')$ to P_b. • $(\text{perm}, [\text{id}'_i]_i^m, [\tilde{\text{id}}'_i]_i^m, \text{id}'_\pi, b'')$ to P_{1-b}. <u>PermuteInv($[\text{id}_i]_i^m, [\tilde{\text{id}}_i]_i^m, \text{id}_\pi, b'$) from P_b:</u> 1. Require $\forall i \text{ id}_i \in T, \tilde{\text{id}}_i \notin T$ and no pending message from P_b. 2. Require $\text{id}_\pi \in T_{\pi,b'}$ and $(ctr, \text{invctr}, \pi, m_\pi) \leftarrow T_{\pi,b'}[\text{id}_\pi]$; $\text{invctr} > 0, m = m_\pi$. 3. If no pending message from P_{1-b}, set $(\text{invperm}, [\text{id}_i]_i^m, [\tilde{\text{id}}_i]_i^m, \text{id}_\pi, b')$ from P_b as pending and return. Else parse $(\text{op}, [\text{id}'_i]_i^m, [\tilde{\text{id}}'_i]_i^m, \text{id}'_\pi, b'')$ from P_{1-b} and require $\text{op} = \text{invperm}$. 4. If mismatch, $\exists i, \text{id}_i \neq \text{id}'_i, \tilde{\text{id}}_i \neq \tilde{\text{id}}'_i, \text{id}_\pi \neq \text{id}'_\pi$, then store $\forall i \in [m], T[\tilde{\text{id}}_i], T[\tilde{\text{id}}'_i] \leftarrow \perp$. 5. Else, store $\forall i \in [m], T[\tilde{\text{id}}_{\pi^{-1}(i)}] \leftarrow T[\text{id}_i]$. 6. Decrement $\text{invctr} \leftarrow \text{invctr} - 1$. 7. Send • $(\text{invperm}, [\text{id}_i]_i^m, [\tilde{\text{id}}_i]_i^m, \text{id}_\pi, b')$ to P_b. • $(\text{invperm}, [\text{id}'_i]_i^m, [\tilde{\text{id}}'_i]_i^m, \text{id}'_\pi, b'')$ to P_{1-b}.
---	---

Figure 16: Functionality for computation of arithmetic circuits with additional permutation and inverse permutation gates over $\mathbb{F}$.

Protocol FLOSS $[\mathbb{F}, \text{FSPDZPrep}, \text{FShufTuplePrep}, \text{FComm}]$	$P_0.\text{Reconstruct}(\text{id}) \leftrightarrow P_1.\text{Reconstruct}(\text{id})$:
$P_b.\text{Init}(\lambda)$:	1. Require $\text{id} \in T$.
1. $T \leftarrow \{\cdot\}$: maps id to shares in $\mathbb{F}$. 2. $L_{\text{inp},0}, L_{\text{inp},1} \leftarrow [\cdot]$: list of preprocessed input tuples. 3. $L_{\text{mul}} \leftarrow [\cdot]$: list of preprocessed multiplication tuples. 4. $T_{\text{shf},0}, T_{\text{shf},1} \leftarrow \{\cdot\}$: preprocessed shuffle tuples. 5. $T_{\text{invshf},0}, T_{\text{invshf},1} \leftarrow \{\cdot\}$: preprocessed inverse shuffle tuples. 6. $L_{\text{auth}} \leftarrow [\cdot]$: list of pending authentication checks. 7. $L_{\text{bauth}} \leftarrow [\cdot]$: list of pending blind authentication checks. 8. $\llbracket \gamma \rrbracket_b \leftarrow \perp$: share of authentication key.	2. P_0 and P_1 run authentication check on all pending multiplication opens in L_{auth} (see Fig. 15): (a) $[(z_i, \llbracket \gamma z_i \rrbracket)]_i \leftarrow L_{\text{auth}}; L_{\text{auth}} \leftarrow [\cdot]$ (b) $P_0.\text{Auth}(\llbracket \gamma \rrbracket, [(z_i, \llbracket \gamma z_i \rrbracket)]_i) \leftrightarrow P_1.\text{Auth}(\llbracket \gamma \rrbracket, [(z_i, \llbracket \gamma z_i \rrbracket)]_i)$ 3. P_0 and P_1 run blind authentication check on all pending shuffle checks in L_{bauth} (see Fig. 1): (a) $\langle \alpha \rangle \leftarrow P_b.\text{Coin}(1) \leftrightarrow P_{1-b}.\text{Coin}(1)$ (b) $[(z_i)]_i \leftarrow L_{\text{bauth}}; L_{\text{bauth}} \leftarrow [\cdot]$ (c) $P_0.\text{BlindAuth}(\llbracket \gamma \rrbracket, \langle \alpha \rangle, [(z_i)_0]_i) \leftrightarrow P_1.\text{BlindAuth}(\llbracket \gamma \rrbracket, \langle \alpha \rangle, [(z_i)_1]_i)$ 4. P_0 and P_1 open $\langle v \rangle \leftarrow T[\text{id}]$ to v by sending $\llbracket v \rrbracket$ to each other. 5. P_0 and P_1 run authentication check on reconstructed value (Fig. 15): $P_0.\text{Auth}(\llbracket \gamma \rrbracket, [(v, \llbracket \gamma v \rrbracket)]) \leftrightarrow P_1.\text{Auth}(\llbracket \gamma \rrbracket, [(v, \llbracket \gamma v \rrbracket)])$ 6. Output (reconstruct, $\text{id}, T[\text{id}]$).
$P_0.\text{Prep}(n_{\text{inp},0}, n_{\text{inp},1}, n_m, \text{in}_\pi) \leftrightarrow P_1.\text{Prep}(n_{\text{inp},0}, n_{\text{inp},1}, n_m, \text{in}_\pi)$:	$P_0.\text{Permute}([id_i]_i^m, [\tilde{id}_i]_i^m, id_\pi, b') \leftrightarrow P_1.\text{Permute}([id_i]_i^m, [\tilde{id}_i]_i^m, id_\pi, b')$:
1. P_0 and P_1 invoke $\text{FSPDZPrep}(n_{\text{inp},0}, n_{\text{inp},1}, n_m)$ for each $P_{b'}$ to receive (see Fig. 15): • $L_{\text{inp},b'} \leftarrow [(r_{i,b'}, \langle r_{i,b'} \rangle_{b'})]_i^{n_{\text{inp},b'}}$ • $L_{\text{inp},1-b'} \leftarrow [(r_{i,1-b'}, \langle r_{i,1-b'} \rangle_{b'})]_i^{n_{\text{inp},1-b'}}$ • $L_{\text{mul}} \leftarrow [(a_i)_{b'}, (b_i)_{b'}, (c_i)_{b'}]_i^{n_m}$ • $\llbracket \gamma \rrbracket_{b'}$ 2. P_0 and P_1 as $P_{b'}$ sample random shuffle tuple keys $\kappa_{i,j} \leftarrow \mathbb{F}$ and inverse shuffle tuple keys $\tilde{\kappa}_{i,j} \leftarrow \mathbb{F}$ for each preprocessed shuffle of $P_{b'}$ and random blinding vectors $\vec{a}_{i,j}, \vec{b}_{i,j} \leftarrow \mathbb{F}^{m_{1-b'},i}$ for each preprocessed shuffle of $P_{1-b'}$, incorporate into input in_π, and then invoke $\text{FShufTuplePrep}(\text{in}_\pi)$ for each $P_{b'}$ to receive (see Fig. 18): • $T_{\text{shf},b'}[id_\pi, b', i] \leftarrow \left[\left(\pi_i, \kappa_{i,j}, \langle \vec{a}_{i,j} \rangle_{b'}^{(\kappa)}, \langle \vec{b}_{i,j} \rangle_{b'}^{(\kappa)} \right) \right]_j^{n_{\pi,b',i}}$ • $T_{\text{shf},1-b'}[id_\pi, b', i] \leftarrow \left[\left(\vec{a}_{i,j}, \vec{b}_{i,j}, \langle \vec{a}_{i,j} \rangle_{b'}^{(\kappa)}, \langle \vec{b}_{i,j} \rangle_{b'}^{(\kappa)} \right) \right]_j^{n_{\pi,1-b',i}}$ • $T_{\text{invshf},b'}[id_\pi, b', i] \leftarrow \left[\left(\pi_i^{-1}, \tilde{\kappa}_{i,j}, \langle \vec{a}_{i,j} \rangle_{b'}^{(\kappa)}, \langle \vec{b}_{i,j} \rangle_{b'}^{(\kappa)} \right) \right]_j^{n_{\pi,b',i}}$ • $T_{\text{invshf},1-b'}[id_\pi, b', i] \leftarrow \left[\left(\vec{a}_{i,j}, \vec{b}_{i,j}, \langle \vec{a}_{i,j} \rangle_{b'}^{(\kappa)}, \langle \vec{b}_{i,j} \rangle_{b'}^{(\kappa)} \right) \right]_j^{n_{\pi,1-b',i}}$ 3. Output (prep).	1. Require $\forall i \text{ id}_i \in T, \tilde{id}_i \notin T$, and $id_\pi \in T_{\text{shf},b'}$. 2. P_0 and P_1 retrieve shuffle tuples: • $P_{b'}$ pops $\left(\pi, \kappa, \langle \vec{a} \rangle_{b'}^{(\kappa)}, \langle \vec{b} \rangle_{b'}^{(\kappa)} \right) \leftarrow T_{\text{shf},b'}[id_\pi]$. • $P_{1-b'}$ pops $\left(\vec{a}, \vec{b}, \langle \vec{a} \rangle_{b'}^{(\kappa)}, \langle \vec{b} \rangle_{b'}^{(\kappa)} \right) \leftarrow T_{\text{shf},b'}[id_\pi]$. 3. P_0 and P_1 apply shuffle tuple to $\langle \vec{v} \rangle = [\langle v \rangle]_i^m \leftarrow T[id_i]_i^m$ to get $\langle \vec{v} \rangle, \langle \kappa \vec{v} \rangle$ (Fig. 2): $P_{b'}.\text{ApplyShufTup}\left(\pi, \kappa, \langle \vec{a} \rangle_{b'}^{(\kappa)}, \langle \vec{b} \rangle_{b'}^{(\kappa)}, \langle \vec{v} \rangle\right) \leftrightarrow P_{1-b'}.\text{ApplyShufTup}\left(\vec{a}, \vec{b}, \langle \vec{a} \rangle_{b'}^{(\kappa)}, \langle \vec{b} \rangle_{b'}^{(\kappa)}, \langle \vec{v} \rangle\right)$ 4. P_0 and P_1 store $T[[\tilde{id}_i]_i^m] \leftarrow \langle \vec{v} \rangle$. 5. P_0 and P_1 push $\langle \kappa \vec{v} \rangle$ to L_{bauth} for blind authentication check. 6. Output (perm).
$P_b.\text{Input}(\text{id}, v \in \mathbb{F}) \leftrightarrow P_{1-b}.\text{RcvInput}(\text{id})$:	$P_b.\text{PermutInv}([id_i]_i^m, [\tilde{id}_i]_i^m, id_\pi, b')$:
Same as SPDZ $[\mathbb{F}]$ (see Fig. 14).	Same as Permute except P_0 and P_1 retrieve shuffle tuple from $T_{\text{invshf},b'}[id_\pi]$.
$P_b.\text{ScalAdd}(\text{id}_1, \alpha \in \mathbb{F}, \text{id}_2)$:	
Same as SPDZ $[\mathbb{F}]$ (see Fig. 14).	
$P_b.\text{Add}(\text{id}_1, \text{id}_2, \text{id}_3)$:	
Same as SPDZ $[\mathbb{F}]$ (see Fig. 14).	
$P_b.\text{ScalMul}(\text{id}_1, \alpha \in \mathbb{F}, \text{id}_2)$:	
Same as SPDZ $[\mathbb{F}]$ (see Fig. 14).	
$P_0.\text{Mul}(\text{id}_1, \text{id}_2, \text{id}_3) \leftrightarrow P_1.\text{Mul}(\text{id}_1, \text{id}_2, \text{id}_3)$:	
Same as SPDZ $[\mathbb{F}]$ (see Fig. 14).	

Figure 17: Our main construction FLOSS enabling secure computation of arithmetic circuits over $\mathbb{F}$ with special purpose efficient gates for permutations and inverse permutations.

Functionality $F_{\text{ShufTuplePrep}}[\mathbb{F}]$ Main(in_π) from P_b: 1. Parse in_π: • $[(\text{id}_{\pi,b,i}, n_{\pi,b,i}, n_{\pi,\text{inv},b,i}, [\kappa_{i,j} \in \mathbb{F}]_j^{n_{\pi,b,i}}, [\kappa_{i,j} \in \mathbb{F}]_j^{n_{\pi,\text{inv},b,i}}, \pi_{b,i} \in \text{Perm}(m_{b,i}))]_i^{n_{\pi,b}}$: Permutation parameters for permutations to be applied by P_b. P_b supplies the permutation and κ-authentication values, additionally specifying the number of forward and inverse applications to preprocess for each permutation. • $[(\text{id}_{\pi,1-b,i}, n_{\pi,\text{inv},1-b,i}, [\vec{a}_{i,j}, \vec{b}_{i,j} \in \mathbb{F}^{m_{1-b,i}}]_j^{n_{\pi,1-b,i}}, [\vec{a}_{i,j}, \vec{b}_{i,j} \in \mathbb{F}^{m_{1-b,i}}]_j^{n_{\pi,\text{inv},1-b,i}})]_i^{n_{\pi,1-b}}$: Permutation parameters for permutations to be applied by P_{1-b}. P_b does not hold the permutation but holds blinding vectors that will be used to construct shuffle tuples. 2. If no pending message from P_{1-b}, set $(\text{prep}, n_{i,0}, n_{i,1}, n_m, \text{in}_\pi)$ from P_b as pending and return. Else parse $(\text{op}, n'_{i,0}, n'_{i,1}, n'_m, \text{in}'_\pi)$ from P_{1-b} and require $\text{op} = \text{prep}$. 3. If mismatch, $\exists b', n_{i,b'} \neq n'_{i,b'}, n_m \neq n'_m, \exists i, b', \text{id}_{\pi,b',i} \neq \text{id}'_{\pi,b',i}, n_{\pi,b',i} \neq n'_{\pi,b',i}, n_{\pi,\text{inv},b',i} \neq n'_{\pi,\text{inv},b',i}$, send abort to all parties. 4. Compute shuffle tuples for each $b' \in \{b, 1-b\}$: • $\forall i \in [n_{\pi,b'}], T_{\text{shf},b',b'}[\text{id}_{\pi,b',i}] \leftarrow \left[\left(\pi_i, \kappa_{i,j}, \langle \vec{a}_{i,j} \rangle_{b'}^{(\kappa)}, \langle \vec{b}_{i,j} \rangle_{b'}^{(\kappa)} \right) \right]_j^{n_{\pi,b',i}}$ • $\forall i \in [n_{\pi,b'}], T_{\text{shf},1-b',b'}[\text{id}_{\pi,b',i}] \leftarrow \left[\left(\vec{a}_{i,j}, \vec{b}_{i,j}, \langle \vec{a}_{i,j} \rangle_{1-b'}^{(\kappa)}, \langle \vec{b}_{i,j} \rangle_{1-b'}^{(\kappa)} \right) \right]_j^{n_{\pi,b',i}}$ (a) Take $\vec{a}_{i,j}, \vec{b}_{i,j}$ supplied by $P_{1-b'}$ and permute to compute $\vec{\vec{a}}_{i,j} \leftarrow \pi_{b',i}(\vec{a}_{i,j})$ and $\vec{\vec{b}}_{i,j} \leftarrow \pi_{b',i}(\vec{b}_{i,j})$. (b) Create κ-authenticated shares of $\vec{\vec{a}}_{i,j}$ and $\vec{\vec{b}}_{i,j}$: • $\langle \vec{\vec{a}}_{i,j} \rangle_{b'} \leftarrow \mathbb{F}^{m_{1-b,i}}, \langle \vec{\vec{a}}_{i,j} \rangle_{1-b'} \leftarrow (\vec{\vec{a}}_{i,j} - \llbracket \vec{\vec{a}}_{i,j} \rrbracket_{b'}, \kappa_{i,j} \vec{\vec{a}}_{i,j} - \llbracket \kappa_{i,j} \vec{\vec{a}}_{i,j} \rrbracket_{b'})$ • $\langle \vec{\vec{b}}_{i,j} \rangle_{b'} \leftarrow \mathbb{F}^{m_{1-b,i}}, \langle \vec{\vec{b}}_{i,j} \rangle_{1-b'} \leftarrow (\vec{\vec{b}}_{i,j} - \llbracket \vec{\vec{b}}_{i,j} \rrbracket_{b'}, \kappa_{i,j} \vec{\vec{b}}_{i,j} - \llbracket \kappa_{i,j} \vec{\vec{b}}_{i,j} \rrbracket_{b'})$ 5. Compute inverse shuffle tuples for each $P_{b'}$: • $\forall i \in [n_{\pi,b'}], T_{\text{invshf},b',b'}[\text{id}_{\pi,b',i}] \leftarrow \left[\left(\pi_i^{-1}, \kappa_{i,j}, \langle \vec{\vec{a}}_{i,j} \rangle_{b'}^{(\kappa)}, \langle \vec{\vec{b}}_{i,j} \rangle_{b'}^{(\kappa)} \right) \right]_j^{n_{\pi,b',i}}$ • $\forall i \in [n_{\pi,b'}], T_{\text{invshf},1-b',b'}[\text{id}_{\pi,b',i}] \leftarrow \left[\left(\vec{a}_{i,j}, \vec{b}_{i,j}, \langle \vec{\vec{a}}_{i,j} \rangle_{1-b'}^{(\kappa)}, \langle \vec{\vec{b}}_{i,j} \rangle_{1-b'}^{(\kappa)} \right) \right]_j^{n_{\pi,b',i}}$ (a) Take $\vec{a}_{i,j}, \vec{b}_{i,j}$ supplied by $P_{1-b'}$ and permute to compute $\vec{\vec{a}}_{i,j} \leftarrow \pi_{b',i}(\vec{a}_{i,j})$ and $\vec{\vec{b}}_{i,j} \leftarrow \pi_{b',i}(\vec{b}_{i,j})$. (b) Create κ-authenticated shares of $\vec{\vec{a}}_{i,j}$ and $\vec{\vec{b}}_{i,j}$: • $\langle \vec{\vec{a}}_{i,j} \rangle_{b'} \leftarrow \mathbb{F}^{m_{1-b,i}}, \langle \vec{\vec{a}}_{i,j} \rangle_{1-b'} \leftarrow (\vec{\vec{a}}_{i,j} - \llbracket \vec{\vec{a}}_{i,j} \rrbracket_{b'}, \kappa_{i,j} \vec{\vec{a}}_{i,j} - \llbracket \kappa_{i,j} \vec{\vec{a}}_{i,j} \rrbracket_{b'})$ • $\langle \vec{\vec{b}}_{i,j} \rangle_{b'} \leftarrow \mathbb{F}^{m_{1-b,i}}, \langle \vec{\vec{b}}_{i,j} \rangle_{1-b'} \leftarrow (\vec{\vec{b}}_{i,j} - \llbracket \vec{\vec{b}}_{i,j} \rrbracket_{b'}, \kappa_{i,j} \vec{\vec{b}}_{i,j} - \llbracket \kappa_{i,j} \vec{\vec{b}}_{i,j} \rrbracket_{b'})$ 6. Send • $(T_{\text{shf},0,0}, T_{\text{shf},0,1}, T_{\text{invshf},0,0}, T_{\text{invshf},0,1})$ to P_0. • $(T_{\text{shf},1,0}, T_{\text{shf},1,1}, T_{\text{invshf},1,0}, T_{\text{invshf},1,1})$ to P_1.

Figure 18: Functionality of preprocessing of authenticated shuffle tuples consumed in FLOSS for fast online permutations.

an operation via the control function C , the simulator intercepts that invocation and acts as described in Figure 19. Whenever A sends a message to the environment, the simulator forwards it to the environment. Finally, S outputs the output of A .

Without loss of generality, we assume that all messages from the adversary parse correctly and do not exceed the available preprocessing material (i.e., the number of input tuples, Beaver triples, and shuffle tuples allocated during Prep). If any message fails to parse or exceeds the available preprocessing, the simulator return an abort signal to the adversary, which is consistent with the honest party's behavior in the real protocol. Similarly, we assume that commitments from the adversary verify correctly, else the simulator implicitly aborts, and that the adversary does not break the security of the commitment scheme (working in the F_{Comm} -hybrid model).

Throughout the protocol execution, the simulator maintains for each wire id the *expected adversary shares* $\text{expShare}[\text{id}] = (\llbracket z \rrbracket_0, \llbracket \gamma z \rrbracket_0)$. At each gate, the simulator computes expShare for the output wire by following what the adversary's shares of the output wire would be if the adversary acted honestly in this gate, given the adversary's prior messages. This can be done by the simulator since it knows the preprocessing shares (known from simulating the $(F_{\text{SPDZPrep}}, F_{\text{ShufTuplePrep}})$ -hybrid model), the adversary's protocol messages, and the simulator's own randomness. When the adversary reveals a share, we write $\llbracket z \rrbracket_0^{\text{real}}$ for the received value. The error $\delta_z = \llbracket z \rrbracket_0^{\text{real}} - \llbracket z \rrbracket_0^{\text{exp}}$ captures only a *new* deviation introduced at this message, but not prior deviations that have affected prior opened values (even if they were used to construct this share).

We now state two lemmas bounding the probability that the adversary can deviate without detection, which are essential to proving indistinguishability between the ideal and real worlds. These lemmas are information-theoretic and hold against unbounded adversaries.

Lemma 3 (Non-Blind MAC Check Security). *Fix any $n \geq 1$ and any values $z_1, \dots, z_n \in \mathbb{F}$ with corresponding shares $\{(\llbracket z_i \rrbracket_0, \llbracket z_i \rrbracket_1)\}_{i=1}^n$. Sample $\gamma \leftarrow \mathbb{F}$ and $\llbracket \gamma \rrbracket_0 \leftarrow \mathbb{F}$, and compute MAC shares $\llbracket \gamma z_i \rrbracket_0, \llbracket \gamma z_i \rrbracket_1$ satisfying $\llbracket \gamma z_i \rrbracket_0 + \llbracket \gamma z_i \rrbracket_1 = \gamma z_i$. Consider the following game with an (unbounded) adversary A :*

1. A receives $\llbracket \gamma \rrbracket_0$ and $\{(\llbracket z_i \rrbracket_0, \llbracket \gamma z_i \rrbracket_0)\}_{i=1}^n$.
2. A outputs (possibly modified) value shares $\{\llbracket z_i \rrbracket'_0\}_{i=1}^n$.
3. A participates in the non-blind authentication check (Figure 15) in the F_{Comm} -hybrid model, with an honest party holding $\llbracket \gamma \rrbracket_1$ and $\{(\llbracket z_i \rrbracket_1, \llbracket \gamma z_i \rrbracket_1)\}_{i=1}^n$, using the opened values $z'_i = \llbracket z_i \rrbracket'_0 + \llbracket z_i \rrbracket_1$.

The adversary wins if $\exists i \in [n] : \llbracket z_i \rrbracket'_0 \neq \llbracket z_i \rrbracket_0$ and the check passes. Then $\Pr[A \text{ wins}] \leq n/|\mathbb{F}|$.

Lemma 4 (Blind MAC Check Security). *Fix any $n \geq 1$ and any values $z_1, \dots, z_n \in \mathbb{F}$ with corresponding shares $\{(\llbracket z_i \rrbracket_0, \llbracket z_i \rrbracket_1)\}_{i=1}^n$. Sample $\gamma \leftarrow \mathbb{F}$ and $\llbracket \gamma \rrbracket_0 \leftarrow \mathbb{F}$, and compute MAC shares $\llbracket \gamma z_i \rrbracket_0, \llbracket \gamma z_i \rrbracket_1$ satisfying $\llbracket \gamma z_i \rrbracket_0 + \llbracket \gamma z_i \rrbracket_1 = \gamma z_i$. Similarly, fix a blinding value $\alpha \in \mathbb{F}$ with shares $(\llbracket \alpha \rrbracket_0, \llbracket \alpha \rrbracket_1)$ and MAC shares satisfying $\llbracket \gamma \alpha \rrbracket_0 + \llbracket \gamma \alpha \rrbracket_1 = \gamma \alpha$. Consider the following game with an (unbounded) adversary A :*

1. A receives $\llbracket \gamma \rrbracket_0, (\llbracket \alpha \rrbracket_0, \llbracket \gamma \alpha \rrbracket_0)$, and $\{(\llbracket z_i \rrbracket_0, \llbracket \gamma z_i \rrbracket_0)\}_{i=1}^n$.
2. Sample $r \leftarrow \mathbb{F}$ and give r to A .
3. Compute the honest shares $\llbracket \ell \rrbracket_1 \leftarrow \llbracket \alpha \rrbracket_1 + \sum_{i=1}^n r^{i-1} \llbracket z_i \rrbracket_1$ and $\llbracket \gamma \ell \rrbracket_1 \leftarrow \llbracket \gamma \alpha \rrbracket_1 + \sum_{i=1}^n r^{i-1} \llbracket \gamma z_i \rrbracket_1$, as well as the adversary expected share $\llbracket \ell \rrbracket_0^{\text{exp}} \leftarrow \llbracket \alpha \rrbracket_0 + \sum_{i=1}^n r^{i-1} \llbracket z_i \rrbracket_0$.
4. Obtain $(\ell'_0, \llbracket \epsilon \rrbracket'_0)$ from A .
5. Compute $\ell' = \ell'_0 + \llbracket \ell \rrbracket_1$ and $\llbracket \epsilon \rrbracket_1 = \llbracket \gamma \ell \rrbracket_1 - \llbracket \gamma \rrbracket_1 \cdot \ell'$.
6. The check passes if $\llbracket \epsilon \rrbracket'_0 + \llbracket \epsilon \rrbracket_1 = 0$.

The adversary wins if $\ell'_0 \neq \llbracket \ell \rrbracket_0^{\text{exp}}$ and the check passes. Then $\Pr[A \text{ wins}] = 1/|\mathbb{F}|$.

The proofs of these lemmas appear after the main proof. We now use them to establish indistinguishability using the following steps. First, we show that all messages from the simulator to the adversary are distributed identically to the messages from the real protocol, conditioned on a bad event not happening, and we show $\Pr[\text{bad}]$ is negligible. Next, we show that any deviation from the protocol by the adversary causes an authentication check to fail before the next reconstruction, except with negligible probability. Finally, we show that conditioned on all deviations being caught, the real and the ideal world behave identically in the view of the environment.

Correct distribution of the adversary's view. We verify that each message sent by the simulator to the adversary has the same distribution as in the real protocol, except on very explicitly stated bad events that happen with negligible probability. Consequently, with high probability the adversary's view is identically distributed in both worlds, which implies that any message that it sends to the environment is identically distributed.

Preprocessing shares. The simulator samples all material (input tuples, Beaver triples, shuffle tuples) by following the protocol of the (idealized) hybrid functionality.

Input phase. In the ideal world when the honest party provides input, the simulator sends a uniformly random $\epsilon \leftarrow \mathbb{F}$. In the real world, the honest party sends $\epsilon = x - s$ where s is the input tuple mask. Since s is uniform and independent of the adversary's view, ϵ is uniformly random in both worlds.

Linear operations. For addition, scalar addition, and scalar multiplication, no messages are exchanged between the parties.

Multiplication. In the ideal world the simulator sends $(d_1, e_1) \leftarrow \mathbb{F}^2$. In the real protocol, $d_1 = (x - a) - d_0$ where a is the Beaver triple mask. Since a is uniform and independent of the adversary's view, d_1 is uniform. Similar logic applies for e_1 .

Permutation. When the adversary applies π (case $b' = 0$), the simulator sends random blinding vectors $(c, d) \leftarrow \mathbb{F}^m \times \mathbb{F}^m$, exactly as the real protocol specifies. When the honest party applies π (case $b' = 1$), the adversary does not receive a message.

Reconstruction. The only non-trivial simulation occurs during reconstruction which involves the authentication checks, where the simulation must account for the adversary's possible deviations from earlier in the protocol. We analyze it part by part.

1. *Non-blind authentication check.* The checked values $z_1, \dots, z_n$ are the opened $d = x - a$ and $e = y - b$ from multiplications, and recall that $\delta_i = \llbracket z_i \rrbracket_0^{\text{real}} - \llbracket z_i \rrbracket_0^{\text{exp}}$ is defined as the error introduced by the adversary in the reconstruction of each value.

After r is determined, which happens in the same way in the real and the ideal world, define $\ell = \sum_i r^{i-1} z_i$, $\Delta = \sum_i r^{i-1} \delta_i$, and $\llbracket \epsilon \rrbracket_0^{\text{exp}} = \llbracket \gamma \ell \rrbracket_0^{\text{exp}} - \llbracket \gamma \rrbracket_0 \cdot \ell$. The simulator sets $\llbracket \epsilon \rrbracket_1 = -\llbracket \epsilon \rrbracket_0^{\text{exp}} - \gamma \cdot \Delta$.

In the real protocol, the MAC shares satisfy $\llbracket \gamma z_i \rrbracket_0^{\text{exp}} + \llbracket \gamma z_i \rrbracket_1 = \gamma(z_i - \delta_i)$, so $\llbracket \gamma z_i \rrbracket_1 = \gamma z_i - \gamma \delta_i - \llbracket \gamma z_i \rrbracket_0^{\text{exp}}$. Thus, the honest party computes

$$\llbracket \epsilon \rrbracket_1 = \sum_i r^{i-1} \llbracket \gamma z_i \rrbracket_1 - \llbracket \gamma \rrbracket_1 \ell = \gamma \ell - \gamma \Delta - \llbracket \gamma \ell \rrbracket_0^{\text{exp}} - \gamma \ell + \llbracket \gamma \rrbracket_0 \ell = -\llbracket \epsilon \rrbracket_0^{\text{exp}} - \gamma \cdot \Delta,$$

which equals the simulator's value.

2. *Blind authentication check.* The checked values $z_1, \dots, z_n$ are permutation outputs, which are not opened individually. Instead, a blinding value $\alpha = \alpha_0 + \alpha_1$ is generated (each party inputs their contribution) and the linear combination $\ell = \alpha + \sum_i r^{i-1} z_i$ is opened.

For the opening message $\llbracket \ell \rrbracket_1$: the simulator sends $\llbracket \ell \rrbracket_1 \leftarrow \mathbb{F}$. In the real protocol, $\llbracket \ell \rrbracket_1 = \llbracket \alpha_1 \rrbracket_1 + \llbracket \alpha_0 \rrbracket_1 + \sum_i r^{i-1} \llbracket z_i \rrbracket_1$. Since α_1 is the honest party's input with mask s_1 , we have $\llbracket \alpha_1 \rrbracket_1 = \llbracket s_1 \rrbracket_1 = s_1 - \llbracket s_1 \rrbracket_0$, which is uniform from the adversary's view. Thus $\llbracket \ell \rrbracket_1$ is uniform.

For the authentication message $\llbracket \epsilon \rrbracket_1$: define $\Delta_\ell = \ell_0 - \llbracket \ell \rrbracket_0^{\text{exp}}$ and $\llbracket \epsilon \rrbracket_0^{\text{exp}} = \llbracket \gamma \ell \rrbracket_0^{\text{exp}} - \llbracket \gamma \rrbracket_0 \ell$. We split into two cases.

Case permInputDev = false: every $b' = 1$ gate satisfies $\vec{w} = (\vec{d} - \vec{d}^{\text{exp}}) - \gamma(\vec{c} - \vec{c}^{\text{exp}}) = \vec{0}$, so the γ -MAC relation holds for all blind auth check entries (MAC error $E_i = 0$). The simulator sets $\llbracket \epsilon \rrbracket_1 = -\llbracket \epsilon \rrbracket_0^{\text{exp}} - \gamma \Delta_\ell$. The same algebraic verification as in the non-blind case shows that the honest party's $\llbracket \epsilon \rrbracket_1$ in the real protocol equals the simulator's value.

Case permInputDev = true: this is the more interesting case, where the adversary attempted to introduce an error into their input for one of the permutation gates. This is also where our proof would fall apart had our protocol permitted the permutation guessing selective failure attack (Section 3).

In this case, there exists some shuffling gate where $b' = 1$ (the honest party knows the permutation) and $\vec{w} \neq \vec{0}$. The simulator sends $\llbracket \epsilon \rrbracket_1 \leftarrow \mathbb{F}$. We show that in the real world, $\llbracket \epsilon \rrbracket_1$ is also uniformly random from the adversary's view.

First, for simplicity, assume a single gate in the blind authentication check with m components. Let π denote the honest party's permutation and κ the honest party's multiplicative key. The adversary sends $(\vec{c}, \vec{d})$ with deviations $\vec{\delta}_c = \vec{c} - \vec{c}^{\text{exp}}$ and $\vec{\delta}_d = \vec{d} - \vec{d}^{\text{exp}}$, and recall $\vec{w} = \vec{\delta}_d - \gamma \vec{\delta}_c$.

Tracing through the real protocol (Fig. 2) with the deviated inputs, the κ -multiplied value shares reconstruct to $\llbracket \kappa \tilde{v} \rrbracket_0^{\text{exp}} + \llbracket \kappa \tilde{v} \rrbracket_1 = \kappa \pi(\vec{v} + \vec{\delta}_c)$ and the γ -MAC on the κ -multiplied shares reconstructs to $\llbracket \gamma \kappa \tilde{v} \rrbracket_0^{\text{exp}} + \llbracket \gamma \kappa \tilde{v} \rrbracket_1 = \gamma \kappa \pi(\vec{v}) + \kappa \pi(\vec{\delta}_d)$ (the adversary's expected output shares $\llbracket \cdot \rrbracket_0^{\text{exp}}$ are set from preprocessing and are unaffected by the deviation).

The real world blind auth check processes m entries $\kappa \tilde{v}_1, \dots, \kappa \tilde{v}_m$, and computes $\llbracket \epsilon \rrbracket_1 = \llbracket \gamma \ell \rrbracket_1 - \llbracket \gamma \rrbracket_1 \cdot \ell$ where $\ell = \alpha + \sum r^{i-1} z_i$. Each entry $\kappa \tilde{v}_j$ has per-component MAC error $\kappa \pi(\vec{\delta}_d)_j - \gamma \kappa \pi(\vec{\delta}_c)_j = \kappa \pi(\vec{w})_j$. Using the same expansion as in the non-blind case:

$$\llbracket \epsilon \rrbracket_1 = -\llbracket \epsilon \rrbracket_0^{\text{exp}} - \gamma \Delta_\ell + \kappa \cdot \sum_{j=1}^m r^{j-1} \pi(\vec{w})_j.$$

Since $S := \sum_{j=1}^m r^{j-1} \pi(\vec{w})_j$ is a polynomial in r of degree at most $m-1$ with at least one nonzero coefficient, by the fundamental theorem of algebra this polynomial has at most $m-1$ roots, and so $\Pr_{r \leftarrow \mathbb{F}}[S=0] \leq (m-1)/|\mathbb{F}|$. We explicitly define $S=0$ as a bad event, in which our simulation fails, but it happens with negligible probability. Conditioned on S being non-zero, $\llbracket \epsilon \rrbracket_1$ is an affine function of κ with nonzero coefficient, and since κ is uniformly random, so is $\llbracket \epsilon \rrbracket_1$.

We point out that this argument easily extends to multiple permutation gates, at the cost of notational complexity. The key point is that if there is any gate where $\vec{w} \neq \vec{0}$, then at least one of the resulting polynomials in r is non-zero, and thus the overall linear combination is non-zero except with negligible probability over r .

3. When opening v , the simulator sends $\llbracket v \rrbracket_1 = v - \llbracket v \rrbracket_0^{\text{exp}}$ which is equal to the message in the real world since the simulator correctly tracks the expected adversary's share. Finally, for the authentication of the opened value the simulation is perfect since the same analysis as the non-blind authentication check applies (with $n=1$).

Deviation detection. We bound the probability that the adversary deviates from the protocol without getting detected in the real world (and equivalently in the ideal world, since conditioned on bad not occurring the simulation is identical), that is the probability that the adversary introduces an error in any value it sends to the honest party without being caught by the subsequent authentication check. We enumerate all messages the adversary sends and, for each possible deviation, apply the appropriate MAC security lemma to bound the forgery probability.

Input from P_0 . This step defines the adversary's input, which it can choose in both worlds, and against which we define consistency. No deviation is possible.

Multiplication. During multiplication, the adversary sends (d_0, e_0) , possibly deviating from the expected $d_0^{\text{exp}} = \llbracket x \rrbracket_0 - \llbracket a \rrbracket_0$, $e_0^{\text{exp}} = \llbracket y \rrbracket_0 - \llbracket b \rrbracket_0$. This changes the opened values $d = d_0 + d_1$, $e = e_0 + e_1$, but does not change the MAC: the honest party's shares and MAC shares still authenticate the original (expected) opened values.

At Reconstruct, the non-blind authentication check processes all n pending entries. Suppose some entry has error $\delta_i = \llbracket z_i \rrbracket_0^{\text{real}} - \llbracket z_i \rrbracket_0^{\text{exp}} \neq 0$ and the check passes. Then the adversary has successfully opened an incorrect value without failing authentication, which by Lemma 3 occurs with negligible probability.

Permutation. In $b' = 0$ gates the adversary sends no message, so no deviation is possible. In $b' = 1$ gates the adversary sends $(\vec{c}, \vec{d})$, possibly deviating from the expected values. Let $\vec{\delta}_c = \vec{c} - \vec{c}^{\text{exp}}$, $\vec{\delta}_d = \vec{d} - \vec{d}^{\text{exp}}$, and $\vec{w} = \vec{\delta}_d - \gamma \vec{\delta}_c$.

If $\vec{w} \neq \vec{0}$, then the correct distribution analysis shows that $\llbracket \epsilon \rrbracket_1$ in the blind auth check is uniformly random (conditioned on $\neg \text{bad}$, which happens with overwhelming probability), so the check catches the deviation with probability $1 - 1/|\mathbb{F}|$.

If $\vec{w} = \vec{0}$ but $(\vec{\delta}_c, \vec{\delta}_d) \neq (\vec{0}, \vec{0})$, then the MAC relation is preserved, and the blind auth check passes. However, this requires the adversary to have guessed γ : for any nonzero $\vec{\delta}_c$, the constraint $\vec{\delta}_d = \gamma \vec{\delta}_c$ fixes γ to a single value, and since γ is information-theoretically hidden from the adversary's view, this occurs with probability at most $1/|\mathbb{F}|$.

Reconstruction. Three checks are invoked. The non-blind check on L_{auth} catches any additional error beyond the opened-value deviations analyzed above (Lemma 3). The blind check on L_{bauth} catches all newly introduced errors by Lemma 4 and the permutation analysis above. Finally, the adversary opens $\llbracket v \rrbracket_0'$, and the per-value non-blind check ($n=1$) catches any deviation by Lemma 3.

Indistinguishability of the honest party's output. Conditioned on all deviations being caught, we argue that the real and ideal worlds are identical from the environment's view.

Recall that the environment receives communication from the adversary, along with the output of the honest party at the end of the protocol. Since the adversary's view is identically distributed in both worlds, any message it sends to the environment is identically distributed. We are left to argue that the honest party's output is identically distributed in both worlds.

Recall that the honest party's output is determined by individual reconstruction gates. In the ideal world, at each reconstruction gate the simulator forwards $\text{Reconstruct}(\text{id})$ to $F_{\text{ArithPermCirc}}$, receives the correct output v , and routes it to the honest party (or aborts if an authentication check fails).

In the real world, conditioned on no undetected deviations, if the protocol does not abort then the adversary did not deviate at any point, so all wire values are correctly computed and the honest party reconstructs $v' = \llbracket v \rrbracket'_0 + \llbracket v \rrbracket_1 = v$. Since the simulator extracts the same input $x_0 = r + \epsilon$ that the adversary uses in the real protocol, the output matches. The abort events are identically distributed by the correct distribution analysis (conditioned on bad not occurring).

Conclusion. Conditioned on bad not occurring and no deviation going undetected, the environment's complete view (the adversary's communication, the abort outcome, and the honest party's output) is identically distributed in both worlds. Since both $\Pr[\text{bad}]$ and the probability of an undetected deviation are both negligible in the security parameter (by the correct distribution and deviation detection analyses, respectively), the real and ideal worlds are indistinguishable with non-negligible probability. $\square$

We now prove the two lemmas used in the security proof.

Proof of Lemma 3. Let $\delta_i = \llbracket z_i \rrbracket'_0 - \llbracket z_i \rrbracket_0$ denote the adversary's deviation in the i -th value share. After opening, the reconstructed values are $z'_i = z_i + \delta_i$ and $\ell' = \sum_{i=1}^n r^{i-1} z'_i$, where r is a public random challenge.

The honest party computes $\llbracket \epsilon \rrbracket_1 = \llbracket \gamma \ell' \rrbracket_1 - \llbracket \gamma \rrbracket_1 \cdot \ell' = -\gamma \cdot \Delta(r) - \llbracket \epsilon \rrbracket_0^{\text{exp}}$, where $\Delta(r) = \sum_{i=1}^n r^{i-1} \delta_i$ and $\llbracket \epsilon \rrbracket_0^{\text{exp}} = \llbracket \gamma \ell' \rrbracket_0 - \llbracket \gamma \rrbracket_0 \cdot \ell'$.

The adversary then sends some $\llbracket \epsilon \rrbracket'_0$, and the check passes if and only if $\llbracket \epsilon \rrbracket'_0 + \llbracket \epsilon \rrbracket_1 = 0$. Thus, to pass the check, the adversary must send $\llbracket \epsilon \rrbracket'_0 = -\llbracket \epsilon \rrbracket_1 = \gamma \cdot \Delta(r) + \llbracket \epsilon \rrbracket_0^{\text{exp}}$. Since the adversary can compute $\llbracket \epsilon \rrbracket_0^{\text{exp}}$ from their view, passing the check allows the adversary to also learn $\gamma \cdot \Delta(r)$.

We bound the probability of a correct guess. By the winning condition, some $\delta_i \neq 0$, so $\Delta(X) = \sum_{i=1}^n X^{i-1} \delta_i$ is a nonzero polynomial of degree at most $n-1$. Define the event $E = \{\Delta(r) = 0\}$. By the Fundamental Theorem of Algebra, $\Pr[E] \leq (n-1)/|\mathbb{F}|$.

Conditioned on E not happening, the value $\gamma \cdot \Delta(r)$ is uniformly distributed over $\mathbb{F}$, since $\gamma \leftarrow \mathbb{F}$ is independent of the adversary's view, and thus can be guessed with probability $1/|\mathbb{F}|$.

Therefore, $\Pr[\text{A wins}] \leq \Pr[E] + \Pr[\text{correct guess} \mid \neg E] \leq n/|\mathbb{F}|$. $\square$

Proof of Lemma 4. Let $\Delta_\ell = \ell'_0 - \llbracket \ell \rrbracket_0^{\text{exp}}$ denote the adversary's deviation from the expected share. The opened value is $\ell' = \ell'_0 + \llbracket \ell \rrbracket_1 = \ell + \Delta_\ell$.

The honest party computes $\llbracket \epsilon \rrbracket_1 = \llbracket \gamma \ell' \rrbracket_1 - \llbracket \gamma \rrbracket_1 \cdot \ell' = -\llbracket \epsilon \rrbracket_0^{\text{exp}} - \gamma \cdot \Delta_\ell$, where $\llbracket \epsilon \rrbracket_0^{\text{exp}} = \llbracket \gamma \ell \rrbracket_0 - \llbracket \gamma \rrbracket_0 \cdot \ell'$.

The adversary sends some $\llbracket \epsilon \rrbracket'_0$, and the check passes if and only if $\llbracket \epsilon \rrbracket'_0 + \llbracket \epsilon \rrbracket_1 = 0$. Thus, to pass the check, the adversary must send $\llbracket \epsilon \rrbracket'_0 = -\llbracket \epsilon \rrbracket_1 = \llbracket \epsilon \rrbracket_0^{\text{exp}} + \gamma \cdot \Delta_\ell$. Since the adversary can compute $\llbracket \epsilon \rrbracket_0^{\text{exp}} = \llbracket \gamma \ell \rrbracket_0 - \llbracket \gamma \rrbracket_0 \cdot \ell'$ from their view, passing the check requires guessing $\gamma \cdot \Delta_\ell$.

By the winning condition, $\Delta_\ell \neq 0$. Since $\gamma \leftarrow \mathbb{F}$ is independent of the adversary's view, the value $\gamma \cdot \Delta_\ell$ is uniformly distributed over $\mathbb{F}$, and thus can be guessed with probability $1/|\mathbb{F}|$.

Therefore, $\Pr[\text{A wins}] = 1/|\mathbb{F}|$. $\square$

E Full Description of Preprocessing Protocol for Authenticated Shuffle Tuples

An authenticated shuffle tuple for permutation π chosen by party P_0 consists of the following values for P_0 (left) and P_1 (right) where P_0 chooses random $\kappa \leftarrow \mathbb{F}$ and P_1 chooses random vectors $\vec{a}, \vec{b} \leftarrow \mathbb{F}^n$:

$$\left(\pi, \kappa, \langle \pi(\vec{a}) \rangle_0^{(\kappa)}, \langle \pi(\vec{b}) \rangle_0^{(\kappa)} \right), \left(\vec{a}, \vec{b}, \langle \pi(\vec{a}) \rangle_1^{(\kappa)}, \langle \pi(\vec{b}) \rangle_1^{(\kappa)} \right).$$

An authenticated inverse shuffle tuple for permutation π chosen by party P_0 consists of the following values for P_0 (left) and P_1 (right) where P_0 chooses random $\kappa \leftarrow \mathbb{F}$ and P_1 chooses random vectors $\vec{a}, \vec{b} \leftarrow \mathbb{F}^n$:

$$\left(\pi, \kappa, \langle \pi^{-1}(\vec{a}) \rangle_0^{(\kappa)}, \langle \pi^{-1}(\vec{b}) \rangle_0^{(\kappa)} \right), \left(\vec{a}, \vec{b}, \langle \pi^{-1}(\vec{a}) \rangle_1^{(\kappa)}, \langle \pi^{-1}(\vec{b}) \rangle_1^{(\kappa)} \right).$$

Simulator $S_{\text{Online}}[\text{Corrupt } P_0](A)$:

Init(λ):

1. $\text{expShare} \leftarrow \{\cdot\}$: maps id to expected adversary share $(\llbracket v \rrbracket_0, \llbracket \gamma v \rrbracket_0)$.
2. $L_{\text{auth}} \leftarrow []$: queue of pending (non-blind) auth checks from multiplication opens, each entry $(z, \llbracket z \rrbracket_0^{\text{exp}}, \llbracket z \rrbracket_0^{\text{real}}, \llbracket \gamma z \rrbracket_0^{\text{exp}})$ storing opened value, expected adversary share, actual adversary share, and expected MAC share.
3. $L_{\text{bauth}} \leftarrow []$: queue of pending blind auth checks from permutations, each entry $(\llbracket z \rrbracket_0, \llbracket \gamma z \rrbracket_0)$ storing expected adversary value and MAC shares.
4. $\text{permInputDev} \leftarrow \text{false}$: deviation in input to permutation.

On Prep($n_{\text{inp},0}, n_{\text{inp},1}, n_m, \text{in}_\pi$):

1. Forward $\text{Prep}(n_{\text{inp},0}, n_{\text{inp},1}, n_m, \text{in}_\pi)$ to $F_{\text{ArithPermCirc}}$.
2. Sample $\gamma \leftarrow \mathbb{F}$ and $\llbracket \gamma \rrbracket_0 \leftarrow \mathbb{F}$.
3. Simulate F_{SPDZPrep} output: sample random shares for input/multiplication tuples.
4. Simulate $F_{\text{ShufTuplePrep}}$: store adversary's input (permutations π , keys κ for tuples P_0 applies; blinding vectors (a, b) for tuples P_1 applies) and return random shares $(\llbracket \tilde{a} \rrbracket_0, \llbracket \kappa \tilde{a} \rrbracket_0, \llbracket \tilde{b} \rrbracket_0, \llbracket \kappa \tilde{b} \rrbracket_0)$.
5. Return simulated preprocessing to A.

On Input(id, v) from P_0 :

1. Pop stored input tuple $(r, \llbracket r \rrbracket_0, \llbracket \gamma r \rrbracket_0)$.
2. Receive ϵ from adversary; compute $v \leftarrow r + \epsilon$.
3. Forward $\text{Input}(\text{id}, v)$ to $F_{\text{ArithPermCirc}}$.
4. Compute $\llbracket v \rrbracket_0 \leftarrow \llbracket r \rrbracket_0 + \epsilon$ and $\llbracket \gamma v \rrbracket_0 \leftarrow \llbracket \gamma r \rrbracket_0 + \llbracket \gamma \rrbracket_0 \cdot \epsilon$.
5. Store $\text{expShare}[\text{id}] \leftarrow (\llbracket v \rrbracket_0, \llbracket \gamma v \rrbracket_0)$.

On Input(id) from P_1 :

1. Pop stored input tuple $(\llbracket r \rrbracket_0, \llbracket \gamma r \rrbracket_0)$.
2. Forward $\text{Input}(\text{id})$ to $F_{\text{ArithPermCirc}}$.
3. Sample $\epsilon \leftarrow \mathbb{F}$ and send to adversary.
4. Compute $\llbracket v \rrbracket_0 \leftarrow \llbracket r \rrbracket_0 + \epsilon$ and $\llbracket \gamma v \rrbracket_0 \leftarrow \llbracket \gamma r \rrbracket_0 + \llbracket \gamma \rrbracket_0 \cdot \epsilon$.
5. Store $\text{expShare}[\text{id}] \leftarrow (\llbracket v \rrbracket_0, \llbracket \gamma v \rrbracket_0)$.

On ScalAdd, Add, ScalMul:

1. Forward operation to $F_{\text{ArithPermCirc}}$.
2. Update expShare for output id by applying the corresponding linear operation to the input entries.

On Reconstruct(id): The non-trivial part of the simulator. See Figure 20.

On Mul($\text{id}_1, \text{id}_2, \text{id}_3$):

1. Forward $\text{Mul}(\text{id}_1, \text{id}_2, \text{id}_3)$ to $F_{\text{ArithPermCirc}}$.
2. Pop Beaver triple shares $(\llbracket a \rrbracket_0, \llbracket \gamma a \rrbracket_0, \llbracket b \rrbracket_0, \llbracket \gamma b \rrbracket_0, \llbracket c \rrbracket_0, \llbracket \gamma c \rrbracket_0)$.
3. Look up $(\llbracket x \rrbracket_0, \llbracket \gamma x \rrbracket_0) \leftarrow \text{expShare}[\text{id}_1]$ and $(\llbracket y \rrbracket_0, \llbracket \gamma y \rrbracket_0) \leftarrow \text{expShare}[\text{id}_2]$.
4. Sample $d_1, e_1 \leftarrow \mathbb{F}$ and send (d_1, e_1) to adversary.
5. Receive (d_0, e_0) from adversary; compute $d \leftarrow d_0 + d_1$ and $e \leftarrow e_0 + e_1$.
6. Compute adversary's output share:
 - $\llbracket z \rrbracket_0 \leftarrow \llbracket c \rrbracket_0 + d \cdot \llbracket b \rrbracket_0 + e \cdot \llbracket a \rrbracket_0 + d \cdot e$
 - $\llbracket \gamma z \rrbracket_0 \leftarrow \llbracket \gamma c \rrbracket_0 + d \cdot \llbracket \gamma b \rrbracket_0 + e \cdot \llbracket \gamma a \rrbracket_0 + \llbracket \gamma \rrbracket_0 \cdot d \cdot e$
7. Store $\text{expShare}[\text{id}_3] \leftarrow (\llbracket z \rrbracket_0, \llbracket \gamma z \rrbracket_0)$.
8. Compute expected shares for pending auth check:
 - $\llbracket d \rrbracket_0^{\text{exp}} \leftarrow \llbracket x \rrbracket_0 - \llbracket a \rrbracket_0, \quad \llbracket \gamma d \rrbracket_0^{\text{exp}} \leftarrow \llbracket \gamma x \rrbracket_0 - \llbracket \gamma a \rrbracket_0$
 - $\llbracket e \rrbracket_0^{\text{exp}} \leftarrow \llbracket y \rrbracket_0 - \llbracket b \rrbracket_0, \quad \llbracket \gamma e \rrbracket_0^{\text{exp}} \leftarrow \llbracket \gamma y \rrbracket_0 - \llbracket \gamma b \rrbracket_0$
9. Push $(d, \llbracket d \rrbracket_0^{\text{exp}}, d_0, \llbracket \gamma d \rrbracket_0^{\text{exp}})$ and $(e, \llbracket e \rrbracket_0^{\text{exp}}, e_0, \llbracket \gamma e \rrbracket_0^{\text{exp}})$ to L_{auth} .

On Permute($[\text{id}_i]_i^m, [\tilde{\text{id}}_i]_i^m, \text{id}_\pi, b'$):

1. Forward $\text{Permute}([\text{id}_i]_i^m, [\tilde{\text{id}}_i]_i^m, \text{id}_\pi, b')$ to $F_{\text{ArithPermCirc}}$.
2. Look up input shares: $\forall i, (\llbracket v_i \rrbracket_0, \llbracket \gamma v_i \rrbracket_0) \leftarrow \text{expShare}[\text{id}_i]$.
3. **Case $b' = 0$** (adversary applies π):
 - (a) Pop shuffle tuple: $(\pi, \kappa, \llbracket \tilde{a} \rrbracket_0, \llbracket \kappa \tilde{a} \rrbracket_0, \llbracket \tilde{b} \rrbracket_0, \llbracket \kappa \tilde{b} \rrbracket_0)$.
 - (b) Sample $\vec{c}, \vec{d} \leftarrow \mathbb{F}^m$ and send to adversary.
 - (c) Compute expected output shares:
 - $\llbracket \vec{v} \rrbracket_0 \leftarrow \pi(\llbracket \vec{v} \rrbracket_0) + \pi(\vec{c}) + \llbracket \tilde{a} \rrbracket_0$
 - $\llbracket \gamma \vec{v} \rrbracket_0 \leftarrow \pi(\llbracket \gamma \vec{v} \rrbracket_0) + \pi(\vec{d}) + \llbracket \tilde{b} \rrbracket_0$
 - $\llbracket \kappa \vec{v} \rrbracket_0 \leftarrow \kappa \cdot \pi(\llbracket \vec{v} \rrbracket_0) + \kappa \cdot \pi(\vec{c}) + \llbracket \kappa \tilde{a} \rrbracket_0$
 - $\llbracket \gamma \kappa \vec{v} \rrbracket_0 \leftarrow \kappa \cdot \pi(\llbracket \gamma \vec{v} \rrbracket_0) + \kappa \cdot \pi(\vec{d}) + \llbracket \kappa \tilde{b} \rrbracket_0$
4. **Case $b' = 1$** (honest party applies π):
 - (a) Pop shuffle tuple: $(\vec{a}, \vec{b}, \llbracket \tilde{a} \rrbracket_0, \llbracket \kappa \tilde{a} \rrbracket_0, \llbracket \tilde{b} \rrbracket_0, \llbracket \kappa \tilde{b} \rrbracket_0)$.
 - (b) Receive $(\vec{c}, \vec{d})$ from adversary.
 - (c) Compute $\vec{c}^{\text{exp}} \leftarrow \llbracket \vec{v} \rrbracket_0 - \vec{a}, \vec{d}^{\text{exp}} \leftarrow \llbracket \gamma \vec{v} \rrbracket_0 - \vec{b}$. If $(\vec{d} - \vec{d}^{\text{exp}}) - \gamma \cdot (\vec{c} - \vec{c}^{\text{exp}}) \neq \vec{0}$, set $\text{permInputDev} \leftarrow \text{true}$.
 - (d) Compute expected output shares:
 - $\llbracket \vec{v} \rrbracket_0 \leftarrow \llbracket \tilde{a} \rrbracket_0$
 - $\llbracket \gamma \vec{v} \rrbracket_0 \leftarrow \llbracket \tilde{b} \rrbracket_0$
 - $\llbracket \kappa \vec{v} \rrbracket_0 \leftarrow \llbracket \kappa \tilde{a} \rrbracket_0$
 - $\llbracket \gamma \kappa \vec{v} \rrbracket_0 \leftarrow \llbracket \kappa \tilde{b} \rrbracket_0$
5. Store $\forall i, \text{expShare}[\tilde{\text{id}}_i] \leftarrow (\llbracket \tilde{v}_i \rrbracket_0, \llbracket \gamma \tilde{v}_i \rrbracket_0)$.
6. Push $(\llbracket \kappa \vec{v} \rrbracket_0, \llbracket \gamma \kappa \vec{v} \rrbracket_0)$ to L_{bauth} .

On PermutInv($[\text{id}_i]_i^m, [\tilde{\text{id}}_i]_i^m, \text{id}_\pi, b'$):

1. Same as Permute , using inverse shuffle tuples and π^{-1} .

Figure 19: Simulator S for the online protocol FLOSS when P_0 is corrupted.

Simulator $\mathcal{S}_{\text{Online-Reconstruct}}[\text{Corrupt } P_0](A)$:

On $\text{Reconstruct}(\text{id})$:

1. **Non-blind auth check** on L_{auth} (see Fig. 15):

- (a) Commit-open for r : Sample $r_1 \leftarrow \mathbb{F}$; commit via F_{Comm} . Receive adversary's commit; extract r_0 . Open; receive adversary's open. Compute $r \leftarrow r_0 + r_1$.
- (b) For each $(z_i, \llbracket z_i \rrbracket_0^{\text{exp}}, \llbracket z_i \rrbracket_0^{\text{real}}, \llbracket \gamma z_i \rrbracket_0^{\text{exp}}) \in L_{\text{auth}}$, compute $\delta_i \leftarrow \llbracket z_i \rrbracket_0^{\text{real}} - \llbracket z_i \rrbracket_0^{\text{exp}}$.
- (c) Compute $\ell \leftarrow \sum_i r^{i-1} z_i$, $\llbracket \gamma \ell \rrbracket_0^{\text{exp}} \leftarrow \sum_i r^{i-1} \llbracket \gamma z_i \rrbracket_0^{\text{exp}}$, $\Delta \leftarrow \sum_i r^{i-1} \delta_i$.
- (d) Compute $\llbracket \epsilon \rrbracket_0^{\text{exp}} \leftarrow \llbracket \gamma \ell \rrbracket_0^{\text{exp}} - \llbracket \gamma \rrbracket_0 \cdot \ell$.
- (e) Commit-open for ϵ : Receive adversary's commit; extract $\llbracket \epsilon \rrbracket_0$. Commit to $\llbracket \epsilon \rrbracket_1 \leftarrow -\llbracket \epsilon \rrbracket_0^{\text{exp}} - \gamma \cdot \Delta$. Open.
- (f) If $\llbracket \epsilon \rrbracket_0 + \llbracket \epsilon \rrbracket_1 \neq 0$, abort.

2. **Blind auth check** on L_{bauth} (see Fig. 1):

- (a) Generate random blind $\alpha = \alpha_0 + \alpha_1$:
 - i. Pop $(\llbracket s_1 \rrbracket_0, \llbracket \gamma s_1 \rrbracket_0)$. Sample $\epsilon_1 \leftarrow \mathbb{F}$ and send to adversary. Compute $\llbracket \alpha_1 \rrbracket_0 \leftarrow \llbracket s_1 \rrbracket_0 + \epsilon_1$ and $\llbracket \gamma \alpha_1 \rrbracket_0 \leftarrow \llbracket \gamma s_1 \rrbracket_0 + \llbracket \gamma \rrbracket_0 \cdot \epsilon_1$.
 - ii. Pop $(s_0, \llbracket s_0 \rrbracket_0, \llbracket \gamma s_0 \rrbracket_0)$. Receive ϵ_0 from adversary. Compute $\llbracket \alpha_0 \rrbracket_0 \leftarrow \llbracket s_0 \rrbracket_0 + \epsilon_0$ and $\llbracket \gamma \alpha_0 \rrbracket_0 \leftarrow \llbracket \gamma s_0 \rrbracket_0 + \llbracket \gamma \rrbracket_0 \cdot \epsilon_0$.
 - iii. Compute $\llbracket \alpha \rrbracket_0 \leftarrow \llbracket \alpha_0 \rrbracket_0 + \llbracket \alpha_1 \rrbracket_0$ and $\llbracket \gamma \alpha \rrbracket_0 \leftarrow \llbracket \gamma \alpha_0 \rrbracket_0 + \llbracket \gamma \alpha_1 \rrbracket_0$.
- (b) Commit-open for r : As in non-blind check.
- (c) Using $(\llbracket z_i \rrbracket_0, \llbracket \gamma z_i \rrbracket_0)$ from L_{bauth} , compute:
 - $\llbracket \ell \rrbracket_0^{\text{exp}} \leftarrow \llbracket \alpha \rrbracket_0 + \sum_i r^{i-1} \llbracket z_i \rrbracket_0$
 - $\llbracket \gamma \ell \rrbracket_0^{\text{exp}} \leftarrow \llbracket \gamma \alpha \rrbracket_0 + \sum_i r^{i-1} \llbracket \gamma z_i \rrbracket_0$
- (d) Open ℓ : Send $\ell_1 \leftarrow \mathbb{F}$. Receive ℓ_0 ; compute $\ell \leftarrow \ell_0 + \ell_1$ and $\Delta_\ell \leftarrow \ell_0 - \llbracket \ell \rrbracket_0^{\text{exp}}$.
- (e) Commit-open for ϵ : Compute $\llbracket \epsilon \rrbracket_0^{\text{exp}} \leftarrow \llbracket \gamma \ell \rrbracket_0^{\text{exp}} - \llbracket \gamma \rrbracket_0 \cdot \ell$. Receive adversary's commit; extract $\llbracket \epsilon \rrbracket_0$. If permInputDev , commit to $\llbracket \epsilon \rrbracket_1 \leftarrow \mathbb{F}$; else commit to $\llbracket \epsilon \rrbracket_1 \leftarrow -\llbracket \epsilon \rrbracket_0^{\text{exp}} - \gamma \cdot \Delta_\ell$. Open.
- (f) If $\llbracket \epsilon \rrbracket_0 + \llbracket \epsilon \rrbracket_1 \neq 0$, abort.

3. **Reconstruct** v :

- (a) Forward $\text{Reconstruct}(\text{id})$ to $F_{\text{ArithPermCirc}}$; receive value v .
- (b) Look up $(\llbracket v \rrbracket_0, \llbracket \gamma v \rrbracket_0) \leftarrow \text{expShare}[\text{id}]$.
- (c) Send $\llbracket v \rrbracket_1 \leftarrow v - \llbracket v \rrbracket_0$ to adversary. Receive $\llbracket v \rrbracket'_0$ from adversary.
- (d) Compute $v' \leftarrow \llbracket v \rrbracket'_0 + \llbracket v \rrbracket_1$ and $\delta \leftarrow \llbracket v \rrbracket'_0 - \llbracket v \rrbracket_0$.
- (e) Compute $\llbracket \epsilon \rrbracket_0^{\text{exp}} \leftarrow \llbracket \gamma v \rrbracket_0 - \llbracket \gamma \rrbracket_0 \cdot v'$. Receive $\llbracket \epsilon \rrbracket_0$ from adversary. Send $\llbracket \epsilon \rrbracket_1 \leftarrow -\llbracket \epsilon \rrbracket_0^{\text{exp}} - \gamma \cdot \delta$.
- (f) If $\llbracket \epsilon \rrbracket_0 + \llbracket \epsilon \rrbracket_1 \neq 0$, abort.
- (g) Route output v from $F_{\text{ArithPermCirc}}$ to the honest party.

Figure 20: Simulator for Reconstruct operation when P_0 is corrupted, used as a subprotocol in the simulator for the proof of security of the online protocol FLOSS.

In Figure 22, we describe our preprocessing protocol for generating authenticated shuffle tuples with the Mohassel et al. [MSS14] malicious-secure permutation network. We divide our preprocessing protocol into three parts: the generation of forward shuffle tuples, the generation of inverse shuffle tuples, and the application of the shuffle tuple key. First, we describe how to generate forward shuffle tuples $(\langle \pi(\vec{a}) \rangle, \langle \pi(\vec{b}) \rangle)$ from P_0 's permutation π and P_1 's vectors $\vec{a}$ and $\vec{b}$ using ShufTupGen (Figure 23). Then, we describe how to generate inverse shuffle tuples $(\langle \pi^{-1}(\vec{a}) \rangle, \langle \pi^{-1}(\vec{b}) \rangle)$ from P_0 's permutation π and P_1 's vectors $\vec{a}$ and $\vec{b}$ using InvShufTupGen (Figure 24). Finally, we describe how to obtain κ -authenticated shares $(\langle \pi(\vec{a}) \rangle^{(\kappa)}, \langle \pi(\vec{b}) \rangle^{(\kappa)})$ (resp. $(\langle \pi^{-1}(\vec{a}) \rangle^{(\kappa)}, \langle \pi^{-1}(\vec{b}) \rangle^{(\kappa)})$) from the forward shuffle tuples $(\langle \pi(\vec{a}) \rangle, \langle \pi(\vec{b}) \rangle)$ (resp. inverse shuffle tuples $(\langle \pi^{-1}(\vec{a}) \rangle, \langle \pi^{-1}(\vec{b}) \rangle)$) using ApplyKappaPrep (Figure 25).

E.1 Permutation Network

Switching Network. A switching network is a directed graph (V, E) defined by vertices V and edges E , where each vertex has two incoming edges and two outgoing edges. The binary switches are represented as vertices of the graph $(V = (v_1, \dots, v_n))$, which we assume to be topologically ordered. The wires are represented as edges of the graph $(E = (e_1, \dots, e_m))$. The value of the wire is represented by the mapping $\text{Val} : E \rightarrow \mathbb{F}$. The control bits of the switches are represented by $C = (c_1, \dots, c_n) \in \{0, 1\}^n$. The input wires are represented by the set $I = \{i_1, \dots, i_{m_{\text{in}}}\} \subset E$, where m_{in} is the number of inputs of the switching network. The input wires are the leftmost wires in the switching network. The output wires are represented by the set $O = \{o_1, \dots, o_{m_{\text{out}}}\} \subset E$, where m_{out} is the number of outputs of the switching network. The output wires are the rightmost wires in the switching network.

We define two mappings from switch to edges: $\text{In} : V \rightarrow E^2$ and $\text{Out} : V \rightarrow E^2$.

For all $j \in [n]$, where n is the number of switches:

- $\text{In}(v_j) \rightarrow (\text{in}_0, \text{in}_1)$ maps switch v_j to its input wire indices $(\text{in}_0, \text{in}_1)$, where in_0 is the index of the top input wire and in_1 is the index of the bottom input wire.
- $\text{Out}(v_j) \rightarrow (\text{out}_0, \text{out}_1)$ maps switch v_j to its output wire indices $(\text{out}_0, \text{out}_1)$, where out_0 is the index of the top output wire and out_1 is the index of the bottom output wire.

From these definitions, when switch v_j with input wire indices $(\text{in}_0, \text{in}_1)$ and output wire indices $(\text{out}_0, \text{out}_1)$ is set to control bit b_j then:

- $\text{Val}(\text{in}_0) = \text{Val}(\text{out}_{c_j})$
- $\text{Val}(\text{in}_1) = \text{Val}(\text{out}_{1-c_j})$

In other words, the switch v_j sets the output wire values equal to its corresponding input wire values when $b_j = 0$. On the other hand, the switch sets the output wire values equal to the opposite input wire value when $b_j = 1$, “switching” the values of the wires.

Permutation Network. A permutation network is a switching network with three properties:

1. The number of input wires is equal to the number of output wires. For simplicity, we represent the number of input/output wires as m' , where $m' = m_{\text{in}} = m_{\text{out}}$.
2. Soundness: $\forall c \in C, \exists \pi \in \mathbb{S}_{m'}, \{(\text{Val}(i_j), j)\}_{j \in [m']} = \{(\text{Val}(o_{\pi^{-1}(j)}), \pi^{-1}(j))\}_{j \in [m']}$
 - In other words, all possible assignments of control bits in the switching network must correspond to some m' -length permutation, where the value of the input wire is the same as the value of the output wire indexed by the permutation applied to the input wire's index.
3. Completeness: $\forall \pi \in \mathbb{S}_{m'}, \exists c \in C, \{(\text{Val}(i_j), j)\}_{j \in [m']} = \{(\text{Val}(o_{\pi^{-1}(j)}), \pi^{-1}(j))\}_{j \in [m']}$
 - In other words, all m' -length permutations must correspond to some assignment of control bits in the switching network, where the value of the input wire is the same as the value of the output wire indexed by the permutation applied to the input wire's index.

According to these definitions, there is a sequence of control bits $(c_1, \dots, c_n)$ uniquely encoding $\pi \in \mathbb{S}_{m'}$ that the permuter locally computes from the permutation network topology (e.g. m' -length Waksman network [Wak68]). Given m' terminals (input/output wires), the Waksman permutation network contains $m' \cdot \lceil \log_2(m') \rceil - m' + 1$ switches.

$P_0.\text{Coin}(n \in \mathbb{F}) \leftrightarrow P_1.\text{Coin}(n \in \mathbb{F})$ Output: $\langle \vec{R} \rangle$, where $\vec{R} \leftarrow \mathbb{F}^n$ 1. $\vec{R}_b \leftarrow \mathbb{F}^n$ 2. $\langle \vec{R}_b \rangle \leftarrow P_b.\text{Input}(\langle \vec{R}_b \rangle_b, \vec{R}_b) \leftrightarrow P_{1-b}.\text{RcvInput}(\langle \vec{R}_b \rangle_{1-b})$ 3. $\langle \vec{R}_{1-b} \rangle \leftarrow P_b.\text{RcvInput}(\langle \vec{R}_{1-b} \rangle_b) \leftrightarrow P_{1-b}.\text{Input}(\langle \vec{R}_{1-b} \rangle_{1-b}, \vec{R}_{1-b})$ 4. $\langle \vec{R} \rangle \leftarrow \langle \vec{R}_b \rangle + \langle \vec{R}_{1-b} \rangle$

Figure 21: Subroutine that generates γ -authenticated shares of a vector of n unknown random field elements.

E.2 Malicious-Secure Permutation Network Construction: Forward Shuffle Tuples

We describe the construction of the malicious-secure permutation network from Mohassel et al. [MSS14]. We can construct subprotocols to generate forward shuffle tuples and inverse shuffle tuples from the above permutation network construction, described in Figure 23 and Figure 24. Here, we describe first the construction of forward shuffle tuples ShufTupGen from the permutation network. We use subroutines from SPDZ since throughout the protocol, we use arithmetic circuit operations such as Reconstruct and Mul.

For simplicity, we describe a subroutine $P_b.\text{Coin}(n) \leftrightarrow P_{1-b}.\text{Coin}(n)$ to generate γ -authenticated shares of a length- n vector of unknown random field elements $\vec{R} \in \mathbb{F}^n$ to both parties. It is described in Figure 21. Note that when running the Mohassel et al. [MSS14] shuffle protocol standalone, the circuit preparation phase can be performed in offline preprocessing.

Preparing the circuit. This portion of the forward shuffle tuple generation protocol prepares the permutation network circuit, which is independent of the shuffle-tuple vectors $\vec{a}$ and $\vec{b}$. First, the parties must generate random authenticated values for each wire in the permutation network. For each wire $e \in E$, the parties must generate a random data blind $\langle r_e \rangle$ and random MAC blind $\langle l_e \rangle$. The parties call the $\text{Coin}(|E|) = \text{Coin}(2m \log_2(m) - m + 1)$ subroutine twice to generate data blinds and authentication blinds for all wires in the network. The MAC blind $\langle l_e \rangle$ is used to authenticate the data blind $\langle r_e \rangle$ as P_0 evaluates the permutation network. To ensure that the MAC blinds are consistent with the data blinds, the parties sample a random multiplicative key $\langle l \rangle$ using the $\text{Coin}(1)$ subroutine. Since P_1 supplies two independent input vectors, $\vec{a}$ and $\vec{b}$, the parties must repeat the above process twice, generating new data blinds, MAC blinds, and multiplicative key for each vector.

The next step is for P_0 to locally parse π into topologically-ordered control bits $(c_1, \dots, c_k) \in \{0, 1\}^k$ for switching network of input length $m = |\pi|$. P_0 uses a permutation network, such as a Waksman network [Wak68], to convert its permutation into control bits. P_0 will then share its control bits with P_1 via SPDZ's Input subroutine. Using authenticated shares of the control bits $\langle c_i \rangle$, P_1 verifies that P_0 inputted properly formatted bits with the bit-check circuit $\langle c_i \rangle \cdot (1 - \langle c_i \rangle)$. P_0 and P_1 Reconstruct the output, and if $c_i \cdot (1 - c_i) \neq 0$, the parties output **abort**.

The last step is for P_0 and P_1 to route the blind values of the switches' input wires to the output wires using P_0 's inputted control bits. Observe that for a switch in the permutation network, the value of the output wire only depends on the switch's control bit and the values of the input wires. Since the parties already precomputed blinds for each wire, and P_0 already inputted its control bits, the parties can compute the proper routing of the input blinds to output blinds of each switch in the network simultaneously.

For each switch index $i \in [k]$, the parties compute the following in parallel:

$$\begin{aligned}
\text{in}_0, \text{in}_1 &\leftarrow \text{In}(v_i) \\
\text{out}_0, \text{out}_1 &\leftarrow \text{Out}(v_i) \\
\langle s_{\text{in}_0} \rangle &\leftarrow (1 - \langle c_i \rangle) \cdot (\langle r_{\text{out}_0} \rangle - \langle r_{\text{in}_0} \rangle) + \langle c_i \rangle \cdot (\langle r_{\text{out}_1} \rangle - \langle r_{\text{in}_0} \rangle) \\
\langle s_{\text{in}_1} \rangle &\leftarrow (1 - \langle c_i \rangle) \cdot (\langle r_{\text{out}_1} \rangle - \langle r_{\text{in}_1} \rangle) + \langle c_i \rangle \cdot (\langle r_{\text{out}_0} \rangle - \langle r_{\text{in}_1} \rangle)
\end{aligned}$$

For each switch index $i \in [k]$, the parties reconstruct the wire routing in parallel:

$$\begin{aligned}
\text{in}_0, \text{in}_1 &\leftarrow \text{In}(v_i) \\
\text{out}_0, \text{out}_1 &\leftarrow \text{Out}(v_i) \\
s_{\text{in}_0} &\leftarrow \text{Reconstruct}(\langle s_{\text{in}_0} \rangle) \\
s_{\text{in}_1} &\leftarrow \text{Reconstruct}(\langle s_{\text{in}_1} \rangle)
\end{aligned}$$

For each switch index $i \in [k]$, the parties compute the following in parallel:

$$\begin{aligned} \text{in}_0, \text{in}_1 &\leftarrow \text{In}(v_i) \\ \text{out}_0, \text{out}_1 &\leftarrow \text{Out}(v_i) \\ \langle \tau_{\text{in}_0} \rangle &\leftarrow (1 - \langle c_i \rangle) \cdot (\langle l_{\text{out}_0} \rangle - \langle l_{\text{in}_0} \rangle) + \langle c_i \rangle \cdot (\langle l_{\text{out}_1} \rangle - \langle l_{\text{in}_0} \rangle) + s_{\text{in}_0} \cdot \langle \iota \rangle \\ \langle \tau_{\text{in}_1} \rangle &\leftarrow (1 - \langle c_i \rangle) \cdot (\langle l_{\text{out}_1} \rangle - \langle l_{\text{in}_1} \rangle) + \langle c_i \rangle \cdot (\langle l_{\text{out}_0} \rangle - \langle l_{\text{in}_1} \rangle) + s_{\text{in}_1} \cdot \langle \iota \rangle \end{aligned}$$

For each switch index $i \in [k]$, the parties reconstruct the authenticated wire routing in parallel:

$$\begin{aligned} \tau_{\text{in}_0} &\leftarrow \text{Reconstruct}(\langle \tau_{\text{in}_0} \rangle) \\ \tau_{\text{in}_1} &\leftarrow \text{Reconstruct}(\langle \tau_{\text{in}_1} \rangle) \end{aligned}$$

Evaluating the permutation network. In this portion of the protocol, the parties prepare the inputs to the permutation network circuit by adding their input shares added to the input wire blinds. We repeat this process twice for $x \in \{\vec{a}, \vec{b}\}$.

For each input index $j \in [m]$, the parties compute the following in parallel:

$$\langle t_{i_j} \rangle \leftarrow \langle x_j \rangle + \langle r_{i_j} \rangle$$

For each input index $j \in [m]$, the parties reconstruct the input wire values in parallel:

$$t_{i_j} \leftarrow \text{Reconstruct}(\langle t_{i_j} \rangle)$$

For each input index $j \in [m]$, the parties compute the following in parallel:

$$\langle \tau_{i_j} \rangle \leftarrow \langle l_{i_j} \rangle + t_{i_j} \cdot \langle \iota \rangle$$

For each input index $j \in [m]$, the parties reconstruct the authenticated input wire values in parallel:

$$\tau_{i_j} \leftarrow \text{Reconstruct}(\langle \tau_{i_j} \rangle)$$

Next, the evaluator locally evaluates the permutation network circuit using the input wire data values and MAC values, starting from the input wires and ending at the output wires.

For each switch index $i \in [k]$, P_0 evaluates the network via topologically-ordered switches:

$$\begin{aligned} \text{in}_0, \text{in}_1 &\leftarrow \text{In}(v_i) \\ \text{out}_0, \text{out}_1 &\leftarrow \text{Out}(v_i) \\ \langle t_{\text{out}_{c_i}} \rangle &\leftarrow s_{\text{in}_0} + t_{\text{in}_0} \\ \langle t_{\text{out}_{1-c_i}} \rangle &\leftarrow s_{\text{in}_1} + t_{\text{in}_1} \\ \langle \tau_{\text{out}_{c_i}} \rangle &\leftarrow \Delta_{\text{in}_0} + \tau_{\text{in}_0} \\ \langle \tau_{\text{out}_{1-c_i}} \rangle &\leftarrow \Delta_{\text{in}_1} + \tau_{\text{in}_1} \end{aligned}$$

Finally, the parties perform an authentication check on the output wires' data values (t_{o_j}) and MAC values (τ_{o_j}) to verify the permuter computed the network correctly. For each output index $j \in [m]$, P_0 sends (t_{o_j}, τ_{o_j}) in parallel to P_1 . Then, the parties compute a check circuit, corresponding to Step 4 in Figure 23. For each output index $j \in [m]$, the parties compute the following in parallel:

$$\langle \gamma_{o_j} \rangle \leftarrow \langle l_{o_j} \rangle + t_{o_j} \cdot \langle \iota \rangle - \tau_{o_j}$$

For each output index $j \in [m]$, the parties reconstruct the authentication check in parallel:

$$\gamma_{o_j} \leftarrow \text{Reconstruct}(\langle \gamma_{o_j} \rangle)$$

Lastly, the parties locally check if $\gamma_{o_j} = 0$; if not, the parties output `abort`. If the check passes, P_0 and P_1 output their shares of the authenticated shuffle tuple:

$$\begin{aligned} \langle \pi(x)_j \rangle_0 &\leftarrow t_{o_j} - \langle r_{o_j} \rangle_0 \\ \langle \pi(x)_j \rangle_1 &\leftarrow t_{o_j} - \langle r_{o_j} \rangle_1 \end{aligned}$$

E.3 Constructing Inverse Shuffle Tuples

The inverse shuffle tuple generation protocol InvShufTupGen is similar to the forward shuffle tuple generation protocol ShufTupGen , but there are three main differences.

Reusing control bits. First, the parties must reuse the same shared control bits $(\langle c_i \rangle)_{i \in [k]}$ as in the forward shuffle tuple generation protocol. The same control bits $[c_i]$ imply that the same permutation π is used in both protocols. This ensures that the inverse shuffle tuples constructed by InvShufTupGen use the same permutation π as the forward shuffle tuples constructed by ShufTupGen . Conveniently, reusing a permutation eliminates k total Input and Reconstruct operations, since there is no need to reshare the control bits and perform another bit-equality check.

Preparing the circuit: reversing the wire routing. Second, recall for a switch v_i , the right wires correspond to out_0 and out_1 from $\text{Out}(v_i)$, and the left wires correspond to in_0 and in_1 from $\text{In}(v_i)$. Because the permutation network evaluation is reversed, the right wires of the switch (out) are routed to the left wires (in), instead of routing left wires to right wires in ShufTupGen . We modify the circuit generation step of our offline portion to instead compute the *reverse* wire routing: For each switch index $i \in [k]$, the parties compute the following in parallel:

$$\begin{aligned} \text{in}_0, \text{in}_1 &\leftarrow \text{In}(v_i) \\ \text{out}_0, \text{out}_1 &\leftarrow \text{Out}(v_i) \\ \langle s_{\text{out}_0} \rangle &\leftarrow (1 - \langle c_i \rangle) \cdot (\langle r_{\text{in}_0} \rangle - \langle r_{\text{out}_0} \rangle) + \langle c_i \rangle \cdot (\langle r_{\text{in}_1} \rangle - \langle r_{\text{out}_0} \rangle) \\ \langle s_{\text{out}_1} \rangle &\leftarrow (1 - \langle c_i \rangle) \cdot (\langle r_{\text{in}_1} \rangle - \langle r_{\text{out}_1} \rangle) + \langle c_i \rangle \cdot (\langle r_{\text{in}_0} \rangle - \langle r_{\text{out}_1} \rangle) \end{aligned}$$

For each switch index $i \in [k]$, the parties reconstruct the *reverse* wire routing in parallel:

$$\begin{aligned} \text{in}_0, \text{in}_1 &\leftarrow \text{In}(v_i) \\ \text{out}_0, \text{out}_1 &\leftarrow \text{Out}(v_i) \\ s_{\text{out}_0} &\leftarrow \text{Reconstruct}(\langle s_{\text{out}_0} \rangle) \\ s_{\text{out}_1} &\leftarrow \text{Reconstruct}(\langle s_{\text{out}_1} \rangle) \end{aligned}$$

For each switch index $i \in [k]$, the parties compute the following in parallel:

$$\begin{aligned} \text{in}_0, \text{in}_1 &\leftarrow \text{In}(v_i) \\ \text{out}_0, \text{out}_1 &\leftarrow \text{Out}(v_i) \\ \langle \tau_{\text{out}_0} \rangle &\leftarrow (1 - \langle c_i \rangle) \cdot (\langle l_{\text{in}_0} \rangle - \langle l_{\text{out}_0} \rangle) + \langle c_i \rangle \cdot (\langle l_{\text{in}_1} \rangle - \langle l_{\text{out}_0} \rangle) + s_{\text{out}_0} \cdot \langle \iota \rangle \\ \langle \tau_{\text{out}_1} \rangle &\leftarrow (1 - \langle c_i \rangle) \cdot (\langle l_{\text{in}_1} \rangle - \langle l_{\text{out}_1} \rangle) + \langle c_i \rangle \cdot (\langle l_{\text{in}_0} \rangle - \langle l_{\text{out}_1} \rangle) + s_{\text{out}_1} \cdot \langle \iota \rangle \end{aligned}$$

For each switch index $i \in [k]$, the parties reconstruct the authenticated *reverse* wire routing in parallel:

$$\begin{aligned} \tau_{\text{out}_0} &\leftarrow \text{Reconstruct}(\langle \tau_{\text{out}_0} \rangle) \\ \tau_{\text{out}_1} &\leftarrow \text{Reconstruct}(\langle \tau_{\text{out}_1} \rangle) \end{aligned}$$

Evaluating the permutation network in reverse. Third, the parties must instantiate their inputs x_j with the output (rightmost) wires o_j in the permutation network structure, instead of the input (leftmost) wires i_j . We change the following steps of the online portion to initialize with the output wires:

For each output index $j \in [m]$, the parties compute the following in parallel:

$$\langle t_{o_j} \rangle \leftarrow \langle x_j \rangle + \langle r_{o_j} \rangle$$

For each output index $j \in [m]$, the parties reconstruct the output wire values in parallel:

$$t_{o_j} \leftarrow \text{Reconstruct}(\langle t_{o_j} \rangle)$$

For each output index $j \in [m]$, the parties compute the following in parallel:

$$\langle \tau_{o_j} \rangle \leftarrow \langle l_{o_j} \rangle + t_{o_j} \cdot \langle \iota \rangle$$

For each output index $j \in [m]$, the parties reconstruct the authenticated output wire values in parallel:

$$\tau_{o_j} \leftarrow \text{Reconstruct}(\langle \tau_{o_j} \rangle)$$

P_0 must compute the permutation network circuit starting from the output wires and ending at the input wires. In other words, P_0 must compute the permutation network circuit traversing from right to left, instead of left to right in ShufTupGen. We change the following steps of the online portion so P_0 evaluates the network in reverse:

For each switch index $i \in [k_{\text{rev}}]$, P_0 evaluates the network via *reverse* topologically-ordered switches:

$$\begin{aligned} \text{in}_0, \text{in}_1 &\leftarrow \text{In}(v_i) \\ \text{out}_0, \text{out}_1 &\leftarrow \text{Out}(v_i) \\ \langle t_{\text{inc}_i} \rangle &\leftarrow s_{\text{out}_0} + t_{\text{out}_0} \\ \langle t_{\text{in}_1 - c_i} \rangle &\leftarrow s_{\text{out}_1} + t_{\text{out}_1} \\ \langle \tau_{\text{inc}_i} \rangle &\leftarrow \Delta_{\text{out}_0} + \tau_{\text{out}_0} \\ \langle \tau_{\text{in}_1 - c_i} \rangle &\leftarrow \Delta_{\text{out}_1} + \tau_{\text{out}_1} \end{aligned}$$

The parties perform an authentication check on the input wires' data values (t_{i_j}) and MAC values (τ_{i_j}) to verify the permuter computed the network correctly, instead of the output wires. P_1 sends (t_{i_j}, τ_{i_j}) in parallel to P_0 . Then for each input index $j \in [m]$, the parties compute the following in parallel:

$$\langle \gamma_{i_j} \rangle \leftarrow \langle l_{i_j} \rangle + t_{i_j} \cdot \langle \iota \rangle - \tau_{i_j}$$

For each input index $j \in [m]$, the parties reconstruct the authentication check in parallel:

$$\gamma_{i_j} \leftarrow \text{Reconstruct}(\langle \gamma_{i_j} \rangle)$$

Lastly, P_0 and P_1 's shares of the inverse shuffle tuple are defined with respect to the input wire values (t_{i_j}):

$$\begin{aligned} \langle \pi^{-1}(x)_j \rangle_0 &\leftarrow t_{i_j} - \langle r_{i_j} \rangle_0 \\ \langle \pi^{-1}(x)_j \rangle_1 &\leftarrow -\langle r_{i_j} \rangle_1 \end{aligned}$$

E.4 Constructing κ -Authenticated Shuffle Tuples

ApplyKappaPrep constructs κ -authenticated shuffle tuples $\langle \vec{a} \rangle^{(\kappa)}$ and $\langle \vec{b} \rangle^{(\kappa)}$ from shuffle tuples $\langle \vec{a} \rangle$ and $\langle \vec{b} \rangle$. First, P_0 shares κ with P_1 using Input. Then, the parties use Mul to multiply their shares $\langle \vec{a} \rangle$ and $\langle \vec{b} \rangle$ with shares $\langle \kappa \rangle$ to receive $\langle \kappa \cdot \vec{a} \rangle$ and $\langle \kappa \cdot \vec{b} \rangle$.

To verify $\langle \kappa \cdot \vec{a} \rangle$ and $\langle \kappa \cdot \vec{b} \rangle$ were computed with unmodified shares of $\vec{a}$ and $\vec{b}$, the parties perform a blind authentication check with respect to γ on the shuffle tuple shares $\langle \vec{a} \rangle$ and $\langle \vec{b} \rangle$ and the κ -authenticated shares $\langle \kappa \cdot \vec{a} \rangle$ and $\langle \kappa \cdot \vec{b} \rangle$. Parties concatenate all of their data shares $\llbracket \vec{a} \rrbracket || \llbracket \kappa \cdot \vec{a} \rrbracket || \llbracket \vec{b} \rrbracket || \llbracket \kappa \cdot \vec{b} \rrbracket$ and γ -authenticated shares $\llbracket \gamma \vec{a} \rrbracket || \llbracket \gamma \kappa \cdot \vec{a} \rrbracket || \llbracket \gamma \vec{b} \rrbracket || \llbracket \gamma \kappa \cdot \vec{b} \rrbracket$ to authenticate the shuffle tuples in one-shot. Lastly, the parties need to verify that the multiplications (from multiplying $\langle \kappa \rangle$ with shuffle tuple shares) were computed correctly; they do this by emptying the authentication check queue and running the Auth subroutine on the remaining opened multiplications. If Auth passes, the parties output shares of their κ -authenticated shuffle tuples:

$$\begin{aligned} \langle \vec{a} \rangle_0^{(\kappa)} &\leftarrow (\llbracket \vec{a} \rrbracket, \llbracket \kappa \cdot \vec{a} \rrbracket) \\ \langle \vec{b} \rangle_0^{(\kappa)} &\leftarrow (\llbracket \vec{b} \rrbracket, \llbracket \kappa \cdot \vec{b} \rrbracket) \end{aligned}$$

E.5 Security

We present a corollary for our preprocessing protocol PermNetSTPrep in the $(F_{\text{SPDZPrep}}, F_{\text{Comm}})$ -hybrid model.

Corollary 1. Let $\mathbb{F}$ be a finite field with $|\mathbb{F}| \geq 2^\lambda$. The protocol PermNetSTPrep $[\mathbb{F}]$ (Figure 22) UC-realizes the ideal functionality $F_{\text{ShufTuplePrep}}[\mathbb{F}]$ (Figure 18) in the $(F_{\text{SPDZPrep}}, F_{\text{Comm}})$ -hybrid model.

Proof sketch: If the corrupted party is sender P_1 , the simulator samples two random vectors $\vec{a}, \vec{b} \leftarrow_{\$} \mathbb{F}^n$. Then, in either case, the simulator simulates $\mathcal{F}_{\text{SPDZPrep}}$ and simulates arithmetic circuit operations using the SPDZ simulator (see S_{Online} in Figure 19) exactly as in the protocol specification for steps 2 and 3 of PermNetSTPrep (Figure 22) to retrieve shares $\langle \vec{a} \rangle$ and $\langle \vec{b} \rangle$. The security of subprotocols $\text{ShufTupGen}[\mathbb{F}]$ and $\text{InvShufTupGen}[\mathbb{F}]$ follow directly from Theorem 3 of Mohassel et al. [MSS14]. The permuter P_0 must correctly guess ι in $\text{ShufTupGen}[\mathbb{F}]$ or $\underline{\iota}$ in $\text{InvShufTupGen}[\mathbb{F}]$ to deviate from either protocol without detection. By union bound, this occurs with probability $\leq \frac{2}{|\mathbb{F}|}$, which is negligible in λ since $|\mathbb{F}| \geq 2^\lambda$. In $\text{ApplyKappaPrep}[\mathbb{F}]$, if the corrupt party is the permuter P_0 , the simulator samples $\kappa \leftarrow_{\$} \mathbb{F}$. The simulator acts like S_{Online} (Figure 19) and follows the protocol specification for steps 1–4 of ApplyKappaPrep (Figure 25) to retrieve shares $\langle \vec{a}^{(\kappa)} \rangle$ and $\langle \vec{b}^{(\kappa)} \rangle$.

F Alternate Circuit for Arbitrary Permutation Application

Parties can avoid using inverse permutation shuffles for Sort as in ApplyPerm (Figure 3) by using our alternate circuit construction ApplyPermAlt . Instead, parties need to preprocess eight forward permutation shuffle tuples (four for π_0 and four for π_1), as opposed to four total in ApplyPerm (two forward and two inverse). The description for ApplyPermAlt is provided in Figure 26.

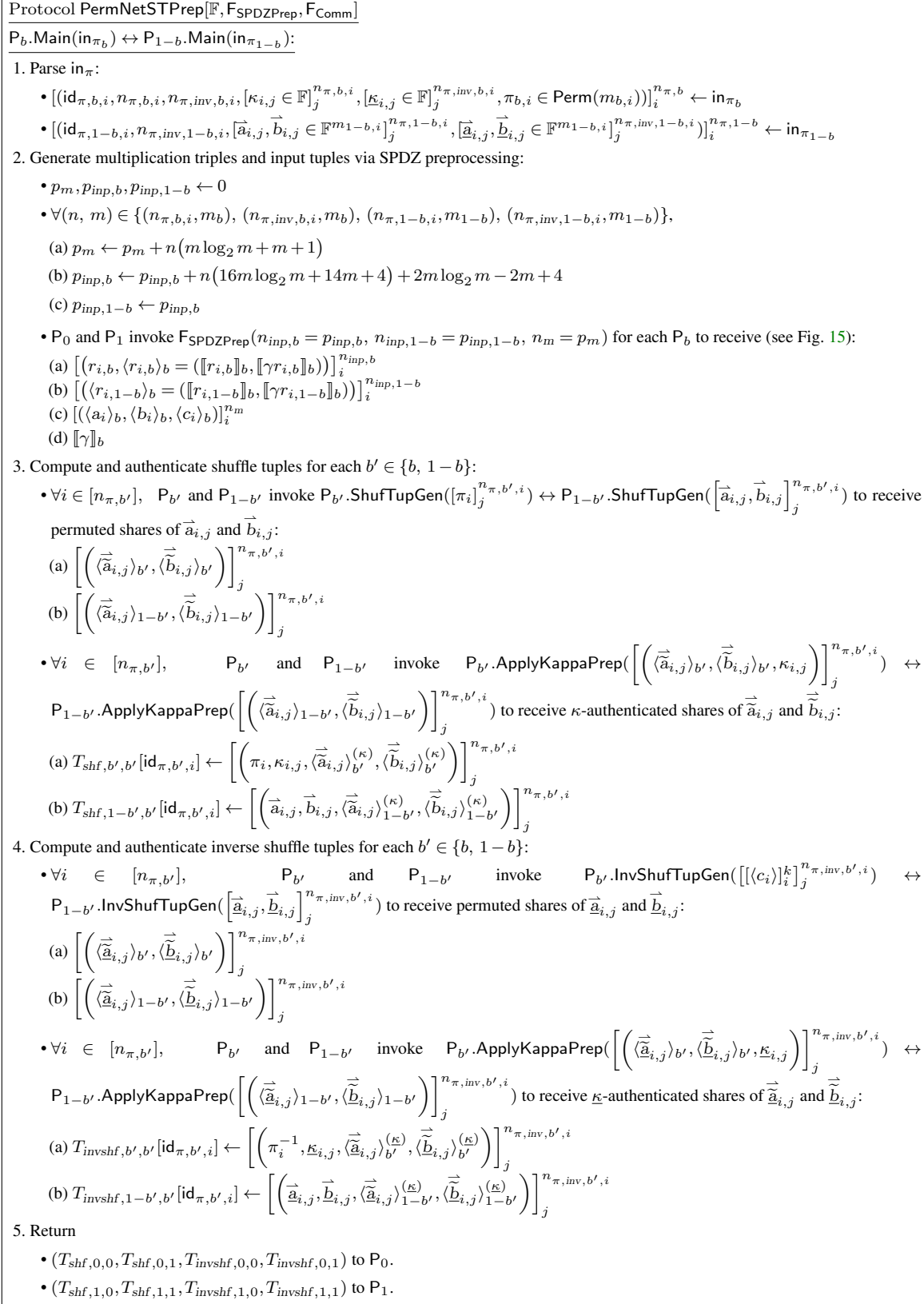


Figure 22: Protocol for preprocessing of authenticated shuffle tuples consumed in FLOSS in the (FSPDZPrep, FComm)-hybrid.



Figure 23: Subprotocol for generating forward authenticated shuffle tuples in the $(F_{\text{SPDZPrep}}, F_{\text{Comm}})$ -hybrid.

$P_b.\text{InvShufTupGen}(\left[\left[\langle c_i \rangle\right]_i^k\right]_j^n) \leftrightarrow P_{1-b}.\text{InvShufTupGen}(\left[\vec{a}_j, \vec{b}_j\right]_j^n):$
1. Prepare the permutation network circuit, which is independent of the shuffle-tuple vectors: • P_b and P_{1-b} reuse their control bit shares $[\langle c_i \rangle]_i^k$ from ShufTupGen. • $\forall i \in [k]$, P_b and P_{1-b} compute and reconstruct $\langle c_i \rangle \cdot (1 - \langle c_i \rangle)$. If $\langle c_i \rangle \cdot (1 - \langle c_i \rangle) \neq 0$, output abort. • $\forall j \in [n]$, P_b and P_{1-b} generate data and authenticated blinds for each wire in a and b's switching network: for $e \in E$, $\langle r_{a,e,j} \rangle, \langle l_{a,e,j} \rangle, \langle r_{b,e,j} \rangle, \langle l_{b,e,j} \rangle \leftarrow P_b.\text{Coin}(4) \leftrightarrow P_{1-b}.\text{Coin}(4)$ • $\forall j \in [n]$, P_b and P_{1-b} generate one-time authentication key for a and b: $\langle \iota_{a,j} \rangle, \langle \iota_{b,j} \rangle \leftarrow P_b.\text{Coin}(2) \leftrightarrow P_{1-b}.\text{Coin}(2)$ • $\forall j \in [n]$, P_b and P_{1-b} compute the following, then reconstruct to obtain $s_{a,out_0,j}$, $s_{a,out_1,j}$, $s_{b,out_0,j}$, and $s_{b,out_1,j}$: (a) $\langle s_{a,out_0,j} \rangle \leftarrow (1 - \langle c_i \rangle) \cdot (\langle r_{a,in_0,j} \rangle - \langle r_{a,out_0,j} \rangle) + \langle c_i \rangle \cdot (\langle r_{a,in_1,j} \rangle - \langle r_{a,out_0,j} \rangle)$ (b) $\langle s_{a,out_1,j} \rangle \leftarrow (1 - \langle c_i \rangle) \cdot (\langle r_{a,in_1,j} \rangle - \langle r_{a,out_1,j} \rangle) + \langle c_i \rangle \cdot (\langle r_{a,in_0,j} \rangle - \langle r_{a,out_1,j} \rangle)$ (c) $\langle s_{b,out_0,j} \rangle \leftarrow (1 - \langle c_i \rangle) \cdot (\langle r_{b,in_0,j} \rangle - \langle r_{b,out_0,j} \rangle) + \langle c_i \rangle \cdot (\langle r_{b,in_1,j} \rangle - \langle r_{b,out_0,j} \rangle)$ (d) $\langle s_{b,out_1,j} \rangle \leftarrow (1 - \langle c_i \rangle) \cdot (\langle r_{b,in_1,j} \rangle - \langle r_{b,out_1,j} \rangle) + \langle c_i \rangle \cdot (\langle r_{b,in_0,j} \rangle - \langle r_{b,out_1,j} \rangle)$ • P_b and P_{1-b} compute the following, then reconstruct to obtain $\Delta_{a,out_0,j}$, $\Delta_{a,out_1,j}$, $\Delta_{b,out_0,j}$, and $\Delta_{b,out_1,j}$: (a) $\langle \Delta_{a,out_0,j} \rangle \leftarrow (1 - \langle c_i \rangle) \cdot (\langle l_{a,in_0,j} \rangle - \langle l_{a,out_0,j} \rangle) + \langle c_i \rangle \cdot (\langle l_{a,in_1,j} \rangle - \langle l_{a,out_0,j} \rangle) + s_{a,out_0,j} \cdot \langle \iota_{a,j} \rangle$ (b) $\langle \Delta_{a,out_1,j} \rangle \leftarrow (1 - \langle c_i \rangle) \cdot (\langle l_{a,in_1,j} \rangle - \langle l_{a,out_1,j} \rangle) + \langle c_i \rangle \cdot (\langle l_{a,in_0,j} \rangle - \langle l_{a,out_1,j} \rangle) + s_{a,out_1,j} \cdot \langle \iota_{a,j} \rangle$ (c) $\langle \Delta_{b,out_0,j} \rangle \leftarrow (1 - \langle c_i \rangle) \cdot (\langle l_{b,in_0,j} \rangle - \langle l_{b,out_0,j} \rangle) + \langle c_i \rangle \cdot (\langle l_{b,in_1,j} \rangle - \langle l_{b,out_0,j} \rangle) + s_{b,out_0,j} \cdot \langle \iota_{b,j} \rangle$ (d) $\langle \Delta_{b,out_1,j} \rangle \leftarrow (1 - \langle c_i \rangle) \cdot (\langle l_{b,in_1,j} \rangle - \langle l_{b,out_1,j} \rangle) + \langle c_i \rangle \cdot (\langle l_{b,in_0,j} \rangle - \langle l_{b,out_1,j} \rangle) + s_{b,out_1,j} \cdot \langle \iota_{b,j} \rangle$ 2. Prepare inputs for the permutation network circuit on the authenticated input shares: • $\forall j \in [n]$, P_{1-b} creates authenticated input shares of its randomized vectors $(\vec{a}_j, \vec{b}_j)$: (a) $(\langle \vec{a}_j \rangle, \langle \vec{b}_j \rangle) \leftarrow P_{1-b}.\text{Input}((\langle \vec{a}_j \rangle_{1-b}, \langle \vec{b}_j \rangle_{1-b}), (\vec{a}_j, \vec{b}_j)) \leftrightarrow P_b.\text{RcvInput}((\langle \vec{a}_j \rangle_b, \langle \vec{b}_j \rangle_b))$ • $\forall j \in [n]$, P_b and P_{1-b} prepare wire values for each paired output index and output wire index ($e \in [m], o_e$) for the switching network, and then reconstruct to obtain $t_{a,i_e,j}$ and $t_{b,i_e,j}$: (a) $\langle t_{a,o_e,j} \rangle \leftarrow \langle \vec{a}_{e,j} \rangle + \langle r_{a,o_e,j} \rangle$ (b) $\langle t_{b,o_e,j} \rangle \leftarrow \langle \vec{b}_{e,j} \rangle + \langle r_{b,o_e,j} \rangle$ • $\forall j \in [n]$, P_b and P_{1-b} prepare authenticated wire values for each output wire index o_e for the switching network, and then reconstruct to obtain $\tau_{a,i_e,j}$ and $\tau_{b,i_e,j}$: (a) $\langle \tau_{a,o_e,j} \rangle \leftarrow \langle l_{a,o_e,j} \rangle + t_{a,o_e,j} \cdot \langle \iota_{a,j} \rangle$ (b) $\langle \tau_{b,o_e,j} \rangle \leftarrow \langle l_{b,o_e,j} \rangle + t_{b,o_e,j} \cdot \langle \iota_{b,j} \rangle$ 3. P_b iterates through all the switches reverse-topologically indexed for each $i \in [k]$ in the network to evaluate the input wires: • $in_0, in_1 \leftarrow \text{In}(v_i)$ • $out_0, out_1 \leftarrow \text{Out}(v_i)$ • $t_{a,in_{c_i},j} \leftarrow s_{a,out_0,j} + t_{a,out_0,j}$ • $t_{a,in_{1-c_i},j} \leftarrow s_{a,out_1,j} + t_{a,out_1,j}$ • $\tau_{b,in_{c_i},j} \leftarrow \Delta_{b,out_0,j} + \tau_{b,in_0,j}$ • $\tau_{b,in_{1-c_i},j} \leftarrow \Delta_{b,out_1,j} + \tau_{b,in_1,j}$ 4. For each paired input index and input wire index ($e \in [m], i_e$), P_b and P_{1-b} perform authentication check with t and τ: • P_b sends input wire values $t_{a,i_e,j}$, $t_{b,i_e,j}$ and authenticated input wire values $\tau_{a,i_e,j}$, $\tau_{b,i_e,j}$ to P_{1-b}. • P_b and P_{1-b} compute the following, then reconstruct to obtain $\gamma_{a,e,j}$ and $\gamma_{b,e,j}$: (a) $\langle \gamma_{a,e,j} \rangle \leftarrow \langle l_{a,i_e,j} \rangle + t_{a,i_e,j} \cdot \langle \iota_{a,j} \rangle$ (b) $\langle \gamma_{b,e,j} \rangle \leftarrow \langle l_{b,i_e,j} \rangle + t_{b,i_e,j} \cdot \langle \iota_{b,j} \rangle$ • P_b and P_{1-b} compute $\gamma_{a,e,j} - \tau_{a,i_e,j}$ and $\gamma_{b,e,j} - \tau_{b,i_e,j}$. If either are $\neq 0$, output abort. 5. P_b and P_{1-b} combine shares from input wires indices i_e to return authenticated inverse shuffle tuples: • $[\langle \vec{a}_j \rangle_b, \langle \vec{b}_j \rangle_b]_j^n \leftarrow [[t_{a,i_e,j} - \langle r_{a,i_e,j} \rangle]_{i_e}, [t_{b,i_e,j} - \langle r_{b,i_e,j} \rangle]_{i_e}]_j^n$ • $[\langle \vec{a}_j \rangle_{1-b}, \langle \vec{b}_j \rangle_{1-b}]_j^n \leftarrow [[-\langle r_{a,i_e,j} \rangle_{1-b}]_{i_e}, [-\langle r_{b,i_e,j} \rangle_{1-b}]_{i_e}]_j^n$

Figure 24: Subprotocol for generating inverse authenticated shuffle tuples in the $(F_{\text{SPDZPrep}}, F_{\text{Comm}})$ -hybrid.

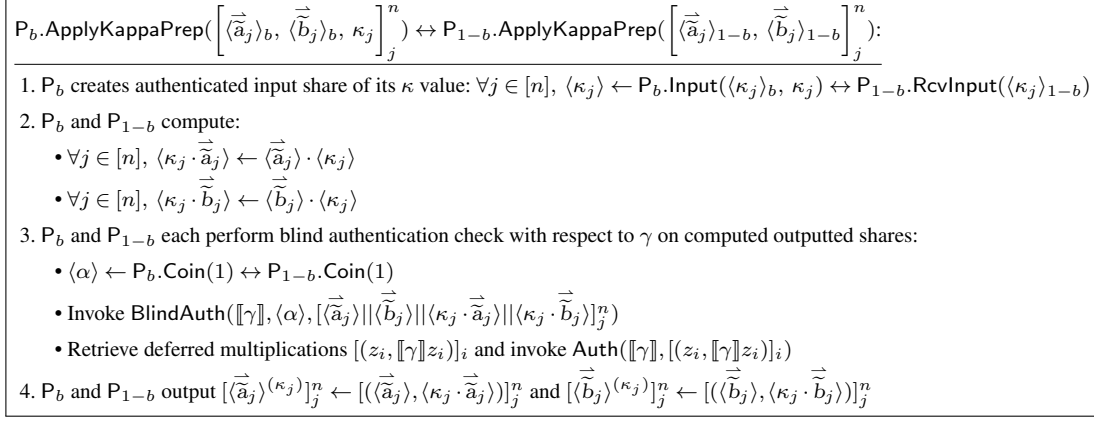


Figure 25: Subprotocol for applying and auth-checking κ -authenticated shares to shuffle tuples in the $(F_{\text{SPDZPrep}}, F_{\text{Comm}})$ -hybrid.

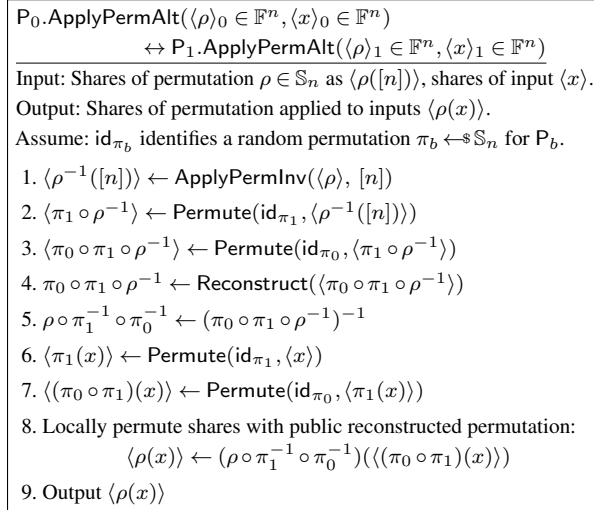


Figure 26: Alternate arithmetic permutation circuit description for ApplyPerm that avoids using the PermuteInv gate.